



RPCA Cyber Club
Faculty of Forensic Science
Royal Police Cadet Academy

Cybersecurity



Digital Forensics

Asst.Prof.Pol.Lt.Col.Dr. Wongyos Keardsri (Bank)

Lecturer (Level 3) | Faculty of Forensic Science | Royal Police Cadet Academy | Thailand
Email: wongyos@gmail.com | Line ID: wongyos | Phone/WhatsApp: +6689-599-3490
Facebook: facebook.com/wongyos

Copyright © 2026

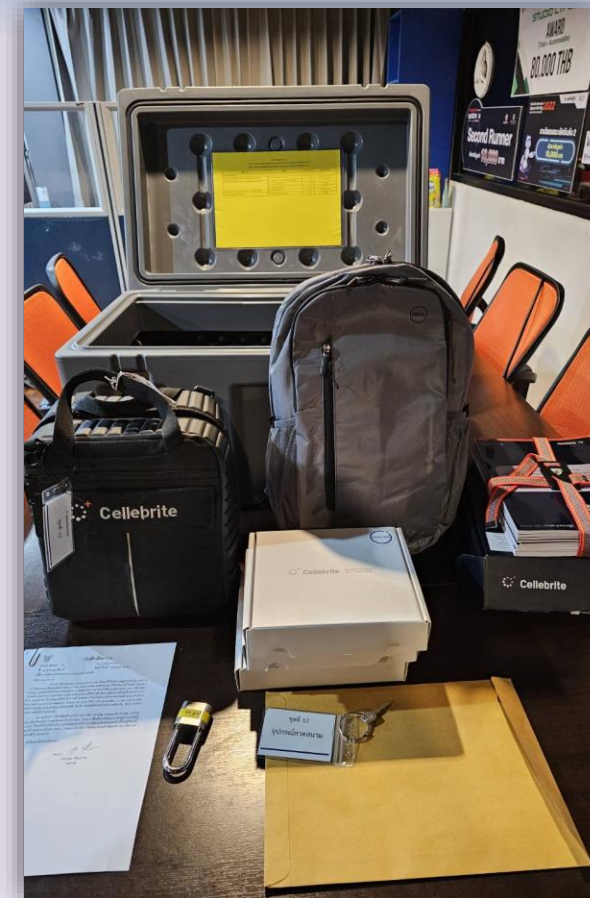
Digital Forensics Overviews





Definition of Digital Forensics

- **Digital Forensics** is a field within **cybersecurity** and **forensic science** that focuses on the **identification**, **preservation**, **analysis**, and **presentation** of **digital evidence** in a legally acceptable manner
- It's essential for investigating **cybercrimes** and criminal activities involving **digital devices**
- The term Digital Forensics was originally used as a synonym for **computer forensics** but has expanded to cover investigation of **all devices capable of storing digital data**





Goals of Digital Forensics

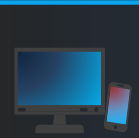




Goals of Digital Forensics (Cont)



Digital Forensics under Law Enforcement in the Suspect Area (Crime Scene)



Process of Digital Forensics

Process 2
Collection

Process 4
Analysis



CTF focuses on
the **analysis**
process

Digital Forensics Processes with **Chain of Custody**

Process 1
Identification

Process 3
Preservation

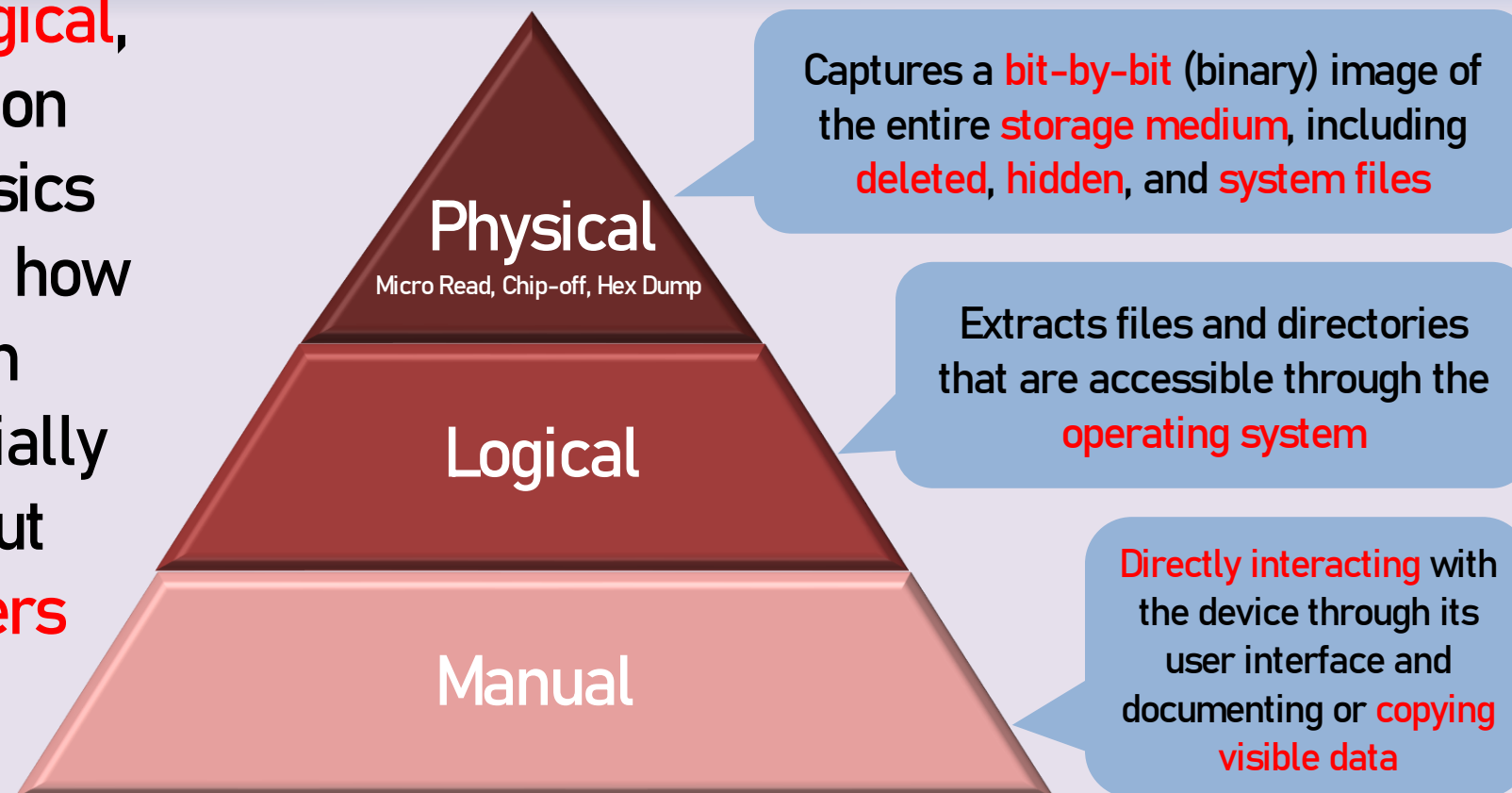
Process 5
Presentation

Keardsri, W. (2023). Digital Forensics in Thailand. *Journal of Criminology and Forensic Science*, 9(1), 228–247.



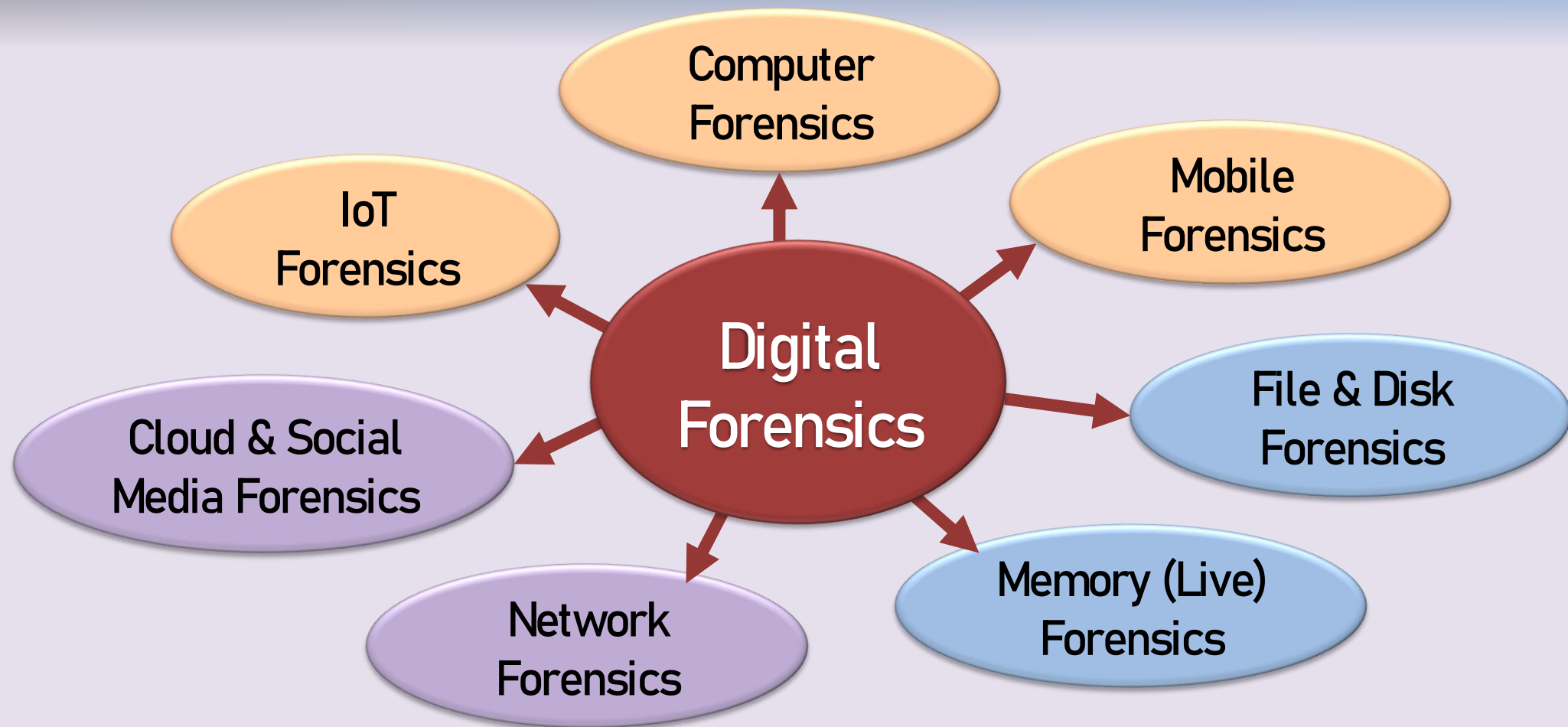
Levels of Digital Forensics

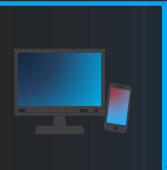
- There are **Manual**, **Logical**, and **Physical** acquisition levels in digital forensics
- These **levels** describe how **data is extracted** from digital devices, especially in **mobile forensics**, but also apply to **computers** and other systems





Branches of Digital Forensics





Branches of Digital Forensics (Cont)

■ Computer Forensics:

- Analyzing **computers** and **storage media** to uncover evidence
- Recovering **deleted files**, analyzing **file systems**, **logs**, and **application data**

■ Mobile Forensics:

- Extracting data from **smartphones**, **tablets**, and **SIM cards**
- Includes **chat logs**, **call records**, **location data**, **photos**, and **app** usage

■ IoT Forensics (Internet of Things Forensics):

- Involves collecting and analyzing data from interconnected **smart devices** such as **smart TVs**, **wearables** (e.g., smartwatches), **smart home** systems (e.g., Alexa, Google Home), and connected **vehicles**



Branches of Digital Forensics (Cont)

■ File Forensics:

- Focuses on the **identification** (file **headers/footers**, **magic numbers**), **recovery**, **carving**, **examination** (**contents** of file), and **analysis** of **files** from digital storage media

■ Disk Forensics:

- Involves the **acquisition** (**disk imaging**), **preservation**, and detailed **examination** of digital storage media (e.g., **hard drives**, **SSDs**, **USB drives**) to uncover digital evidence, including **deleted files**, **system artifacts**, and **hidden data**

■ Memory Forensics:

- Analyzing a system's **RAM** to extract information about **running processes**, **malware**, and **user activity**
- Useful in **live system** investigations



Branches of Digital Forensics (Cont)

■ Network Forensics:

- Monitoring and analyzing **network traffic** to detect intrusions or trace attacks
- Involves **packet capture**, **firewall logs**, and **intrusion detection** systems

■ Cloud and Social Media Forensics:

- Investigating data stored in **cloud services** (e.g., AWS, Google Drive)
- Requires cooperation with **service providers** and understanding virtual environments
- Investigating communications and interactions on **email**, **Facebook**, **Instagram**, etc.
- Often critical in **harassment**, **fraud**, and **exploitation** cases

We will focus on **File Forensics**, **Disk Forensics**, **Memory Forensics**, and **Network Forensics**



File Forensics



2



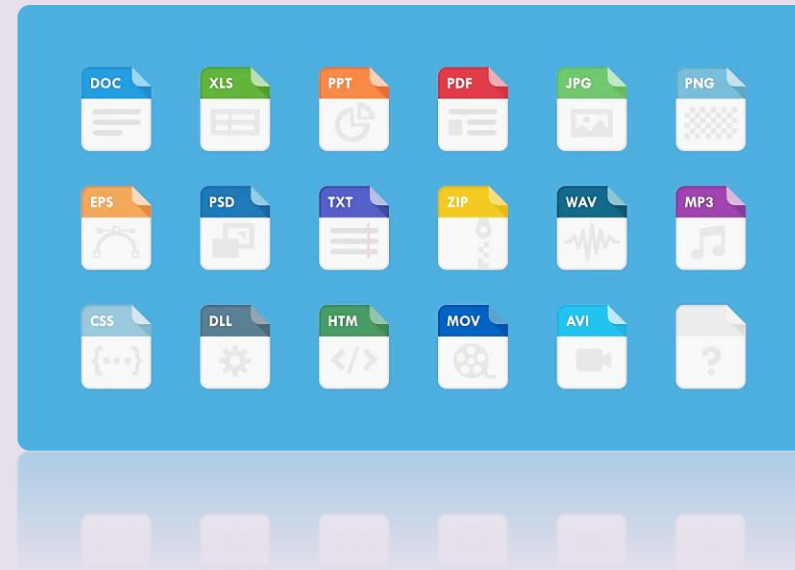
File Forensics Overviews

- **File Forensics** is a branch of **Digital Forensics** that focuses on the **identification, recovery, examination, and interpretation** of **files** and associated metadata to uncover digital evidence
- It plays a vital role in investigating **cybercrimes**, data breaches, and policy violations
- **Core Components:**
 - **File Identification** - Determine type, format, and origin using headers and signatures
 - **Metadata Extraction** - Analyze file properties: timestamps, user data, geolocation
 - **File Recovery** - Restore files using file carving
 - **Content Analysis** - Search file content for keywords, scripts, or embedded data
 - **Detection of Hidden Data** - Find steganography, ADS, or encrypted segments



File Types and Extensions

- There are many common **file types** and **extensions** that can be found in digital **file forensics**
 - **Text** .txt, .dat, .log
 - **Documents** .docx, .xlsx, .pptx, .pdf, .ps
 - **Images** .jpg, .jpeg, .png, .gif, .bmp
 - **Video** .mp4, .mov, .avi, .wmv, .flv
 - **Audio** .mp3, .acc, .ogg, .wav, .wma
 - **Executables** .exe, .dll, .elf (no extension), .apk
 - **Scripts/Code** .js, .py, .bat, .java, .c, .cpp, .html, .php
 - **Archives** .zip, .rar, .tar, .7z, .alz





File Signatures

- **File signature** (also called a **magic number**, **magic byte**) is a **unique** hexadecimal sequence of bytes at the **beginning of a file** (the **header**) used to identify the **file type**
- **File signature** (for some file type) also includes a file **trailer** (or **footer**) that is a specific hexadecimal sequence of bytes that marks the **end of a file**
- This is crucial in forensics because malicious actors often change **file extensions** to **hide content** or **avoid detection**
- Each file will have a different **file signature** (**header** and **trailer**) based on the **file type**



File Signatures (Cont)

JPG header



RPCACyberClub.jpg

RPCACyberClub.jpg																	Decoded text
Offset (h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	
00000000	FF	D8	FF	E0						6	00	01	02	00	00	01	ÿøÿà...JFIF.....
00000010			00	00						8	6F	74	6F	73	68	6F	ÿí.æPhotosho
00000020	70	20	33	2E						D	04	04	00	00	00	00	p 3.0.8BIM.....
00000030	00	80	1C	02	28	00	62	46	42	4D	44	30	31	30	30	30	.€..(.bFBMD01000
00000040	61	61	32	30	31	30	30	30	30	65	37	31	63	30	30	30	aa2010000e71c000
00000050	30	32	64	33	37	30	30	30	30	37	34	33	63	30	30	30	02d370000743c000
00000060	30	35	36	33	65	30	30	30	30	66	61	35	64	30	30	30	0563e0000fa5d000
00000070	30	32	66	37	65	30	30	30	30	32	38	38	61	30	30	30	02f7e0000288a000
00000080	30	62	37	38	66	30	30	30	30	37	62	39	32	30	30	30	0b78f00007b92000
00000090	30	33	32	63	31	30	30	30	30	1C	02	67	00	14	33	41	032c10000..g..3A
000000A0	57	42	43	7A	37	72	39	78	52	5F	52	44	75	7A	72	4F	WBCz7r9xR_RDuzrO
000000B0	71	4D	FF	DB	00	43	00	09	06	07	08	07	06	09	08	08	qMÿÛ.C.....
000000C0	08	0A	0A	09	0B	0E	17	0F	0E	0D	0D	0E	1C	14	15	11	
000000D0	17	22	1E	23	23	21	1E	20	20	25	2A	35	2D	25	27	32	..".##!. %*5-%'2
000000E0	28	20	20	2E	3F	2F	32	37	39	3C	3C	3C	24	2D	42	46	(.?/279<<<\$-BF
000000F0	41	3A	46	35	3B	3C	39	FF	DB	00	43	01	0A	0A	0A	0E	A:F5;<9ÿÛ.C.....
00000100	0C	0E	1B	0F	0F	1B	39	26	20	26	39	39	39	39	39	39	9& &999999
00000110	39	39	39	39	39	39	39	39	39	39	39	39	39	39	39	39	9999999999999999

JPG file type



File Signatures (Cont)



RPCACyberClub.jpg

JPG trailer

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
0000C020	EE	FA	B7	95	15	1E	11	B3	15	1F	E2	63	C8	8F	33	10	îú·•....³...âcÈ.3.
0000C030	29	DD	34	B8	8D	C9	4A	C0	F7	59	0D	5C	BA	1D	F4	77	)Ý4, .ÉJÀ÷Y.\°.ôw
0000C040	24	CA	17	44	B8	F1	AC	39	15	25	DD	56	23	BC	A2	45	\$Ê.D, ñ→9.%ÝV#¼¢E
0000C050	EE	45	32	18	E8	8B	3B	AA	16	12	83	0F	11	C8	DC	5A	îE2.è<;ª...f..ÈÛZ
0000C060	9A	E4	91	4A	E8	17	5E	14	AC	AB	65	D0	D4	CB	A9	DF	šä`Jè.^.→«eĐÔË©ß
0000C070	4E	61	A5	C4	6F	5A	32	9F	E8	A2	A3	6E	2A	3C	A2	C4	Na¥Äoz2ÿè¢£n*<¢Ä
0000C080	5F	5A	CC	51	AA	CE	83	04	DC	D6	42	A8	6D	CD	F7	3D	_zîQªîf.ÜÖB`mí÷=
0000C090	EB	F1	F0	B2	AB	64	B7	37	CC	F7	2F	C2	B3	1E	6A	B3	ěñð²«d·7î÷/Â³.j³
0000C0A0	A1	80	6E	28	30	97	D7	D1	C5	47	9D	1F	E1	E3	6A	3C	¡€n(0-×ÑĀG...ääj<
0000C0B0	98	9B	50	07	44	64	86	E7	0A	F9	A0	70	FD	CF	75	22	~>P.Dd†ç.ù pýİu"
0000C0C0	75	56	59	6E	F1	A8	8B	7A	28	FF	00	1F	15	1E	4C	54	uVYnñ`<z (ÿ....LT
0000C0D0	6D	C5	46	DE	41	43	B3	15	1E	4C	54	7F	A9	8F	43	1F	mÅFßAC³...LT.©.C.
0000C0E0	E5	E2	A3	C6	2A	3C	98	A8	DB	8A	8A	8A	8A	8A	8D	B8	ââ£Æ*<~`Ûšššššš.,
0000C0F0	A8	F2	63	FC	64	7A	58	A8	F3	E3	D2	C7	F8	68	DB	8F	"òcüdzX"óãðçøhÛ.
0000C100	41	1E	11	51	51	E8	A3	6E	3F	FA	6C	7A	28	A8	A8	F2	A...QQè£n?úlz (" "ò
0000C110	62	A3	C2	36	A3	FC	E4	7A	C8	FF	00	49	1B	51	E1	1E	b£Â6£üäzÈÿ.I.Qá.
0000C120	44	6D	47	FA	B8	F3	A3	FD	A4	54	78	45	47	FE	67	3F	DmGú, ó£ýªTxEGpg?
0000C130	FF	D9															ÿÛ



File Signatures (Cont)

- File signature table of popular file type to identify the origin of a file

File Type	Extension(s)	File Header	File Trailer	Markers	Notes
Microsoft Word	.docx	50 4B 03 04 14 00 06 00	50 4B 05 06	PK...	ZIP-based format, use content types to verify, trailer starts at 18 bytes before the end of the file
Microsoft Excel	.xlsx				
Microsoft PowerPoint	.pptx				
PDF Document	.pdf	25 50 44 46	25 25 45 4F 46	%PDF / %%EOF	—
JPEG Image	.jpg, .jpeg	FF D8	FF D9	ÿØÿà / ÿÙ	—
PNG Image	.png	89 50 4E 47 0D 0A 1A 0A	49 45 4E 44 AE 42 60 82	%PNG.... / IEND®B` ,...	—



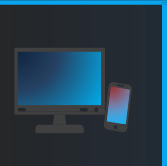
File Signatures (Cont)

File Type	Extension(s)	File Header	File Trailer	Markers	Notes
GIF Image	.gif	47 49 46 38 37 61 or 47 49 46 38 39 61	00 3B	GIF87a or GIF89a / .;	—
Bitmap Image	.bmp	42 4D	—	BM	—
MP4 Video	.mp4	[4 byte offset] 66 74 79 70 4D 53 4E 56	—	ftypMSNV	Offset is the byte position where the header starts
QuickTime Video	.mov	[4 byte offset] 66 74 79 70 71 74 20 20	—	ftypqt	Apple QuickTime format
AVI Video	.avi	52 49 46 46 xx xx xx xx 41 56 49 20 4C 49 53 54	—	RIFF.... AVI LIST	Where xx xx xx xx is the file size (little endian)



File Signatures (Cont)

File Type	Extension(s)	File Header	File Trailer	Markers	Notes
Windows Media Video	.wmv	30 26 B2 75 8E 66 CF 11	—	—	ASF format
MP3 Audio	.mp3	49 44 33	—	ID3	—
WAV Audio	.wav	52 49 46 46 xx xx xx xx 57 41 56 45 66 6D 74 20	—	RIFF.... WAVEfmt	RIFF format
Windows Executable	.exe	4D 5A	—	MZ	PE header follows MZ
Linux Executable	.elf	7F 45 4C 46	—	.ELF	ELF file is no extension file
ZIP Archive	.zip	50 4B 03 04	—	PK...	—
RAR Archive	.rar	52 61 72 21 1A 07 00	—	Rar!...	—



File Signatures (Cont)

■ File signature table from Gary Craig Kessler (GCK)

GCK'S FILE SIGNATURES TABLE

This table of file signatures (aka "magic numbers") is a continuing work from *A Practitioner's Guide* by T. Sammes & B. Jenkinson (Springer, 2004). Queries can be sent to Gary Kessler at gck@garykessler.net.

This list is not exhaustive although I add new files as I find them. However, the file type does not always have the given magic number extensions.

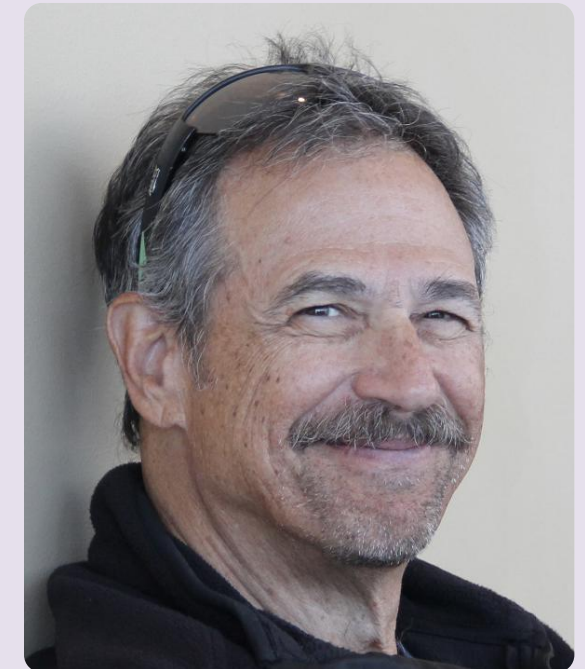
Some other useful information:

- My [software utility](#) page contains a custom signature file based on a JSON file of signatures.
- Tim Coakley's [Filesig.co.uk](#) site, with Filesig Manager and Filesig Explorer.
- Marco Pontello's [TrID - File Identifier](#) utility designed to identify file types.
- The National Archives' [PRONOM](#) site provides on-line information on file formats and [Object Identification](#) software.
- Additional details on graphics file formats can be found at [The Graphics File Formats](#) site.
- Additional details on audio and video file formats can be found at [Audio and Video File Formats](#).

If you are using a Linux/MacOS/Unix system, you can use the [file](#) command.

And, one last and final item — if you are searching for network traffic signatures, see [Network Traffic Signatures](#).

Hex Signature	File Extension	ASCII Signature	File Description
	TGA	Truevision Targa Graphic file	
		Trailer:	
		54 52 55 45 56 49 53 49	TRUEVISI
		4F 4E 2D 58 46 49 4C 45	ON-XFILE
		2E 00	..
00		.	
	PIC	IBM Storyboard bitmap file	
	MOV	Apple QuickTime movie file	
	PIF	Windows Program Information File	
	SEA	Mac Stuffit Self-Extracting Archive	
	YTR	IRIS OCR data file	
00 00 00 00 00 00 00 00			
00 00 00 00 00 00 00 00			
	XXX	Compucon/Singer embroidery design file	



<https://www.garykessler.net/>

https://www.garykessler.net/library/file_sigs_GCK_latest.html





File Signatures (Cont)

■ GCK File Signature Table Powered by SEARCH

Select Search Field ▾	Enter Search Term	Prev	Page 1 of 25	Next	Show All
ASCII File Description	Header (HEX)	File Extension	File Class	Header Offset	Trailer (Hex)
MPEG-4 v1	00 00 00 14 66 74 79 70 69 73 6F 6D	MP4	Multimedia	0	(null)
Windows Disk Image	00 00 00 00 14 00 00 00	TBI	Windows	0	(null)
Bitcoin Core wallet.dat file	00 00 00 00 62 31 05 00 09 00 00 00 00 20 00 00 00 09 00 00 00 00 00 00	DAT	Finance	8	(null)
MPEG-4 video_1	00 00 00 18 66 74 79 70	3GP5 M4V MP4	Multimedia	0	(null)
Apple audio and video	00 00 00 20 66 74 79 70 4D 34 41	M4A	Multimedia	0	(null)
Lotus 1-2-3 (v1)	00 00 02 00 06 04 06 00	WK1	Spreadsheet	0	(null)
Netscape Navigator (v4) database	00 06 15 61 00 00 00 02 00 00 04 D2 00 00 10 00	DB	Network	0	(null)

<https://filesig.search.org/>



File Metadata

- **File metadata** refers to structured **information embedded** in or associated with a file that describes its **characteristics, history, and usage**
- In digital forensics, metadata is a critical source of evidence that can reveal **how, when, and by whom** a file was created, accessed, or modified
- Types of file metadata:
 - **File System Metadata** - Managed by the file system (e.g., NTFS, FAT32)
 - **Application Metadata** - Embedded by software (e.g., Microsoft Office, Photoshop)
 - **Embedded Metadata** - Encoded directly into file content (e.g., EXIF in images, ID3 in MP3s)
 - **System Metadata** - Managed by the OS (e.g., user access history, ownership, permissions)



File Metadata (Cont)

- Common **Metadata Fields**

Metadata Field	Description
Created Date	When the file was first created
Modified Date	Last time the content was changed
Accessed Date	Last time the file was opened
Author/Owner	Who created or owns the file
File Size	Size in bytes
File Path	Location in the file system



File Metadata (Cont)

Metadata Field	Description
Permissions	Read/write/execute access rights
MAC Times	Modified, Accessed, and Created timestamps
Version Info	Application version that created/edited the file
Camera Info (EXIF)	Device model, GPS location, shutter speed (for images)



File Forensics Tools

- There are many popular file forensics tools

Category	Purpose	Example Tools
File Analysis	Inspect file contents, headers, structure	HxD, Hex Fiend, file
File Carving	Recover deleted or fragmented files from unallocated space	foremost, Scalpel, Photorec
Metadata Extraction	Extract metadata from documents, images, and media	ExifTool, FTK Imager



File Forensics Tools (Cont)

Category	Purpose	Example Tools
File Type Identification	Identify file types via headers or magic bytes	TrID, file, binwalk
Hashing	Verify file integrity or find duplicates	md5sum, sha256sum, HashMyFiles
Content Searching	Search text or patterns in files	grep, strings, bulk_extractor
Timeline Construction	Correlate file events (creation, modification)	Autopsy, Plaso/Log2timeline



HxD

- **HxD** is a powerful, **free** hex editor, disk editor, and memory editor developed by **Maël Hörz** for **Windows** widely used in **digital forensics**, **reverse engineering**, and **file analysis**
- It allows investigators to directly **view** and **manipulate** the **binary structure** of files, making it an essential tool for identifying file headers, trailers, and patterns
- It can open files larger than **4 GiB** and open and edit the raw contents of **disk drives**, as well as display and edit the **memory** used by running processes





HxD (Cont)

mh-nexus

Blog

Articles

Programs

About

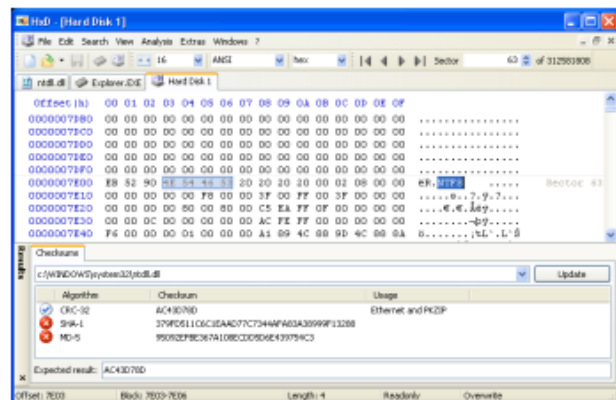
Contact

Forum

Donate



HxD - Freeware Hex Editor and Disk Editor



[Click to enlarge](#)

HxD is a carefully designed and fast hex editor which, additionally to raw disk editing and modifying of main memory (RAM), handles files of any size.

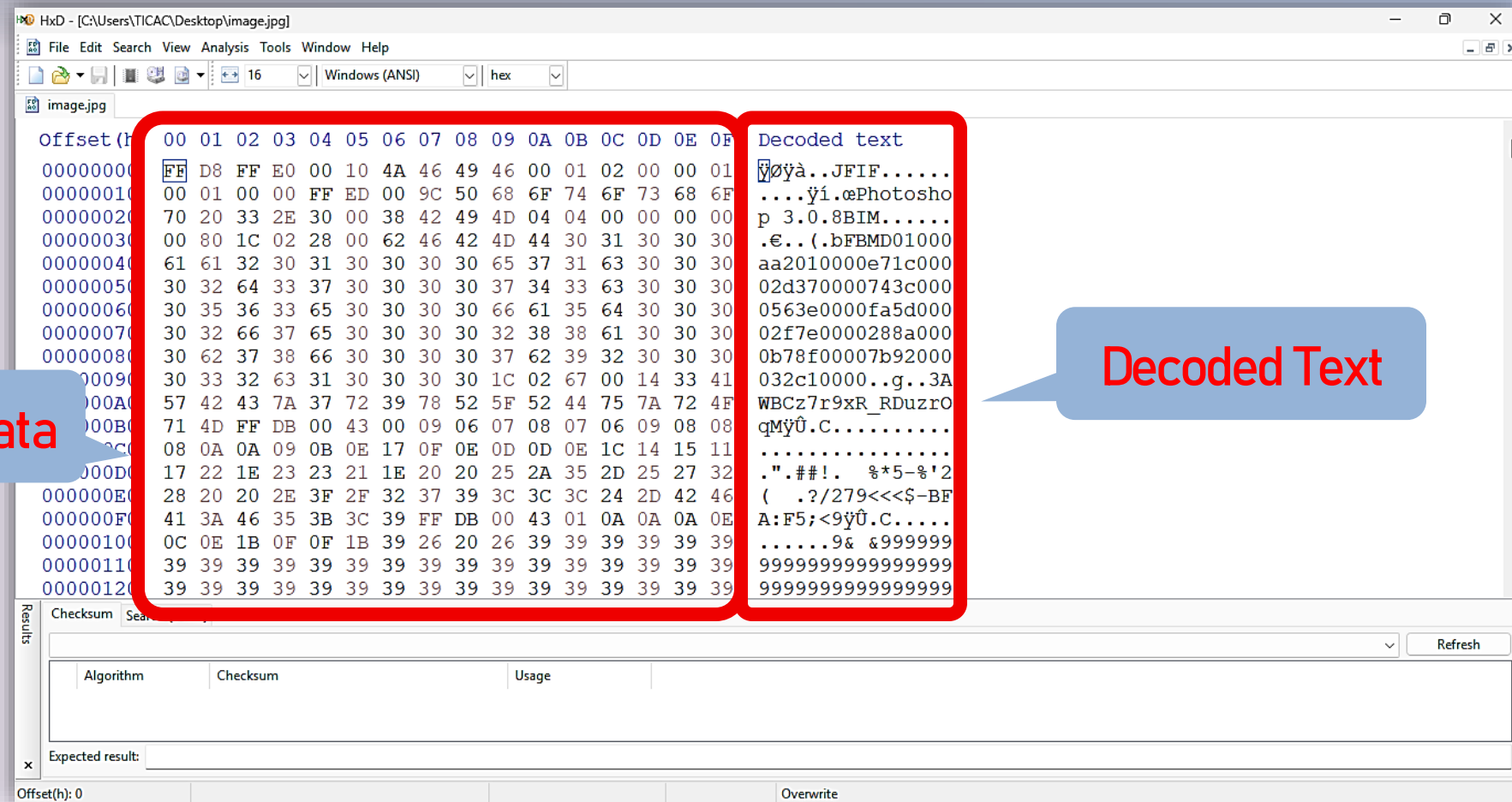
The easy to use interface offers features such as searching and replacing, exporting, checksums/digests, insertion of byte patterns, a file shredder, concatenation or splitting of files, statistics and much more.

Editing works like in a text editor with a focus on a simple and task-oriented operation, as such functions were streamlined to hide differences that are purely technical. For example, drives and memory are presented similar to a file and are shown as a whole, in contrast to a sector/region-limited view that cuts off data which potentially belongs together. Drives and memory can be edited the same way as a regular file including support for undo. In addition memory-sections define a foldable region and inaccessible sections are hidden by default.

<https://mh-nexus.de/en/hxd/>



HxD (Cont)





Hex Fiend

- **Hex Fiend** is a **free hex editor** for **macOS** (Most recommended)
- **Hex Fiend** is one of the most powerful and efficient hex editors available for macOS, especially suited for **file forensics**, **reverse engineering**, and raw **file analysis**
- There are many **features**, for example, fast and lightweight, can handle large files (multi-gigabyte), binary diff support, plugin support and scripting





Hex Fiend (Cont)



Hex Fiend

by ridiculous_fish

A fast and clever open source hex editor
for macOS.

<https://hexfiend.com/>

Features

- **Insert, delete, rearrange.** Hex Fiend does not limit you to in-place changes like some hex editors.
- **Work with huge files.** Hex Fiend can handle as big a file as you're able to create.



Hex Fiend (Cont)

lenna.jpg

0	FFD8FFE0	1004A46	49460001	01000001	00010000
20	FFDB0043	00030202	03020203	03030304	03030405
40	08050504	04050A07	0706080C	0A0C0C0B	0A0B0B0D
60	0E12100D	0E110E0B	0B101610	11131415	15150C0F
80	17181614	18121415	14FFDB00	43010304	04050405
100	09050509	140D0B0D	14141414	14141414	14141414
120	14141414	14141414	14141414	14141414	14141414
140	14141414	14141414	14141414	14141414	1414FFC0
160	00110802	00020003	01220002	11010311	01FFC400
180	1F000001	05010101	01010100	00000000	00000001
200	02030405	06070809	0A0BFFC4	00B51000	02010303
220	02040305	05040400	00017D01	02030004	11051221
240	31410613	51610703	71143281	91A10823	42B1C115
260	52D1F024	33627282	090A1617	18191A25	26272829
280	2A343536	3738393A	43444546	4748494A	53545556
300	5758595A	63646566	6768696A	73747576	7778797A
320	83848586	8788898A	92939495	96979899	9AA2A3A4
340	A5A6A7A8	A9AAB2B3	B4B5B6B7	B8B9BAC2	C3C4C5C6
360	C7C8C9CA	D2D3D4D5	D6D7D8D9	DAE1E2E3	E4E5E6E7
380	E8E9EAF1	F2F3F4F5	F6F7F8F9	FAFFC400	1F010003
400	01010101	01010101	01000000	00000001	02030405
420	06070809	0A0BFFC4	00B51100	02010204	04030407
440	05040400	01027700	01020311	04052131	06124151
460	07617111	02328108	144291A1	B1C10923	3352F015
480	6272D10A	62434F1	25F11718	191A2627	28292A35

Floats: big 88.877113
Signed Int: big 1118945557

4 bytes selected at offset 0x100 out of 67.6 kilobytes

lenna.jpg

Hex Text Find 99 8C D7 4F

Replace

Replace All Replace Replace & Find Previous Next

22428	CD6B28C6	7AC9233E	674D3699	91E21F89	BA8EA504	D1DCCD2B	1600062C	72181183	9EB9E3BF
22464	BD45A168	5E27D475	4169A35C	4D757F77	01DF1C72	6EF31403	CBF3D47B	F35CFF00	86341D53
22500	E21F886D	F47B1B79	2E6EE660	81A3524A	81EA7E98	1CD7DA1F	08BE0FE8	5F012D2D	AE9A25D7
22536	FC733FCA	B181B96D	C9CE063D	7D68CDC4	E228E123	EF2BB7B2	58BFF81E	66B7A959	F2C09BF6
22572	65F85F7D	F0ABC317	37DE3072	23D42449	A1D250E5	8BF3CB7F	9FCEBE80	D27C3FA9	F8B634B9
22608	BB22CEC0	7305BC63	01467A81	FD6B33C0	DE0FBA2D	2EB3E299	3ED3A848	4B081BEE	459E831E
22644	B5D95BEA	AEADE5C2	58A0C819	F4CFAD7C	562310F1	151CA6AE	FCB65FE6	3A937493	54DDDF7F
22680	F2FF0033	421D2ED7	4DB5F2A3	89463192	2A49E5DE	76062147	39F5A89E	E32A147D	EA824883
22716	B942F38E	BE82B9B6	7A9E6DDC	B565D6C7	9781D875	F4EF50A6	65704F18	FCE9BE79	207F191E
22752	BF0052A0	27AE79EA	066B4573	192B17E1	C05CFA8A	91A408A4	F27039EE	6AB463EE	E7903F0A
22788	4D575DB2	D02C8DC6	A3730595	B2FDE965	60A3DBAD	5BD11095	DD91AB02	9F210F52	E38EFC56
22824	57897C55	A2783ECD	AE357D46	0B18C018	F35C64FD	07535E4F	E26FDA29	75880D42	DBC271B2
22860	C80930FD	BAE13017	D191788F	73F957CA	BE208BD4	F578FBAB	8D56E26B	DBD76CBB	492166CE
22896	7F955FB3	A9D55BD7	FC8F5F09	96BC436E	6EC974EA	7D2BE34F	DAF34DB2	325AF876	D9EE89E3
22932	ED0E36E7	D719E82B	C03C65F1	3FC45E2F	998CD74F	0231F992	2273823A	16EFFF00	D7ACCD17
22968	C0FABEB5	3C4D6F65	20463C16	040AF55F	0EFC1157	21EF5B2F	91955FE2	A9946952	D66F5F3F
23004	F23E968E	1A861BE0	4795F853	C19A86B1	736F1DB4	0EC1DC12	7A11EBCD	7D7BF06F	E17DA783
23040	2D567BC0	8264CB3C	D2E00403	AF3FD6AD	F83FC29A	2F84F496	BA9922B6	B4B58CBC	B7129012
23076	35032493	ED5F14FE	D49FB4AD	DFC50F13	49A17867	509ADFCE	364768D8	4A1BC93B	C8DEABE8
23112	0F07BF17	4BDA6327	ECE8EDD5	B3CFC4D7	9566E947	45D48DFB	70FC7C8F	E2A78AD7	C3BA26A2
23148	93786F43	7DBF21CA	DD5C1043	38EC5570	147FBD6C	7B7CE369	6E638543	13B41C8D	FD893496
23184	B63B9F7E	DDC01DC5	B1FAD4B7	798B60C9	5C8E83D3	B63F1AFA	BA34A342	9AA5039E	9D254D24
23220	88E58BE5	40DB8465	CF2A339C	F4C55863	F62B5450	4827182C	7000F7FD	2934F57B	99582939
23256	C83B76B1	23F2AB77	9A7FDB4C	2B1FEF03	36C01473	D718C1C1	FE9C8E6B	78C79A56	29CB923A
23292	9E85FB3E	F860F887	5AFB4CB1	E638F041	3CE4E7FF	00D5D6BE	B4722085	20894055	1818AF3C

4 bytes selected at offset 0x59A4 out of 67.6 kilobytes



xxd

- The **xxd** is a **command-line tool** used in **digital forensics** to create **hexadecimal (hex) dumps** of files and to reverse hex dumps back into binary
- It's especially useful when analyzing **file headers**, **signatures**, and **content structure** in Linux/macOS environments
- The **xxd** command usage in **Kali Linux**

```
xxd [options] [input_file] [output_file]
```

```
#Hex Dump of a File
```

```
xxd myfile.jpg
```

```
#Hex Dump with Specific Line Width (e.g., 16 bytes)
```

```
xxd -c 16 myfile.png
```



xxd (Cont)

```
(kali㉿kali)-[~]
$ xxd finding_the_flag.jpg
00000000: ffd8 ffe0 0010 4a46 4946 0001 0101 0060 .....JFIF. ...
00000010: 0060 0000 ffdb 0043 0003 0202 0302 0203 .. ...C.....
00000020: 0303 0304 0303 0405 0805 0504 0405 0a07 .....
00000030: 0706 080c 0a0c 0c0b 0a0b 0b0d 0e12 100d .....
00000040: 0e11 0e0b 0b10 1610 1113 1415 1515 0c0f .....
00000050: 1718 1614 1812 1415 14ff db00 4301 0304 .....C...
00000060: 0405 0405 0905 0509 140d 0b0d 1414 1414 .....
00000070: 1414 1414 1414 1414 1414 1414 1414 1414 .....
00000080: 1414 1414 1414 1414 1414 1414 1414 1414 .....
00000090: 1414 1414 1414 1414 1414 1414 1414 ffc0 .....
000000a0: 0011 0802 d005 0003 0122 0002 1101 0311 .....
000000b0: 01ff c400 1f00 0001 0501 0101 0101 0100 .....
000000c0: 0000 0000 0000 0001 0203 0405 0607 0809 .....
000000d0: 0a0b ffc4 00b5 1000 0201 0303 0204 0305 .....
000000e0: 0504 0400 0001 7d01 0203 0004 1105 1221 ... ..} ...!
000000f0: 3141 0613 5161 0722 7114 3281 91a1 0823 1A..Qa."q.2....#
00000100: 42b1 c115 52d1 f024 3362 7282 090a 1617 B...R..$3br...
00000110: 1819 1a25 2627 2829 2a34 3536 3738 393a ...%&'()*456789:
00000120: 4344 4546 4748 494a 5354 5556 5758 595a CDEFGHIJSTUVWXYZ
00000130: 6364 6566 6768 696a 7374 7576 7778 797a cdefghijstuvwxyz
00000140: 8384 8586 8788 898a 9293 9495 9697 9899 .....
00000150: 9aa2 a3a4 a5a6 a7a8 a9aa b2b3 b4b5 b6b7 .....
00000160: b8b9 bac2 c3c4 c5c6 c7c8 c9ca d2d3 d4d5 .....
```

```
(kali㉿kali)-[~]
$ xxd ooovrrr_server
00000000: 7f45 4c46 0201 0100 0000 0000 0000 0000 .ELF...
00000010: 0300 3e00 0100 0000 9010 0000 0000 0000 ..>...
00000020: 4000 0000 0000 0000 f037 0000 0000 0000 @.....7
00000030: 0000 0000 4000 3800 0d00 4000 1f00 1e00 ....@.8..@
00000040: 0600 0000 0400 0000 4000 0000 0000 0000 . ... @
00000050: 4000 0000 0000 0000 4000 0000 0000 0000 @.....@
00000060: d802 0000 0000 0000 d802 0000 0000 0000 ..
00000070: 0800 0000 0000 0000 0300 0000 0400 0000 .....
00000080: 1803 0000 0000 0000 1803 0000 0000 0000 ..
00000090: 1803 0000 0000 0000 1c00 0000 0000 0000 ..
000000a0: 1c00 0000 0000 0000 0100 0000 0000 0000 .....
000000b0: 0100 0000 0400 0000 0000 0000 0000 0000 . ...
000000c0: 0000 0000 0000 0000 0000 0000 0000 0000 .....
000000d0: d807 0000 0000 0000 d807 0000 0000 0000 ..
000000e0: 0010 0000 0000 0000 0100 0000 0500 0000 .....
000000f0: 0010 0000 0000 0000 0010 0000 0000 0000 .....
00000100: 0010 0000 0000 0000 e902 0000 0000 0000 .....
00000110: e902 0000 0000 0000 0010 0000 0000 0000 ..
00000120: 0100 0000 0400 0000 0020 0000 0000 0000 . ...
00000130: 0020 0000 0000 0000 0020 0000 0000 0000 .
00000140: 5c01 0000 0000 0000 5c01 0000 0000 0000 \.....\
00000150: 0010 0000 0000 0000 0100 0000 0600 0000 .....
00000160: d02d 0000 0000 0000 d03d 0000 0000 0000 .-.....=
```




file

- The **file** command is a standard Unix/Linux utility used to **identify** the **type of a file** based on its **magic number** (i.e., the header bytes) — not its extension
- It's one of the fastest and most reliable tools in digital forensics for verifying the **actual content of a file**
- The **file** command usage

```
file [options] <filename>
```



file (Cont)

```
(kali㉿kali)-[~]
```

```
$ file image.jpg
```

```
image.jpg: JPEG image data, JFIF standard 1.02, aspect ratio, density 1×1, segment length 16, progressive, precision 8, 1000×1000, components 3
```

```
(kali㉿kali)-[~]
```

```
$ file -f list.txt
```

```
image.jpg:          JPEG image data, JFIF standard 1.02, aspect ratio, density 1×1, segment length 16, progressive, precision 8, 1000×1000, components 3
```

```
data.pdf:           PDF document, version 1.7, 1 page(s)
```

```
information.zip: Zip archive data, at least v2.0 to extract, compression method=deflate
```

```
(kali㉿kali)-[~]
```

```
$
```



TrID

- TrID is a **lightweight** yet powerful tool designed for **Windows** and **Linux** to identify the **file type** based on its **binary signature** (not just the extension)
- TrID is **free** for **personal use**, no commercial use
- **Download TrID** (<https://mark0.net/soft-trid-e.html>)

Marco Pontello's Home Page

- Home
- Old News
- Contacts Info
- Privacy
- Software
- Awards
- TrID
 - TrIDNet
 - Online TrID
 - TrIDBot
 - TrIDScan
 - File extension defs
 - PBTracer
 - Extra Redcode Kit
 - Screenshots

 Questa pagina in italiano

(Last updated: 25/05/25)



TrID - File Identifier

TrID is an utility designed to identify file types from their binary signatures. While there are similar utilities with hard coded logic, TrID has no fixed rules. Instead, it's extensible and can be trained to recognize new formats in a fast and automatic way.



TrID (Cont)

- **Offline TrID** file identifier (command line mode)

```
C:\Users\HP\Desktop>trid test.doc
```

```
TrID/32 - File Identifier v2.24 - (C) 2003-16 By M.Pontello  
Definitions found: 19380  
Analyzing...
```

```
Collecting data from file: test.doc
```

```
78.9% (.DOC) Microsoft Word document (30000/1/2)
```

```
21.0% (.) Generic OLE2 / Multistream Compound (8000/1)
```

```
C:\Users\HP\Desktop>
```



TrID (Cont)

- **Online TrID** file identifier (<https://mark0.net/onlinetrid.html>, **file size <= 10MB**)



Share 212

(Powered by [TrIDEngine/Py](#) v1.0, 19380 definitions)

Online TrID File Identifier

Identification results:

File size: 50KB

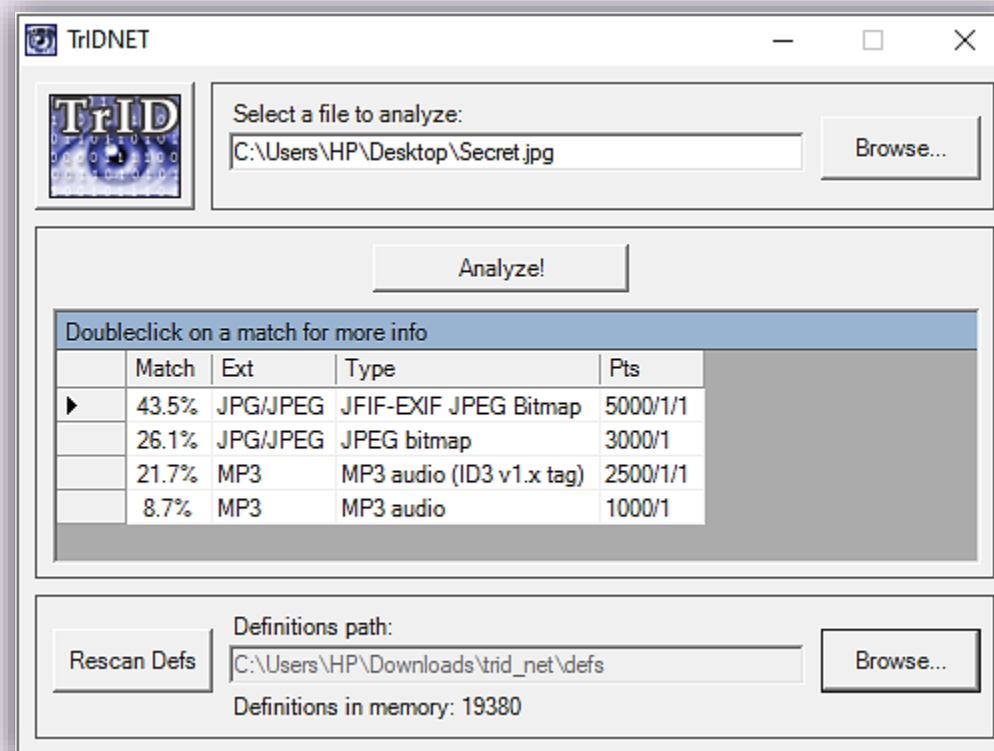
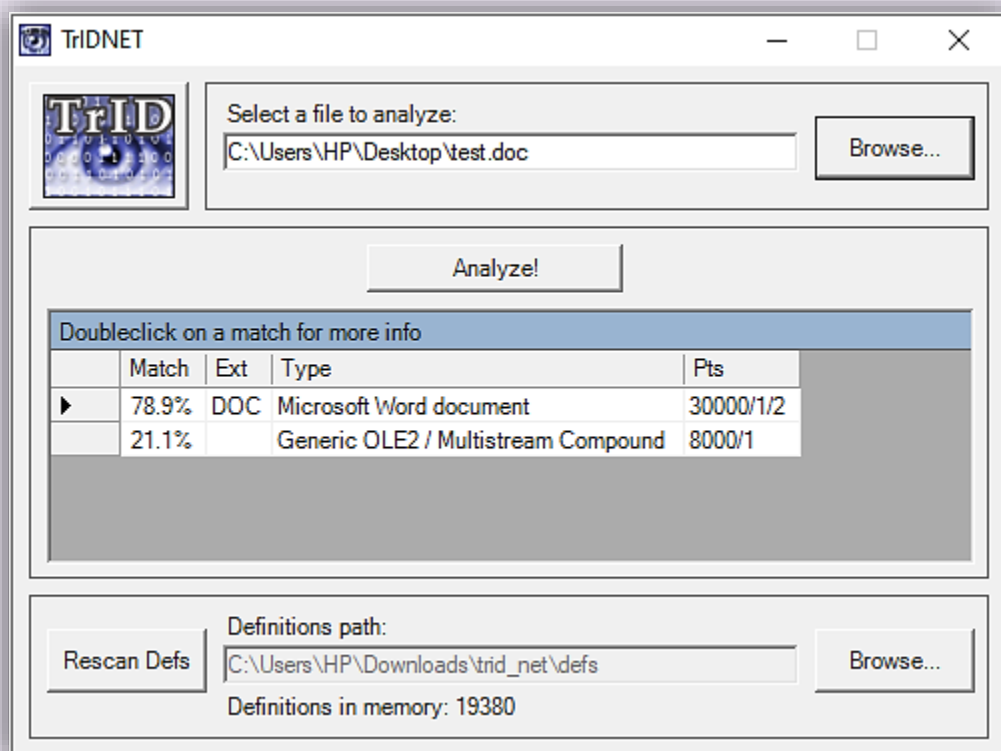
Match	Ext	File type	MIME type	Related URL	Def's author
78.95%	DOC	Microsoft Word document	application/msword	https://en.wikipedia.org/wiki/Microsoft_Word	Marco Pontello
21.05%		Generic OLE2 / Multistream Compound	application/x-ole-storage	https://en.wikipedia.org/wiki/Compound_File_Binary_Format	Marco Pontello

Used by various applications of the Microsoft Office suite, for example.



TrID (Cont)

- **TrIDNet** file identifier **GUI** (<https://mark0.net/soft-tridnet-e.html>)





Binwalk

- **Binwalk** is a powerful tool used in **digital forensics**, reverse engineering, and firmware analysis
- It **scans binary files** to identify **embedded file** signatures, compressed data, and executable code, making it ideal for **detecting** and **extracting** hidden files or contents from raw data
- The **binwalk** command usage

```
binwalk [options] <filename>
```

- The **binwalk** extract all **embedded files**

```
binwalk -e <filename>
```



Binwalk (Cont)

```
(kali㉿kali)-[~]  
$ binwalk information.zip
```

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	Zip archive data, at least v2.0 to extract, name: image.jpg
42838	0xA756	Zip archive data, at least v2.0 to extract, name: data.pdf
106737	0x1A0F1	End of Zip archive, footer length: 22

```
(kali㉿kali)-[~]  
$
```



Foremost

- **Foremost** is an open-source **forensic data recovery tool** that uses **file headers** (magic numbers) and footers (trailers) to **carve** and **recover** deleted or lost files from disk images, memory dumps, or raw binary data
- The **foremost** command usage

```
foremost -i <input_file> -o <output_directory>
```

```
#Recover only PDFs and PNGs
```

```
foremost -i dump.img -t pdf,png -o pdf_png_output/
```



Foremost (Cont)

```
(kali@kali)-[~]  
$ foremost information.zip  
Processing: information.zip  
|foundat=image.jpgux
```

*|

```
(kali@kali)-[~]  
$ ls -l ./output  
total 8  
-rw-rw-r-- 1 kali kali 670 May 28 04:39 audit.txt  
drwxrwxr-- 2 kali kali 4096 May 28 04:39 zip
```

```
(kali@kali)-[~]  
$
```



Strings

- The **strings** command is a lightweight but powerful tool in **digital forensics** used to **extract printable ASCII** (sometimes **Unicode**) text from **binary files**
- It's extremely useful for quickly identifying **hidden messages**, **file paths**, **URLs**, **commands**, **credentials**, or **metadata** embedded in files
- The **strings** command usage

```
strings [options] <filename>
```

```
#Basic usage
```

```
strings malware.exe
```

```
#Redirect to a File for Analysis
```

```
strings firmware.img > strings_output.txt
```




Grep

- The **grep** is a command-line utility used to **search for patterns in text**
- In **digital forensics**, it is invaluable for **filtering** through large volumes of extracted data, log files, or strings outputs to **find keywords**, IP addresses, URLs, usernames, passwords, or malicious indicators
- The **grep** command usage

```
grep [options] "pattern" <filename>
```

```
#Search for a Keyword in a File
```

```
grep "password" strings_output.txt
```

```
#Use Regular Expressions
```

```
grep -E "[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}" file.txt
```



Strings with Grep

```
(kali㉿kali)-[~]  
$ strings data.pdf | grep "PDF"  
%PDF-1.7  
/Producer (Microsoft: Print To PDF)  
  
(kali㉿kali)-[~]  
$ strings information.zip | grep "image"  
image.jpgux  
image.jpgux  
  
(kali㉿kali)-[~]  
$ strings information.zip | grep "data"  
data.pdfux  
data.pdfux
```



Bulk Extractor

- The **bulk_extractor** is a forensic tool (C++) that **scans disk images, files, or directories** and **extracts** useful **artifacts** such as **emails, URLs, credit card numbers**, and other **strings of interest** without parsing the file system
- It's especially useful in investigations involving **large datasets** or when the file system is **damaged** or **unavailable**
- The **bulk_extractor** command usage

```
bulk_extractor -o <directory> -x all <filename>
```

#Basic usage

```
bulk_extractor -o output_directory -x all image.dd
```



Bulk Extractor (Cont)

```
(kali㉿kali)-[~]  
$ bulk_extractor -o bulk_data -x all information.zip  
bulk_extractor version: 2.1.1  
Input file: "information.zip"  
Output directory: "bulk_data"  
Disk Size: 106759  
Scanners:  
Threads: 4  
going multi-threaded... ( 4 )  
bulk_extractor      Wed May 28 05:52:09 2025  
  
available_memory: 1219837952  
bytes_queued: 0  
depth0_bytes_queued: 0  
depth0_sbufs_queued: 0
```



ExifTool

- **ExifTool** is a powerful command-line utility used in digital forensics to read, write, and edit **metadata** in a wide variety of file types
- It is especially valuable for analyzing images, documents, audio, and video files, as it can reveal **hidden information** such as **timestamps**, **GPS coordinates**, **camera models**, **software versions**, and more
- The **exiftool** command usage

```
exiftool [options] <filename>
```

```
#Basic usage
```

```
exiftool image.jpg
```



ExifTool (Cont)

```
(kali㉿kali)-[~]  
$ exiftool finding_the_flag.jpg  
ExifTool Version Number      : 12.76  
File Name                    : finding_the_flag.jpg  
Directory                   : .  
File Size                    : 709 kB  
File Modification Date/Time   : 2024:05:11 03:37:23-04:00  
File Access Date/Time        : 2024:08:04 07:40:05-04:00  
File Inode Change Date/Time   : 2024:08:04 07:40:05-04:00  
File Permissions              : -rw-----  
File Type                    : JPEG  
File Type Extension          : jpg  
MIME Type                    : image/jpeg  
JFIF Version                 : 1.01  
Resolution Unit               : inches  
X Resolution                  : 96  
Y Resolution                  : 96  
Image Width                   : 1280  
Image Height                  : 720  
Encoding Process              : Baseline DCT, Huffman coding
```




Aperi'Solve

- **Aperi'Solve** is a powerful **web-based CTF** (Capture The Flag) and **online forensic analysis tool** designed to help forensic investigators and cybersecurity learners analyze and reverse-engineer image and **document files**
- It performs **automated static analysis**, revealing hidden content and metadata often used in **steganography**, CTFs, and digital forensics

What is this ?

Aperi'Solve is an online platform which performs layer analysis on image. The platform also uses zsteg, steghide, outguess, exiftool, binwalk, foremost and strings for deeper steganography analysis. The platform supports the following images format: .png, .jpg, .gif, .bmp, .jpeg, .jfif, .jpe, .tiff...



Select a file or drag here

PLEASE SELECT AN IMAGE

SUBMIT

<https://www.aperisolve.com/>



Aperi'Solve (Cont)

Informations



[+] Name
[+] Size
[+] First
[+] Last
[+] Uplo
[+] Comm

Steghide

steghide: could not extract any data with that passphrase!

Outguess

Reading /app/uploads/aa1de0f3c75cc
Extracting usable bits: 161543 bit
Steg retrieve: seed: 24595, len: 4
Extracted datalen is too long: 425

ExifTool

Foremost

DOWNLOAD FILES

Strings

JFIF
ICC_PROFILE
lcms
mnrRGB XYZ
acspMSFT
sawscrl
-hand
=@t,

desc

EXIFT00L

ExifToolVersion

12.16



Fcrackzip

- **Fcrackzip** is a free, fast **command-line tool** used in digital forensics and ethical hacking to **crack password-protected .zip files** using **brute-force** or **dictionary attacks**
- It's widely used by forensic investigators and **CTF players**
- The **fcrackzip** command usage

```
fcrackzip [options] filename.zip
```

```
#Dictionary Attack
```

```
fcrackzip -v -u -D -p ./rockyou.txt ./file.zip
```

```
#Custom Character Set (alphanumeric)
```

```
fcrackzip -b -c a1 -l 4-6 protected.zip      #a1 = lowercase + digits
```



Fcrackzip (Cont)

```
(kali㉿kali)-[~]  
$ fcrackzip -v -u -D -p rockyou.txt malware.zip  
found file 'malware', (size cp/uc 2607/ 16240, flags 9, chk 6847)
```

PASSWORD FOUND!!!!: pw = number1

```
(kali㉿kali)-[~]  
$ █
```



cRARk

- **cRARk** (pronounced “**crark**”) is a **command-line tool** specifically designed to **crack passwords** of **.rar** archives (RAR3 and RAR5), especially those encrypted using **AES**, which tools like **fcrackzip cannot handle** (using **GPU**)
- It is one of the few tools that supports **brute-force** and **dictionary attacks** for strongly encrypted RAR archives
- The **crack** command usage

```
crark -c -p:yourwordlist.txt yourfile.rar
```

#Dictionary Attack

```
crark -c -p:rockyou.txt secrets.rar
```



cRARk (Cont)

- **cRARk download** (<http://www.crark.net/>) – **Windows** and **Linux** support

The screenshot shows the cRARk website homepage. At the top, there is a navigation bar with links: cRARk home, Purpose, Manual, FAQ, Download, Support, History, and Other password recovery. Below the navigation bar, there is a section for "cRARk 5.5" with a "DOWNLOAD LATEST VERSION" button and a note "New version with Unicode support". To the right, there is a "News" section with a link "cRARk for 7-Zip released!". Below this, there are five main sections: "Fastest freeware utility to crack RAR password", "Download", "Manual", "Password Definition Language", "Professional RAR password recovery", and "cRARk for 7-Zip". On the left side, there is a list of milestones: 1996, cRARk 1.0 - just first RAR cracker; 2006, cRARk 3.0 - fastest recovery of RAR 3.0 passwords; 2009, cRARk 3.2 - world's first CUDA-aware RAR password recovery; 2011, cRARk 3.4 - world's first OpenCL-aware RAR password recovery; 2013, cRARk 5.0 - world's first RAR 5.0 password recovery. At the bottom, there is a section titled "cRARk purpose" with the text: "The primary goal of cRARk is to recover your forgotten password on RAR archives. Any Rar/WinRar versions 2.x-5.x are supported. Please bear in mind you have quite no chance to crack unknown password (longer than 6-7 symbols) if you have no additional info". To the right of this section, there are logos for "OpenCL" and "DESIGNED FOR NVIDIA CUDA".

cRARk 5.5

DOWNLOAD LATEST VERSION

New version with Unicode support

News

cRARk for 7-Zip released!

Fastest freeware utility to crack RAR password

Download

Manual

Password Definition Language

Professional RAR password recovery

cRARk for 7-Zip

1996, cRARk 1.0 - just first RAR cracker
2006, cRARk 3.0 - fastest recovery of RAR 3.0 passwords
2009, cRARk 3.2 - world's first CUDA-aware RAR password recovery
2011, cRARk 3.4 - world's first OpenCL-aware RAR password recovery
2013, cRARk 5.0 - world's first RAR 5.0 password recovery

cRARk purpose

The primary goal of cRARk is to

- ♦ **recover your forgotten password** on RAR archives.

Any Rar/WinRar versions 2.x-5.x are supported.

Please bear in mind you have quite no chance to crack unknown password (longer than 6-7 symbols) if you have no additional info

OpenCL

DESIGNED FOR NVIDIA CUDA



PDFCrack

- **PDFCrack** is a **command-line tool** used to **recover passwords** (primarily user or owner passwords) from **PDF** documents
- It's popular in **digital forensics**, penetration testing, and **CTFs** where **password-protected PDFs** need to be analyzed or opened
- The **pdfcrack** command usage

```
pdfcrack [options] filename.pdf
```

```
#Brute-force
```

```
pdfcrack -f secret.pdf -l 1 -m 4 -c abc123
```

```
#Dictionary Attack
```

```
pdfcrack -f confidential.pdf -w /usr/share/wordlists/rockyou.txt
```



PDFCrack (Cont)

```
(kali㉿kali)-[~]  
$ pdfcrack -f secret2025_protected.pdf -w rockyou.txt  
PDF version 1.7  
Security Handler: Standard  
V: 2  
R: 3  
P: -4  
Length: 128  
Encrypted Metadata: True  
FileID: 9b37b4eb264d2c4680aec162f23b7ea6  
U: 8cf252c4cdc5de1c7c4f6afb8b9b9a1928bf4e5e4e758a4164004e56fffa0108  
O: 340d3ae14a29da016670fd9b417521534b4a948da71ac30995648e42bca1ad60  
found user-password: 'inlove'
```

```
(kali㉿kali)-[~]  
$ █
```



Office2John

- Office2John is a Python script bundled with John the Ripper that extracts password hashes from Microsoft Office files (e.g., .doc, .docx, .xls, .xlsx, .pptx)
- These hashes can then be cracked using tools like John the Ripper or Hashcat
- It is a powerful forensic utility for investigators needing to access encrypted Office documents without altering the evidence
- Install John the Ripper Jumbo version (The office2john.py script is inside the run/ directory)

```
git clone https://github.com/openwall/john.git
cd john/run
```



Office2John (Cont)

- Extracts the **hash** and saves it to the **output text file** that contains the hash

```
python3 office2john.py protected.docx > office_hash.txt
```

- **Cracks the hash** with specifies a **dictionary file**

```
john office_hash.txt --wordlist=rockyou.txt
```

- **Tips**

- For stronger files (**Office 2013+**), cracking may take longer due to **PBKDF2** with **SHA-1** and **AES-256**
- You can combine it with **hashcat** if needed (requires hash mode 9600, etc.)
- Use **--format=office** in john if auto-detection fails



Office2John (Cont)

```
(kali㉿kali)-[~/john/run]
$ python3 office2john.py secret.xlsx > hash.txt
```

```
(kali㉿kali)-[~/john/run]
$ cat hash.txt
```

```
secret.xlsx:$office$*2013*100000*256*16*168f3a3c99bda3d5febd876dd06a9665*eb285a8ada2605fb839e9b904dba5dad*b120476bc6ba316452be48dcc9168ce2c9e735ae9f0cf3842fc7df6ccc66c574
```

```
(kali㉿kali)-[~/john/run]
$ john hash.txt --wordlist=rockyou.txt
```

Using default input encoding: UTF-8

Loaded 1 password hash (Office, 2007/2010/2013 [SHA1 128/128 AVX 4x / SHA512 128/128 AVX 2x AES])

Cost 1 (MS Office version) is 2013 for all loaded hashes

Cost 2 (iteration count) is 100000 for all loaded hashes

Will run 4 OpenMP threads

Press 'q' or Ctrl-C to abort, almost any other key for status

spiderman (secret.xlsx)

1g 0:00:00:01 DONE (2025-05-29 11:58) 0.6711g/s 182.5p/s 182.5c/s 182.5C/s angelo..school

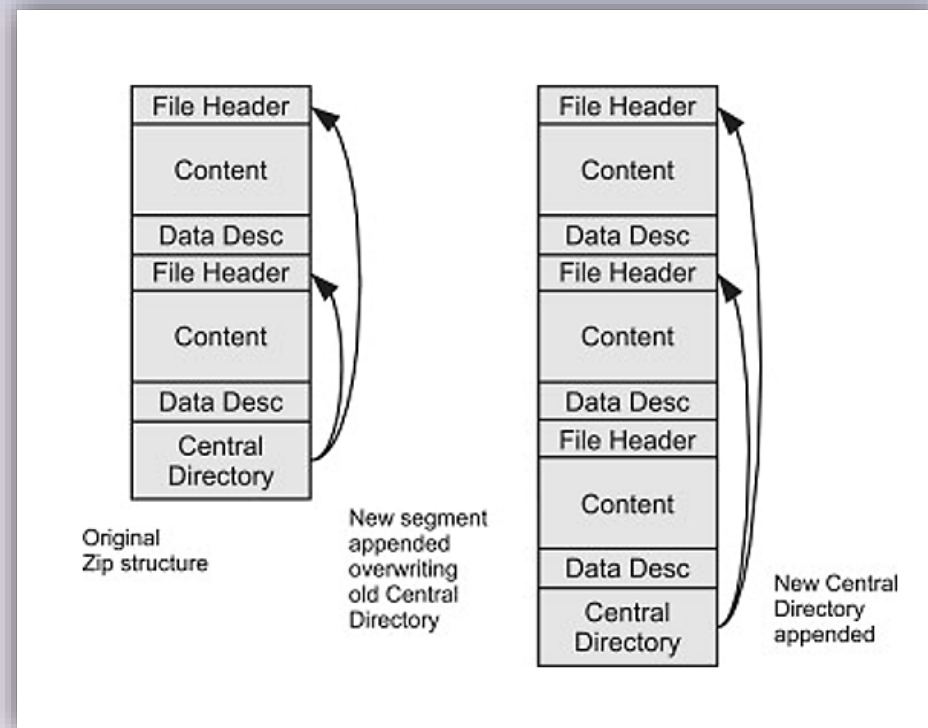
Use the "--show" option to display all of the cracked passwords reliably

Session completed.



File Structures

- In digital forensics, understanding the **internal structure of files** is essential for **identifying, carving, analyzing, and validating** evidence
- Each file format has a **specific binary** layout often starting with a file signature (magic number) and organized into **structured segments**



Cohen, M., Garfinkel, S., & Schatz, B. (2009). Extending the advanced forensic format to accommodate multiple data sources, logical evidence, arbitrary information and forensic workflow. *digital investigation*, 6, S57-S68.



File Structures (Cont)

■ JPEG File Structures

Element	Hex (Signature)	Description
SOI	FFD8	Start of Image
APPx	FFE0 – FFEF	Metadata (JFIF, EXIF, etc.)
DQT, DHT	FFDB, FFC4	Compression tables
SOS	FFDA	Start of scan (image data)
EOI	FFD9	End of Image

Offset	Bytes	Description
00000000	FFD8	SOI (Start of Image)
00000002	FFE0 0010	APP0 segment with length
00000006	4A464946	JFIF (ASCII)
...		
[Image Data]		
...		
0001ABCD	FFD9	EOI (End of Image)



File Structures (Cont)

■ PNG File Structures

Element	Bytes	Description
Signature	89 50 4E 47 0D 0A 1A 0A	PNG magic number
Chunks	[Length][Type][Data][CRC]	Organized data structure
Key Chunks	IHDR, IDAT, IEND, tEXt	Header, image, end, metadata

Offset	Bytes	Description
00000000	89 50 4E 47 0D 0A 1A 0A	PNG Signature
00000008	00 00 00 0D	Chunk Length: 13 bytes
0000000C	49 48 44 52	Chunk Type: IHDR (image header)
00000010	00 00 01 00 00 00 01 00	Width and Height = 256 x 256
00000018	08 02 00 00 00	Bit Depth, Color Type, etc.
0000001D	XX XX XX XX	CRC (4 bytes)
...		
nnnnnnnn	00 00 00 00 49 45 4E 44 XX XX XX XX	→ IEND chunk (end of PNG)



File Structures (Cont)

■ PDF File Structures

Element	Text/Hex	Description
Header	%PDF-1.x	Identifies PDF version
Body	obj, endobj	Object data
XRef Table	xref	Index of objects
Trailer	trailer	Metadata, root
EOF Marker	%%EOF	End of file

Offset	Bytes	Description
00000000	25 50 44 46 2D 31 2E 34	%PDF-1.4 (Header, version)
...	...	Body with "obj" and
"stream"		
nnnnnnnn	78 9C ...	Compressed stream (zlib)
...	...	XRef table
FFFFFFF0	74 72 61 69 6C 65 72	'trailer'
FFFFFFFC	25 25 45 4F 46	%%EOF (End of File marker)



File Structures (Cont)

■ ZIP File Structures

Element	Hex Signature	Description
Local Header	504B0304	Start of a zipped file
Central Dir	504B0102	Directory of zip entries
End of Dir	504B0506	End of central directory

Offset	Bytes	Description
00000000	50 4B 03 04	Local file header signature
00000004		File version, flags, etc.
0000001E	File name + compressed data	
...		
nnnnnnnn	50 4B 01 02	Central directory file header
...		
nnnnnnnn	50 4B 05 06	End of central directory



File Structures (Cont)

■ DOCX/XLSX/PPTX File Structures

Property	Details
Format	ZIP archive
Key Files	document.xml, [Content_Types].xml, etc.
File Signature	50 4B 03 04 (like ZIP)

Offset	Bytes	Description
00000000	50 4B 03 04	ZIP signature (same as .zip)
...	...	Local headers and XML files
0000002A	77 6F 72 64 2F 64 6F 63 ...	Starts with 'word/document.xml'
...	...	Contains main Office content



File Structures (Cont)

■ MP3 File Structures

Element	Hex Signature	Description
ID3 Tag	49 44 33	Metadata tag (ID3v2)
Frame Header	Varies (e.g., FF FB)	Start of audio frame

Offset	Bytes	Description
00000000	49 44 33 03 00 00	'ID3' tag (version 2.3, metadata)
00000006		Metadata fields (title, artist, etc.)
...		
00000100	FF FB 90 64	Frame header (MPEG 1 Layer III)
...	...	Audio frame data



File Structures (Cont)

■ Executable File (.exe) File Structures

Element	Hex Signature	Description
DOS Header	4D 5A ("MZ")	Start of Windows executable
PE Header	50 45 00 00	Indicates PE format
Sections	.text, .data, .rsrc, etc.	

Offset	Bytes	Description
00000000	4D 5A	'MZ' header (DOS stub)
00000002		DOS stub instructions
0000003C	 80	Offset to PE header (e.g., 0x80)
00000080	50 45 00 00	'PE\0\0' signature
00000084	4C 01	Machine type (Intel x86)
000000F8	2E 74 65 78 74	Section: .text (code)
00000100	2E 72 64 61 74	Section: .rdata (constants)



File Structures: PNG

- **PNG (Portable Network Graphics)** is a lossless, non-patented **image format** that stores image data in a **well-structured, chunk-based** binary format
- Example of **Minimal PNG in Hex**

00000000:	8950 4e47 0d0a 1a0a	→ PNG Signature
00000008:	0000 000d	→ Length = 13 bytes
0000000C:	4948 4452	→ Chunk type: IHDR
00000010:	0000 0020	→ Width = 32
00000014:	0000 0020	→ Height = 32
00000018:	08 02 00 00 00	→ Bit depth, color type...
0000001D:	90 91 68 36	→ CRC



File Structures: PNG (Cont)

■ Minimal PNG File Structures

Field	Size	Description
Signature	8 bytes	PNG signature (magic number)
Length	4 bytes	Length of the chunk's data (not including header or CRC)
Type	4 bytes	ASCII letters of chunk's type (e.g., IHDR , IDAT)
Data	Variable (depend on Length)	Chunk's content
CRC	4 bytes	Cyclic Redundancy Check for error detection



File Structures: PNG (Cont)

- **PNG Signature** (Magic Number): an **8-byte** fixed signature

00000000:	8950 4e47 0d0a 1a0a	→ PNG Signature
00000008:	0000 000d	→ Length = 13 bytes
0000000C:	4948 4452	→ Chunk type: IHDR
00000010:	0000 0020	→ Width = 32
00000014:	0000 0020	→ Height = 32

Hex	ASCII
89	Non-ASCII to detect binary
50 4E 47	PNG
0D 0A	CR LF (newline markers)
1A	EOF (DOS-style)
0A	LF (Unix-style)



File Structures: PNG (Cont)

■ Chunk Format

00000000:	8950 4e47 0d0a 1a0a	→ PNG Signature
00000008:	0000 000d	→ Length = 13 bytes
0000000C:	4948 4452	→ Chunk type: IHDR
00000010:	0000 0020	→ Width = 32
00000014:	0000 0020	→ Height = 32
00000018:	08 02 00 00 00	→ Bit depth, color type...
0000001D:	90 91 68 36	→ CRC

Field	Description
Length	4 bytes (not including header or CRC)
Chunk Type	4 bytes (e.g., IHDR, IDAT)
Width/Height	4 bytes each



File Structures: PNG (Cont)

■ Chunk Format (Cont)

00000000:	8950 4e47 0d0a 1a0a	→ PNG Signature
00000008:	0000 000d	→ Length = 13 bytes
0000000C:	4948 4452	→ Chunk type: IHDR
00000010:	0000 0020	→ Width = 32
00000014:	0000 0020	→ Height = 32
00000018:	08 02 00 00 00	→ Bit depth, color type...
0000001D:	90 91 68 36	→ CRC

Field	Description
Bit Depth	1 byte (e.g., 8)
Color Type	1 byte (e.g., 2 = RGB)
Compression	1 byte (usually 0)



File Structures: PNG (Cont)

■ Chunk Format (Cont)

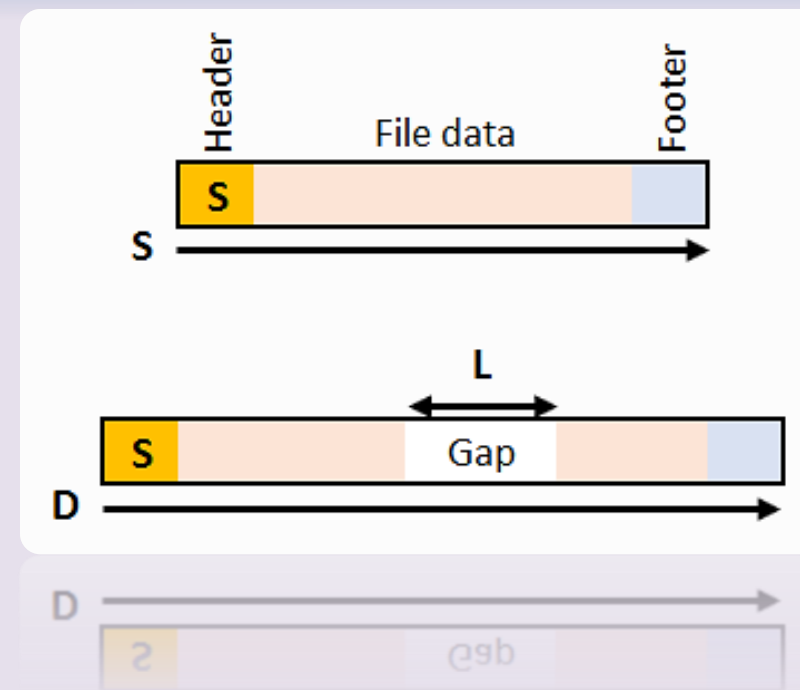
00000000:	8950 4e47 0d0a 1a0a	→ PNG Signature
00000008:	0000 000d	→ Length = 13 bytes
0000000C:	4948 4452	→ Chunk type: IHDR
00000010:	0000 0020	→ Width = 32
00000014:	0000 0020	→ Height = 32
00000018:	08 02 00 00 00	→ Bit depth, color type...
0000001D:	90 91 68 36	→ CRC

Field	Description
Filter	1 byte (usually 0)
Interlace	1 byte (0 = none, 1 = Adam7)
CRC	4 bytes



File Carving

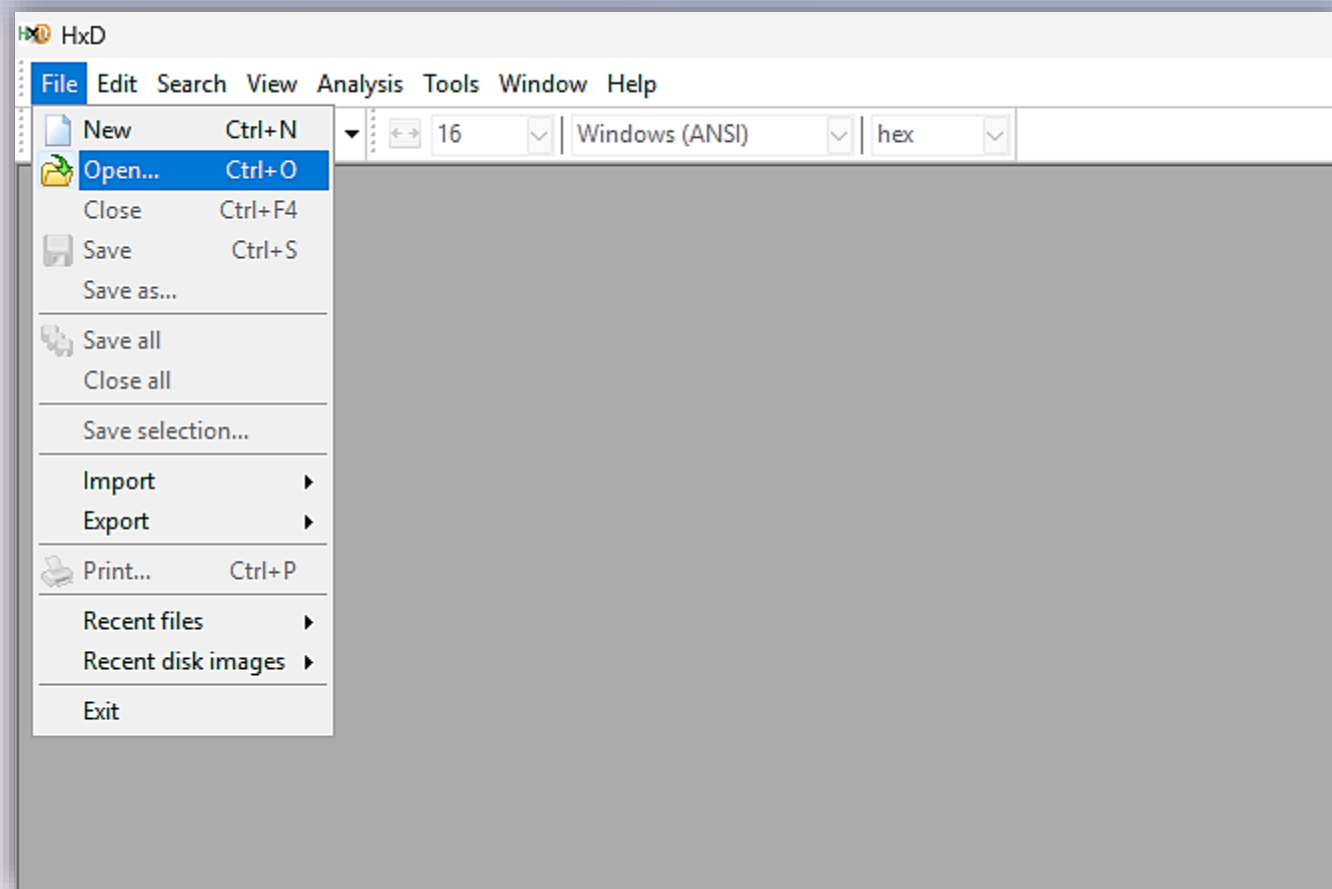
- **File Carving** is a digital forensics technique used to **recover files** without relying on **file system metadata** (such as file tables or directory entries)
- **File Carving** works by **identifying** and **extracting** files based on known **file signatures**
- You can **manually** perform **file carving** using **HxD** by identifying **file headers, trailers**, and extracting raw **byte ranges** into **new files**





File Carving with HxD

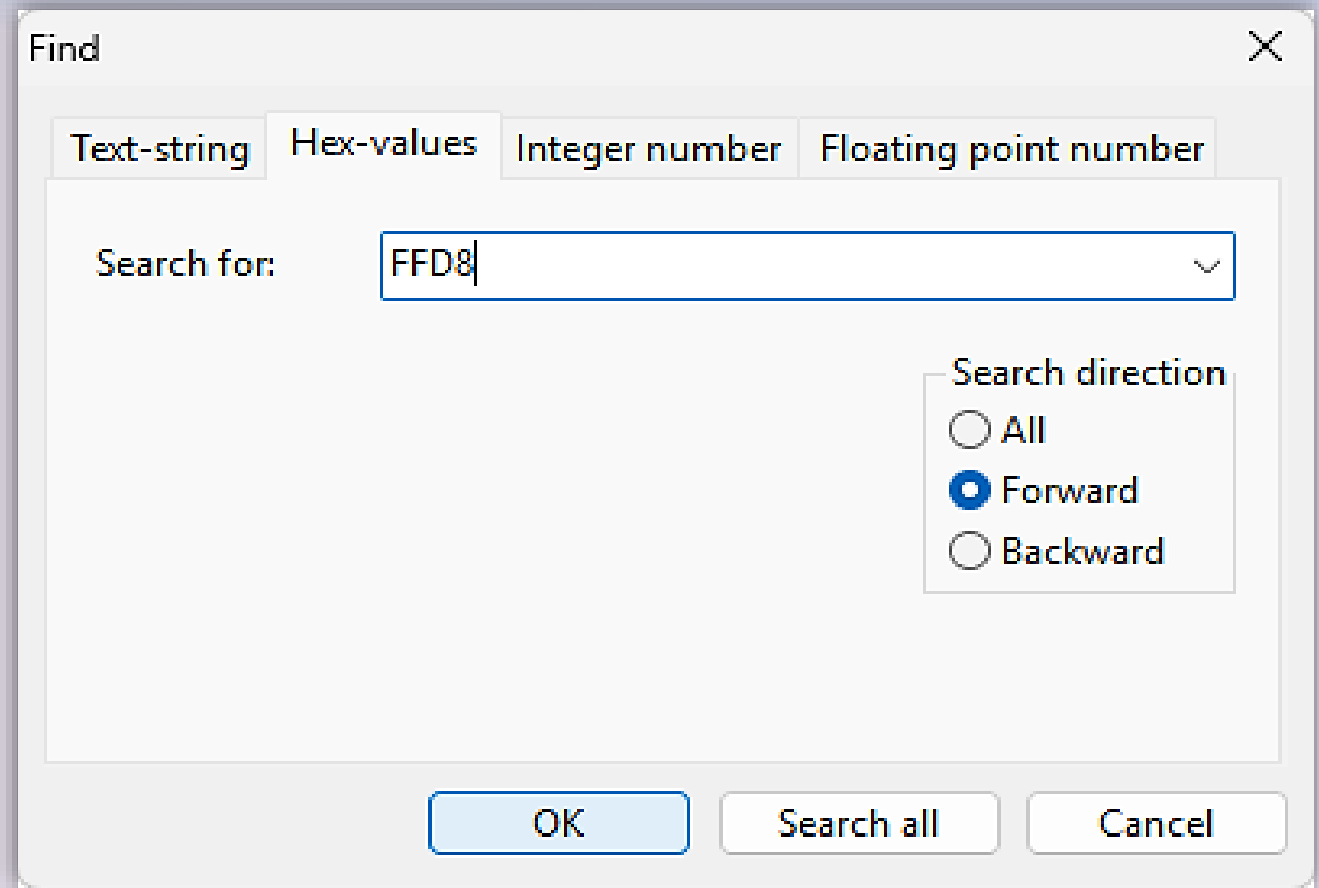
- **Step 1: Open the Image File**
 - Launch **HxD**
 - Click File > Open and select your **raw image** (e.g., disk.dd or usb.img)





File Carving with HxD (Cont)

- **Step 2: Search for File Header** (To find a **JPEG file**)
 - Press **Ctrl + F**
 - Set Datatype to "**Hex-values**"
 - Enter JPEG header: **FFD8** or **FFD8FF**
 - Click **Search All**(You'll see offsets where JPEG files potentially **begin**)





File Carving with HxD (Cont)

- **Step 3: Mark the Start Offset**
 - Note the **hexadecimal offset** shown (e.g., 0001A450)
 - This is the **start of the hidden file**

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
000244F0	3E	F5	13	20	52	AC	0F	5C	60	D4	D9	F9	47	FB	D5	35	>ð. R-. \`ÔÙùGûÕ5
00024500	CC	6A	CC	D9	50	79	14	01	12	B3	47	BB	9E	5B	AF	F8	ìjìÙPy...³G»ž[¯ø
00024510	8A	8A	40	4B	32	F2	4F	60	6A	4E	FF	00	8D	59	B5	50	ŠŠ@K2ðo`jNÿ..YµP
00024520	D7	18	23	23	22	98	14	7C	93	0F	13	26	08	EF	DA	A1	×.##"~. "...&.iŮ;
00024530	65	FD	E0	38	3B	4F	23	E9	57	27	45	16	EB	81	DC	D5	eýà8;0#éW'E.ë.ÜÖ
00024540	6F	E2	14	00	C6	4E	32	07	06	98	14	98	D9	88	24	01	oâ...ÆN2...~.~Ü^\$.
00024550	53	CC	7F	74	9F	EF	1A	88	70	64	03	A5	30	28	12	56	Sì.tŸi.^pd.¥0(.V
00024560	52	71	8F	6F	51	51	CE	AA	B2	9C	1E	0F	35	6A	E4	7E	Rq.oQQîª²æ..5jã~
00024570	E5	3E	B5	52	41	FB	B1	F5	34	C4	22	9A	53	4C	15	27	å>µRAû+ø4Ä"šSL.'
00024580	6A	04	25	32	95	68	34	C0	FF	D9	FF	D8	FF	E0	00	10	j.%2•h4ÄÿÜÿÿà..
00024590	4A	46	49	46	00	01	02	00	00	01	00	01	00	00	FF	ED	JFIF.....ÿí
000245A0	00	9C	50	68	6F	74	6F	73	68	6F	70	20	33	2E	30	00	.æPhotoshop 3.0.
000245B0	38	42	49	4D	04	04	00	00	00	00	00	80	1C	02	28	00	8BIM.....€..(.
000245C0	62	46	42	4D	44	30	31	30	30	30	61	61	32	30	31	30	bFBMD01000aa2010
000245D0	30	30	30	65	37	31	63	30	30	30	30	32	64	33	37	30	000e71c00002d370
000245E0	30	30	30	37	34	33	63	30	30	30	30	35	36	33	65	30	000743c0000563e0
000245F0	30	30	30	66	61	35	64	30	30	30	30	32	66	37	65	30	000fa5d00002f7e0
00024600	30	30	30	32	38	38	61	30	30	30	30	62	37	38	66	30	000288a0000b78f0
00024610	30	30	30	37	62	39	32	30	30	30	30	33	32	63	31	30	0007b92000032c10
00024620	30	30	30	1C	02	67	00	14	33	41	57	42	43	7A	37	72	000..g...3AWBCz7r
00024630	39	78	52	5F	52	44	75	7A	72	4F	71	4D	FF	DB	00	43	9xR_RDuzrOqMÿÛ.C
00024640	00	09	06	07	08	07	06	09	08	08	08	0A	0A	09	0B	0E	



File Carving with HxD (Cont)

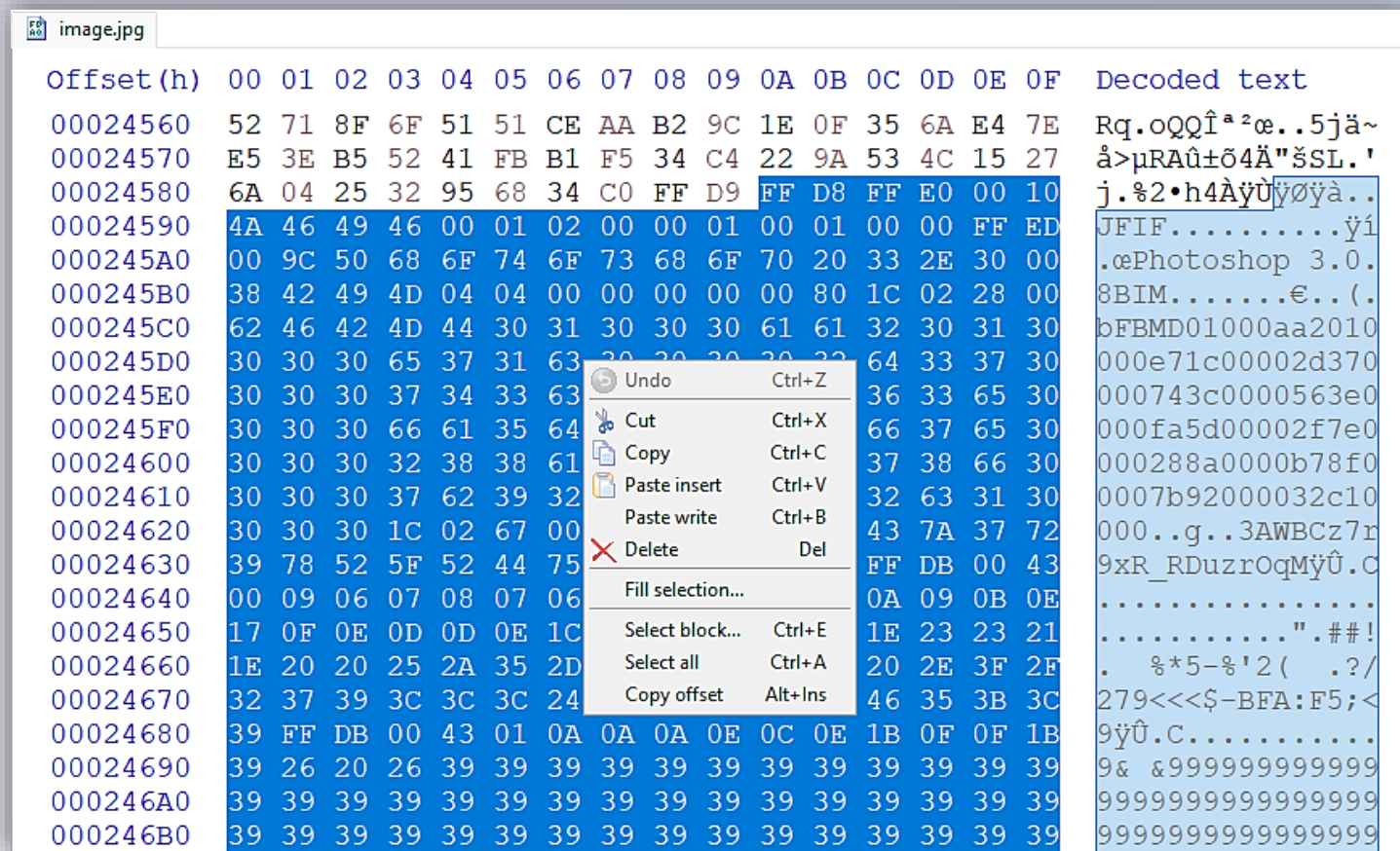
- **Step 4: Search for File Trailer (End of File)**
 - Press **Ctrl + F** again
 - Search for JPEG trailer: **FFD9**
 - Choose an offset after the header you found
 - Note the trailer's offset

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00030590	6A	FE	02	05	4E	5B	27	DB	31	7E	78	54	86	69	08	90	jþ..N['Û1~xT+i..
000305A0	D7	4F	63	C6	B1	FA	22	7A	94	2D	EE	FA	B7	95	15	1E	×OcÆ±ú"z"-îú·...
000305B0	11	B3	15	1F	E2	63	C8	8F	33	10	29	DD	34	B8	8D	C9	.³...âcÈ.3.)Ý4,.É
000305C0	4A	C0	F7	59	0D	5C	BA	1D	F4	77	24	CA	17	44	B8	F1	JÀ÷Y.\°.ôw\$Ê.D,ñ
000305D0	AC	39	15	25	DD	56	23	BC	A2	45	EE	45	32	18	E8	8B	¬9.¥ÝV#¼¢EîE2.è<
000305E0	3B	AA	16	12	83	0F	11	C8	DC	5A	9A	E4	91	4A	E8	17	;ª...f...ÈÛZŠä`Jè.
000305F0	5E	14	AC	AB	65	D0	D4	CB	A9	DF	4E	61	A5	C4	6F	5A	^.-«eÐÔË©ßNa¥ÄoZ
00030600	32	9F	E8	A2	A3	6E	2A	3C	A2	C4	5F	5A	CC	51	AA	CE	2Ýèç£n*<çÄ_zìQªî
00030610	83	04	DC	D6	42	A8	6D	CD	F7	3D	EB	F1	F0	B2	AB	64	f.ÜÖB`mÍ÷=ëñð²«d
00030620	B7	37	CC	F7	2F	C2	B3	1E	6A	B3	A1	80	6E	28	30	97	·7î÷/Â³.j³;îen(0-
00030630	D7	D1	C5	47	9D	1F	E1	E3	6A	3C	98	9B	50	07	44	64	×ÑÂG...áãj<~>P.Dd
00030640	86	E7	0A	F9	A0	70	FD	CF	75	22	75	56	59	6E	F1	A8	†ç.ù pýÿu"uVYnñ"
00030650	8B	7A	28	FF	00	1F	15	1E	4C	54	6D	C5	46	DE	41	43	<z(ÿ....LTmÄFÆPAC
00030660	B3	15	1E	4C	54	7F	A9	8F	43	1F	E5	E2	A3	C6	2A	3C	³...LT.©.C.ää£Æ* <
00030670	98	A8	DB	8A	8A	8A	8A	8A	8D	B8	A8	F2	63	FC	64	7A	~"ÛŠŠŠŠŠ.,"òcüdZ
00030680	58	A8	F3	E3	D2	C7	F8	68	DB	8F	41	1E	11	51	51	E8	X"óãðÇøhÛ.A..QQè
00030690	A3	6E	3F	FA	6C	7A	28	A8	A8	F2	62	A3	C2	36	A3	FC	£n?úlz("``ðb£Â6£ü
000306A0	E4	7A	C8	FF	00	49	1B	51	E1	1E	44	6D	47	FA	B8	F3	äzÈÿ.I.Qá.DmGú,ó
000306B0	A3	FD	A4	54	78	45	47	FE	67	3F	FF	D9					£ý=TxEGpç?ÿÛ



File Carving with HxD (Cont)

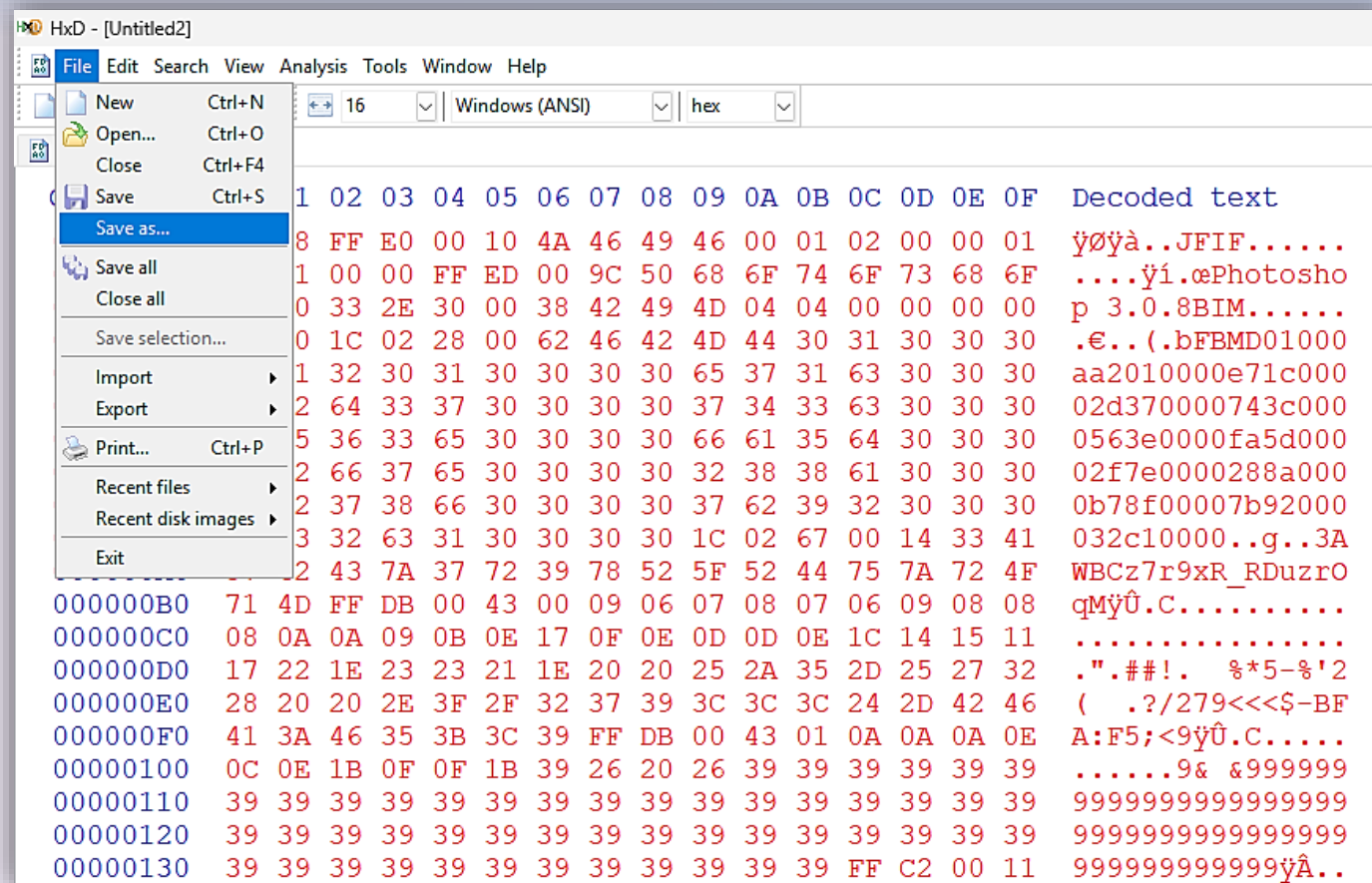
- **Step 5: Select and Copy the File Bytes**
 - Highlight from **start offset** to just after **trailer offset**
 - Right-click and choose **Copy** as > Hex Bytes or Save selection





File Carving with HxD (Cont)

- **Step 6: Save Extracted Data**
 - Go to File > **New**
 - **Paste** the data (**Ctrl + V**) or open a new window and paste selection
 - **Save as a file** (e.g., image1.jpg)

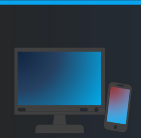




Python for File Forensics

■ File Type Detection via Header

```
def detect_file_type(file_path):  
    with open(file_path, 'rb') as f:  
        header = f.read(8).hex().upper()  
        if header.startswith('89504E47'):  
            return "PNG"  
        elif header.startswith('FFD8FF'):  
            return "JPG"  
        elif header.startswith('25504446'):  
            return "PDF"  
        else:  
            return "Unknown"  
  
print(detect_file_type('example.png'))
```



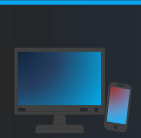
Python for File Forensics (Cont)

- **Extract File Metadata (Exif)** from Images

```
import exifread

def extract_exif(file_path):
    with open(file_path, 'rb') as f:
        tags = exifread.process_file(f)
        for tag in tags:
            print(f"{tag}: {tags[tag]}")

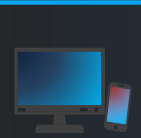
extract_exif('image.jpg')
```



Python for File Forensics (Cont)

■ File Carving - Extract JPEGs from Raw Data

```
def carve_jpeg(raw_data_path, output_dir):  
    with open(raw_data_path, 'rb') as f:  
        data = f.read()  
        start = 0  
        count = 0  
        while True:  
            soi = data.find(b'\xff\xd8', start)  
            eoi = data.find(b'\xff\xd9', soi)  
            if soi == -1 or eoi == -1:  
                break  
            carved = data[soi:eoi+2]  
            with open(f"{output_dir}/carved_{count}.jpg", 'wb') as out:  
                out.write(carved)  
            start = eoi + 2  
            count += 1  
  
carve_jpeg('disk.img', 'output_folder')
```



Python for File Forensics (Cont)

- Read **ZIP** or **DOCX** Contents

```
import zipfile

def list_zip(file_path):
    with zipfile.ZipFile(file_path, 'r') as zipf:
        zipf.printdir()

list_zip('evidence.docx')
```

Disk Forensics



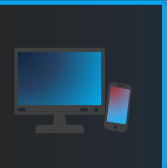


Disk Forensics Overviews

- **Disk forensics** is the process of acquiring, analyzing, and reporting on data stored on **digital storage media** such as **Hard drives, SSDs, USB drives, CD, DVD, Floppy Disk, and memory cards**
- It plays a crucial role in cybercrime investigations, incident response, and evidence recovery



<https://www.cyberprivilege.com/disk-memory-forensics>



Disk Forensics Overviews (Cont)

- **Key Components** of Disk Forensics

Component	Description
Disk Structure	Understanding of partitions , file systems , MBR/GPT
Imaging	Creating a forensically sound clone (bit-by-bit) of the drive
Analysis	Searching for artifacts (e.g., files, logs, browser history, deleted data)
Metadata	Timestamps, access logs, permissions
Data Recovery	Carving deleted or hidden files



File Systems

- Common **File Systems** Encountered

File System	OS/Platform	Features
FAT12/16/32	Windows, USB	Simple, easily recovered
NTFS	Windows	Rich metadata, journaling
exFAT	Removable media	Large file support
HFS+, APFS	macOS	Apple native formats
ext3/ext4	Linux	Journaling and rich metadata



Digital Artifacts

- In **digital forensics**, an **artifact** is any **data** left behind by the user, system, or software that can serve as evidence in an investigation
- **Artifacts** are indicators of **user behavior**, **system events**, and **file interactions** that are crucial for building timelines, proving intent, and reconstructing activity
- **Artifacts** are the **digital "footprints"** that help forensic analysts understand actions, timelines, and user intent





Digital Artifacts (Cont)

■ Common Artifacts of Interest

Artifact	Description
Deleted Files	May reveal intent, actions
Browser History	User behavior analysis
System Logs	Can show logins, installs, tampering
Page file/Swap file	May contain plaintext passwords or artifacts
Prefetch, Registry	Application use and timeline
\$MFT, \$LogFile (NTFS)	Metadata of files including deletions



Digital Artifacts (Cont)

Artifact	Description
Search Terms	Stored in browser or system logs
Clipboard Data	Copy-paste history
Recently Used Files	"Recent Documents", LNK files
User Assist	Tracks application execution
Windows Registry	Stores system and user config
Event Logs	System activity, errors, logins
Shellbags	Tracks folder views, USBs



Digital Artifacts (Cont)

Artifact	Description
File Metadata	Timestamps (MAC times), author, etc.
Alternate Data Streams	Hidden data in NTFS
Web Cache	Cached pages, media
DNS Cache	Resolved domain names
Network Logs	Firewalls, proxies, packet captures
Remote Access	SSH/RDP/VPN connections
Login Logs	Success/failure attempts



Digital Artifacts (Cont)

Artifact	Description
Password Hashes	SAM file (Windows), /etc/shadow (Linux)
Malware Indicators	Dropped files, registry keys, startup items
Encrypted Volumes	BitLocker, VeraCrypt headers





Disk Forensic Tools

■ Popular Disk Forensic Tools

Tool	Purpose
FTK Imager	Acquisition & viewing
Autopsy/SleuthKit	Full forensic analysis
dd / dcfldd	Disk cloning (Linux)
Guymager	Imaging with hash verification
X-Ways Forensics	Windows-based powerful suite
Magnet AXIOM	Advanced artifact recovery
EnCase	Industry-standard analysis



FTK Imager

- **FTK Imager** is a **free** digital forensics acquisition tool developed by **AccessData** (now part of **Exterro**)
- It allows investigators to create **forensic images** of **drives**, **partitions**, or **files** and to preview and export evidence in a forensically sound manner
- **FTK Imager** is a **data preview** and **imaging tool** that assess electronic evidence to determine the evidence





FTK Imager (Cont)

- **Key Features** of FTK Imager

Feature	Description
Forensic Imaging	Create exact bit-by-bit copies of drives/partitions/files
Memory Capturing	Create a memory dumps
Evidence Previewing	Browse contents of drives, including deleted files, without altering data
Mount Image Files	View and extract content from .E01, .dd, .001, .bin images
Export Capabilities	Export files, folders, registry, or specific artifacts



FTK Imager (Cont)

- Supported Output **Image Formats**

Format	Extension	Description
Raw (DD)	.dd, .img	Flat bit-by-bit clone
EWf	.E01	EnCase format with compression and metadata
SMART	.S01	Similar to EWf, used by SMART tools



FTK Imager (Cont)

■ Download & Installation FTK Imager

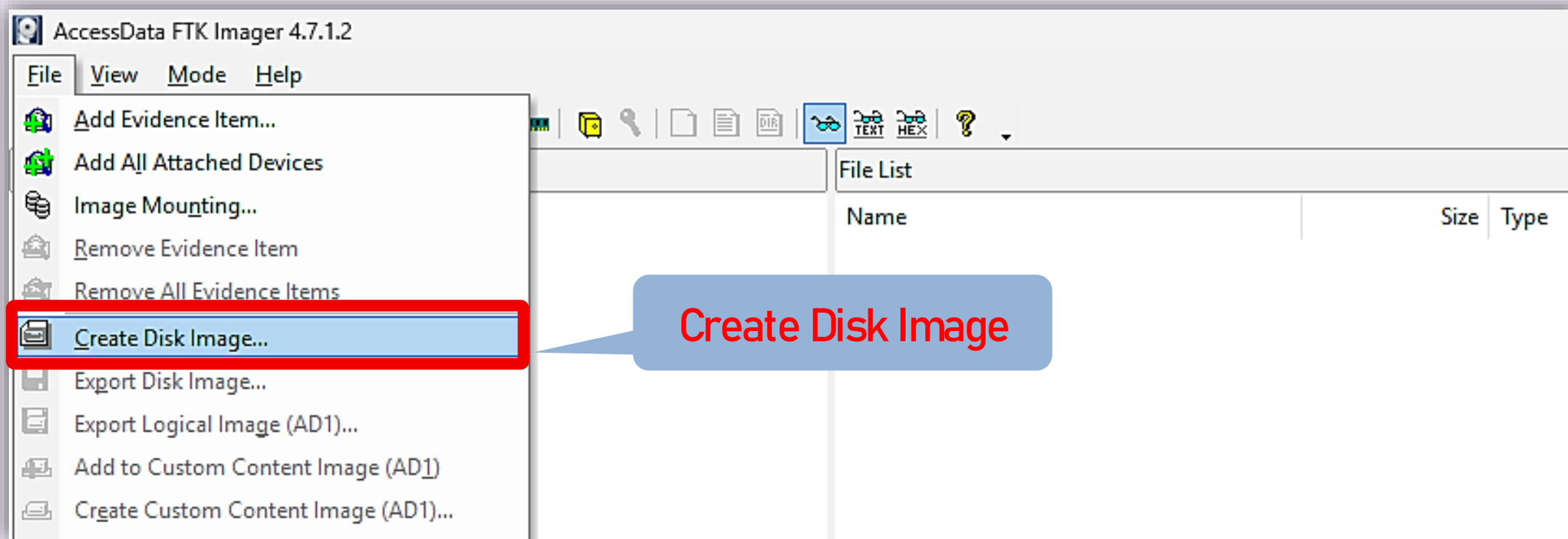
The screenshot shows the Exterro website for FTK Imager 4.7.3.81. The page has a white header with the Exterro logo and navigation links: Products, Solutions, Customers, Resources, and Company. On the right, it says 'Exterro for Law Enforcement' with a search icon and a 'GET A DEMO' button. The main content area is divided into two columns. The left column is titled 'FTK Imager 4.7.3.81' and contains a 'Release Information' section with buttons for 'RELEASE NOTES' and 'USER GUIDE'. Below these is the MD5 hash value: 3815b9c2a6aa8898ecbe55353aaf4b79. The bottom section is 'Product Downloads' with a button for 'FTK IMAGER 4.7.3.81'. The right column is a blue 'Support Portal' with a 'GET SUPPORT' button and two paragraphs of text: 'In version 4.7.3.21 of FTK Imager, users may have observed the unintentional inclusion of Decryption support.' and 'The latest release (4.7.3.81) does not include this functionality.'

Official site:
<https://www.exterro.com/ftk-product-downloads/ftk-imager-4-7-3-81>



Creating Forensic Image with FTK Imager

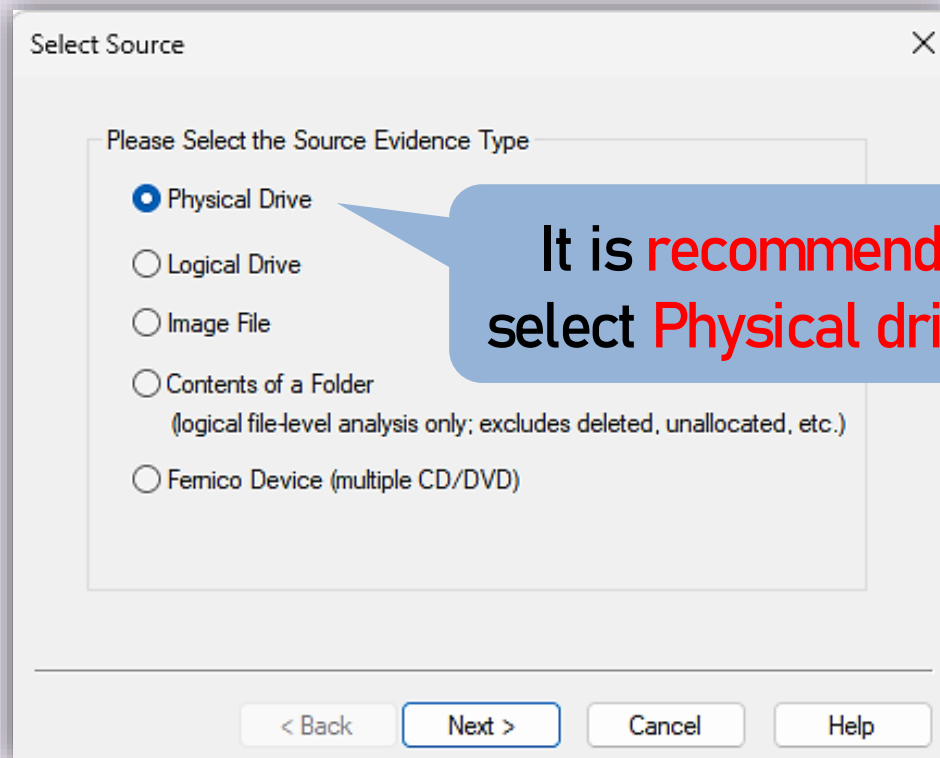
- Open FTK Imager and Click **File** > **Create Disk Image**



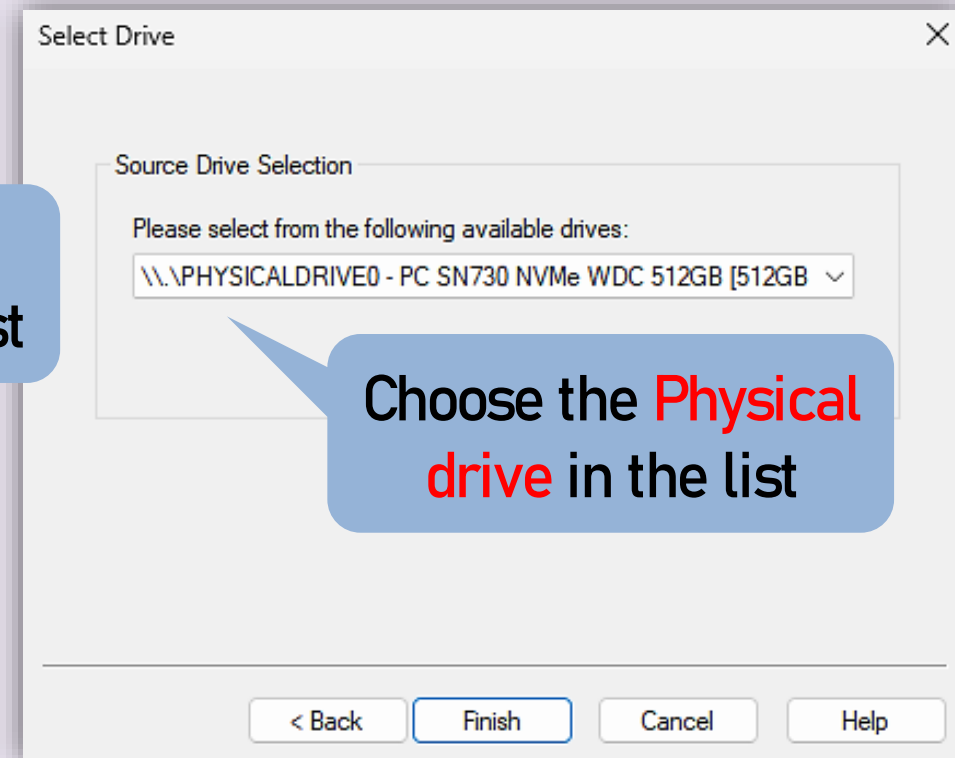


Creating Forensic Image with FTK Imager (Cont)

- Choose the **data source**: Physical drive, Logical drive, Image file, Folder



It is **recommended** to select **Physical drive** first



Choose the **Physical drive** in the list



Creating Forensic Image with FTK Imager (Cont)

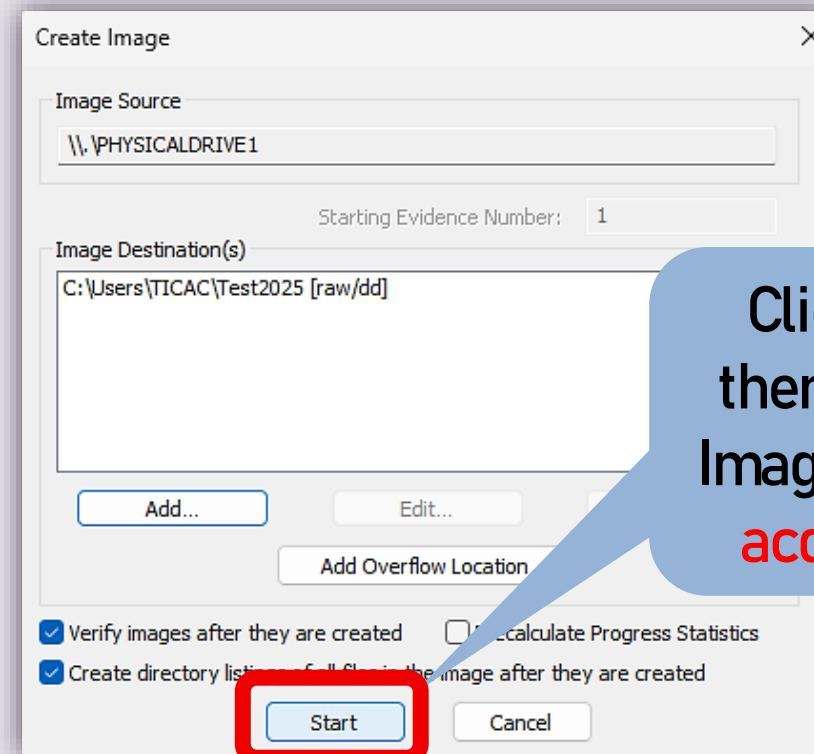
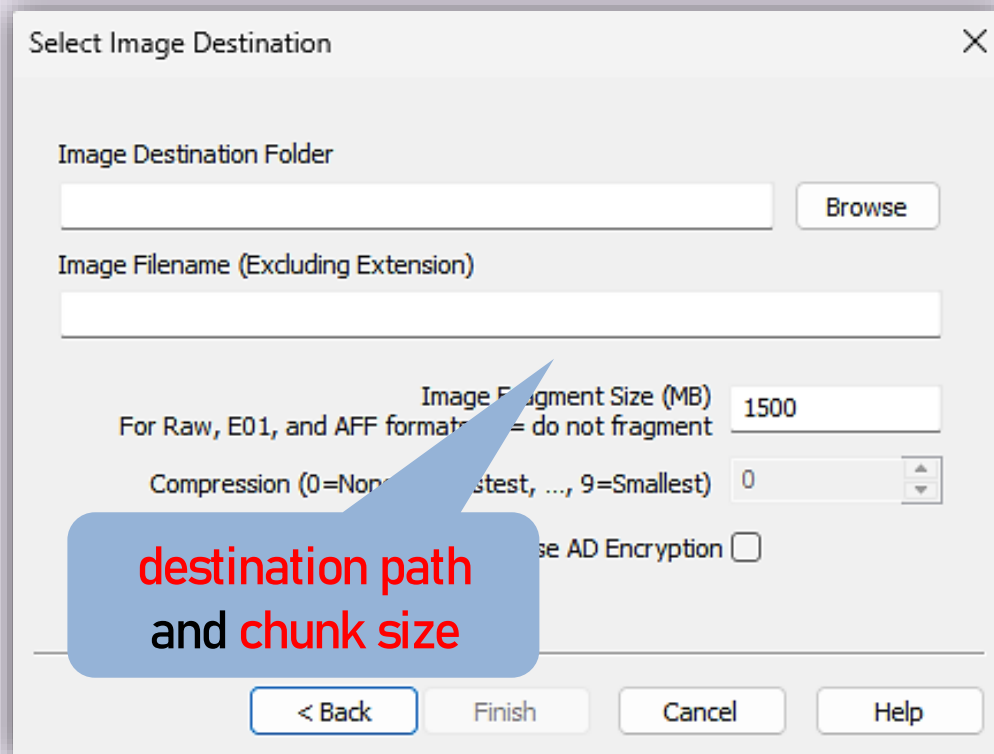
- Choose the **image type** and fill in **case details**

The image shows two side-by-side windows from the FTK Imager software. The left window, titled 'Select Image Type', contains a section 'Please Select the Destination Image Type' with four radio button options: 'Raw (dd)' (selected), 'SMART', 'E01', and 'AFF'. A blue callout bubble points to these options with the text 'Raw (DD), E01, SMART' in red. The right window, titled 'Evidence Item Information', contains five text input fields: 'Case Number:', 'Evidence Number:', 'Unique Description:', 'Examiner:', and 'Notes:'. A blue callout bubble points to these fields with the text 'Examiner name, case number, evidence number' in red. Both windows have a footer with four buttons: '< Back', 'Next >', 'Cancel', and 'Help'.



Creating Forensic Image with FTK Imager (Cont)

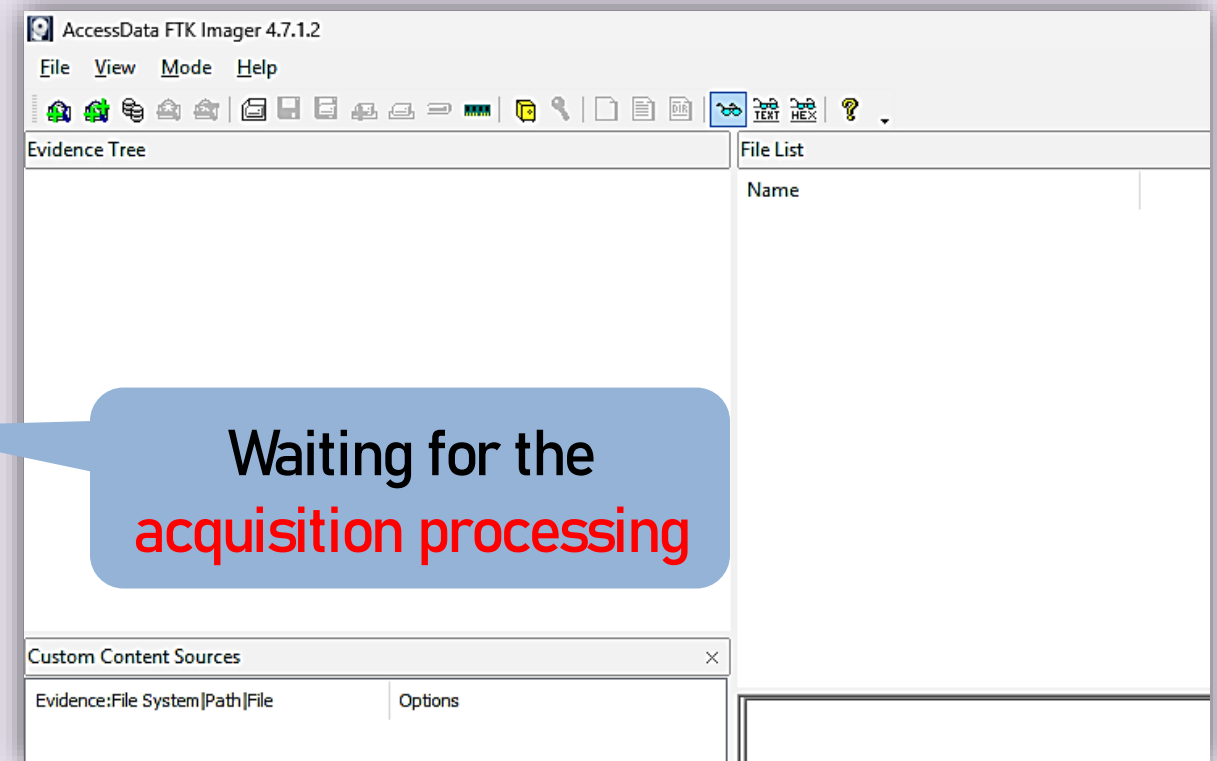
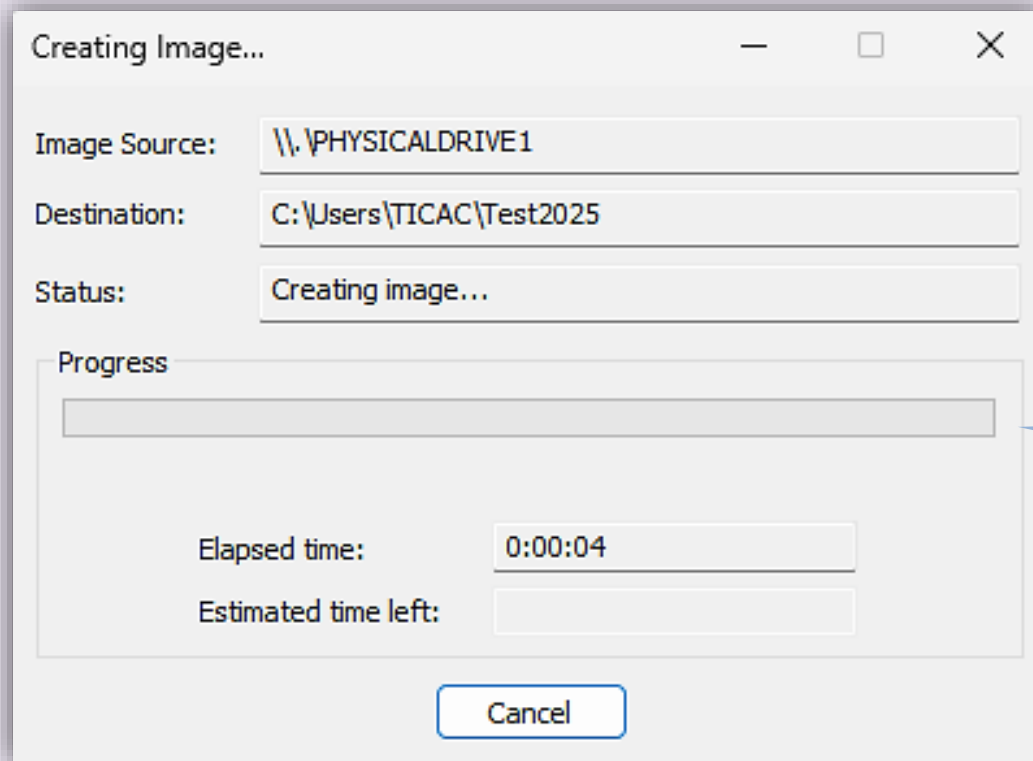
- Set the **destination path** and **chunk size** and click **start**





Creating Forensic Image with FTK Imager (Cont)

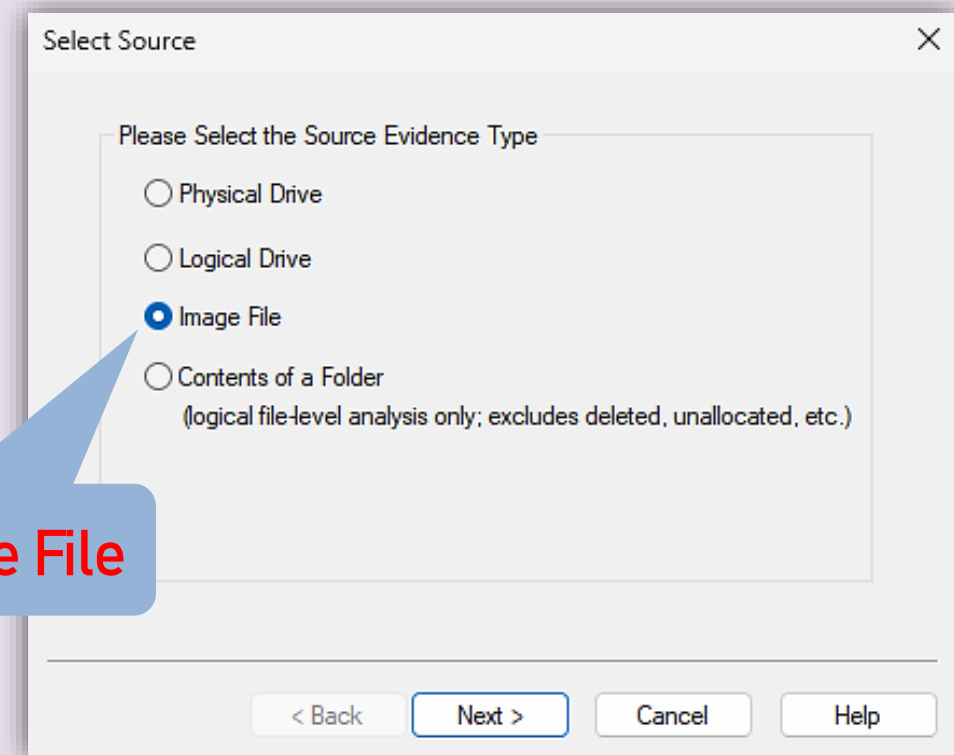
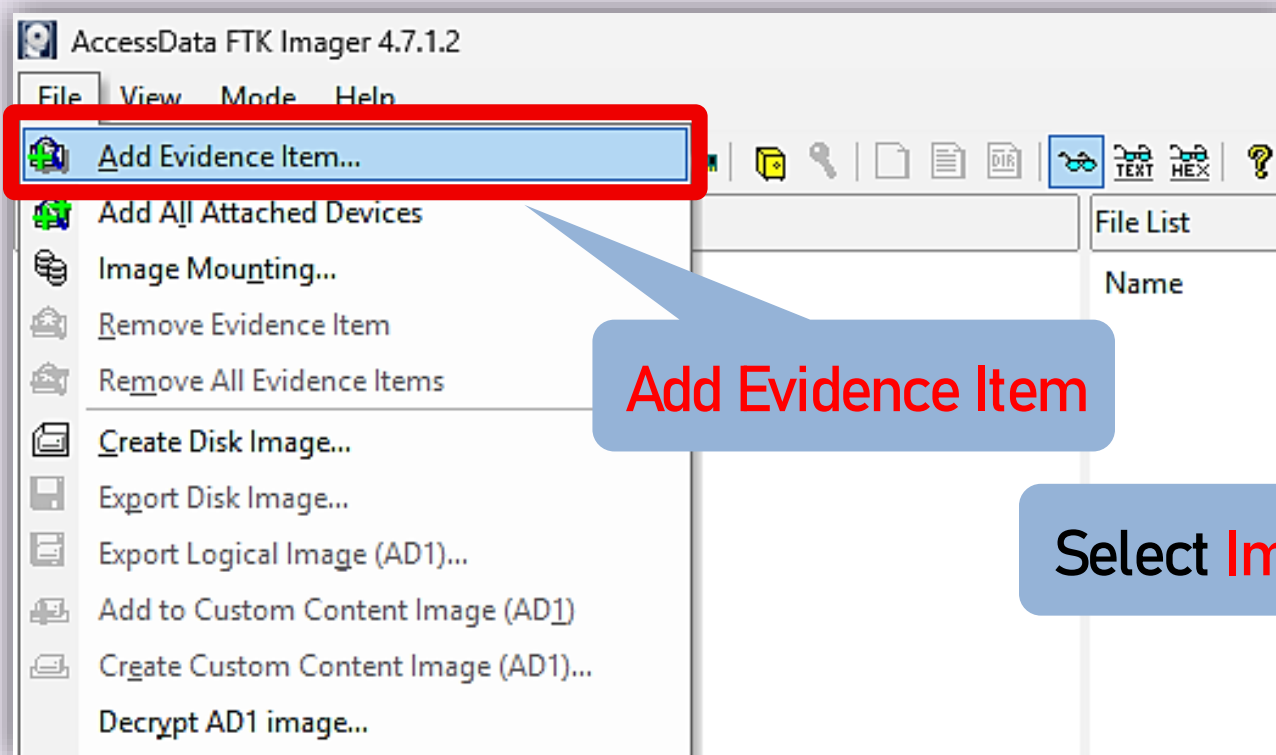
- Result: A **forensic image** with **log file** and **hashes**





Previewing Forensic Image with FTK Imager

- Mount an image: **File > Add Evidence Item > Image File**



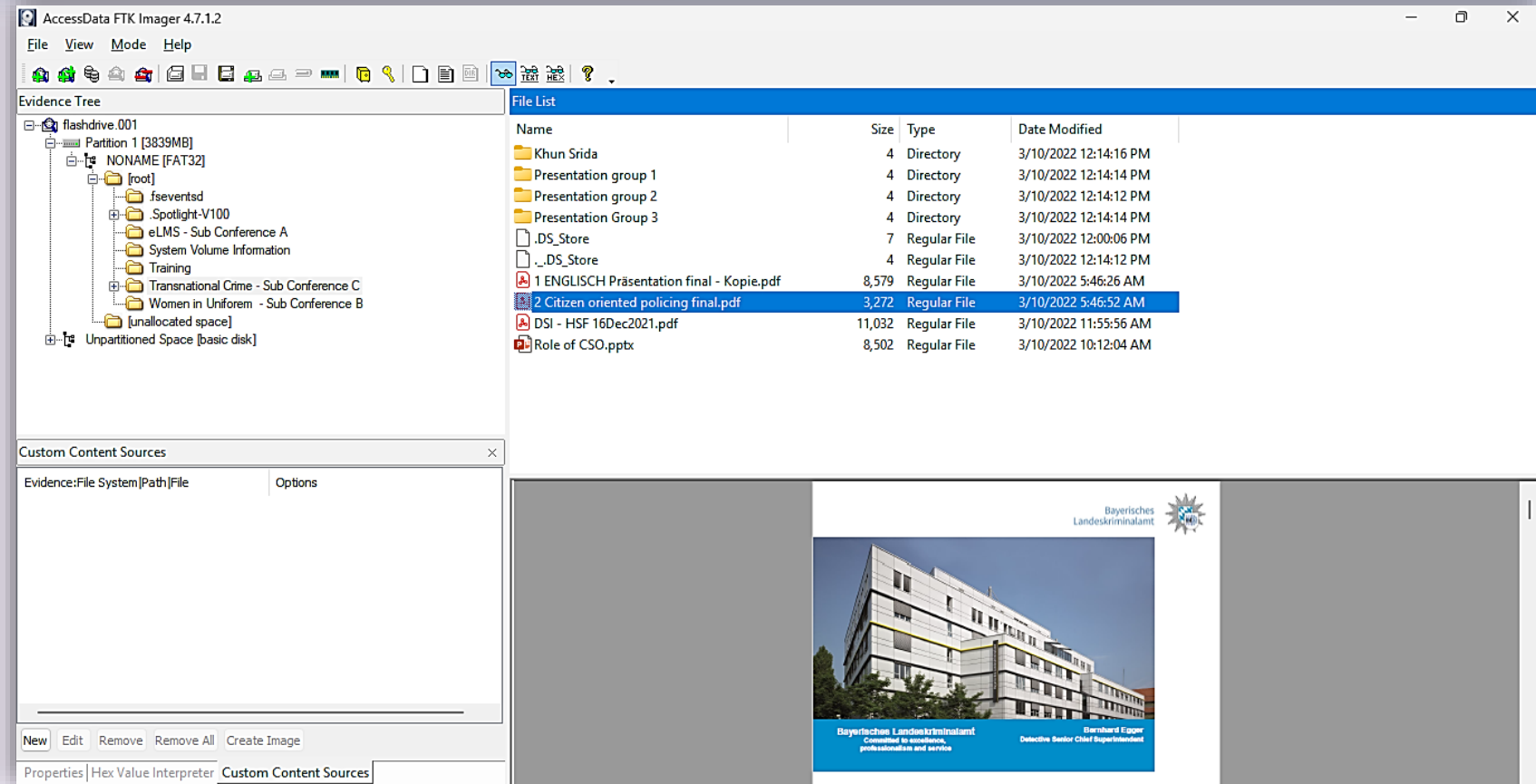


- [illegible]



Previewing Forensic Image with FTK Imager (Cont)

- Navigate through:
NTFS/FAT
structures,
target files,
deleted files,
system files
(e.g., \$MFT,
pagefile.sys,
NTUSER.DAT)





Previewing Forensic Image with FTK Imager (Cont)

- Select the **target** and **right-click** to **export** files or folders

The screenshot displays the FTK Imager interface. On the left, the 'Evidence Tree' shows a hierarchy starting with 'flashdrive.001', followed by 'Partition 1 [3839MB]', 'NONAME [FAT32]', and a '[root]' folder containing various subfolders like '.fseventsd', '.Spotlight-V100', and several 'Sub Conference' folders. On the right, the 'File List' pane shows a table of files and folders. The file '2 Citizen oriented policir' is selected. A right-click context menu is open over this file, with 'Export Files...' highlighted. A blue callout bubble with the text 'Export Files' points to this menu item.

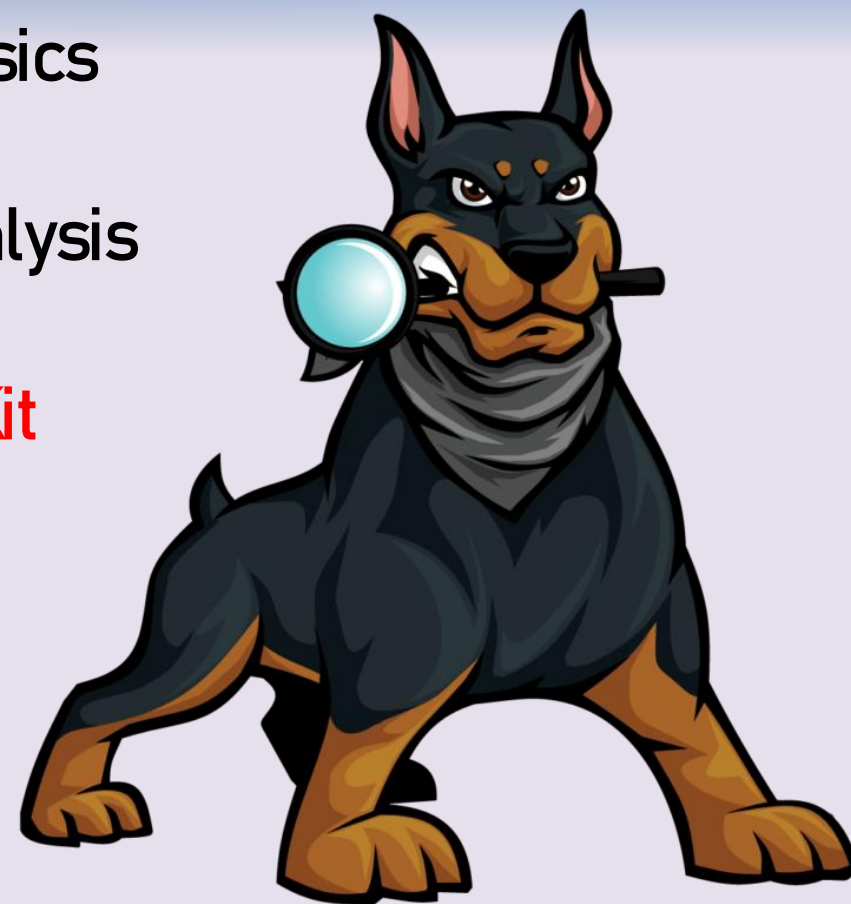
Name	Size	Type	Date Modified
Khun Srida	4	Directory	3/10/2022 12:14:16 PM
Presentation group 1	4	Directory	3/10/2022 12:14:14 PM
Presentation group 2	4	Directory	3/10/2022 12:14:12 PM
Presentation Group 3	4	Directory	3/10/2022 12:14:14 PM
.DS_Store	7	Regular File	3/10/2022 12:00:06 PM
_.DS_Store	4	Regular File	3/10/2022 12:14:12 PM
1 ENGLISH Präsentation final - Kopie.pdf	8,579	Regular File	3/10/2022 5:46:26 AM
2 Citizen oriented policir			3/10/2022 5:46:52 AM
DSI - HSF 16Dec2021.pdf			3/10/2022 11:55:56 AM
Role of CSO.pptx			3/10/2022 10:12:04 AM

Export Files



Autopsy

- **Autopsy** is a **free** and **open-source** digital forensics tool used by **law enforcement**, **military**, and corporate **investigators** to conduct in-depth analysis of digital evidence
- It provides a **graphical interface** to **The Sleuth Kit (TSK)** and supports **analysis** of **disk images**, **file systems**, **user activity**, and more
- **Autopsy** is used for analyzing **file systems**: NTFS, FAT, ExFAT, HFS+, Ext2/Ext3/Ext4, and YAFFS2





Autopsy (Cont)

■ Key Features of Autopsy

Feature	Description
Disk Image Analysis	Analyze .E01, .dd, .img, .raw formats
File Recovery	Recover deleted, hidden, or fragmented files
User Activity Analysis	Emails, web history, downloads, logins
Keyword Search	Search for strings, regex, or indexed terms
Timeline View	Reconstruct events based on file system metadata
Report Generation	Export findings in PDF, HTML, Excel
Modular Plug-ins	Add modules for email, registry, EXIF, YARA, etc.



Autopsy (Cont)

- **Download** Autopsy (<https://www.autopsy.com/>)

The screenshot shows the Autopsy Digital Forensics website. At the top left is the Autopsy logo, which features a dog's head and the text "AUTOPSY DIGITAL FORENSICS". To the right of the logo is a navigation menu with links: "DOWNLOAD", "ADD-ON MODULES", "COMMUNITY & SUPPORT", "ABOUT", and "BLOG". Below the navigation menu is a blue banner with the text "Endpoint Investigations 2025: How to Find the Clues You Need [WEBINAR]". The main content area has a dark background with a hexagonal pattern. It features the heading "Download Autopsy for free" in large white text. Below the heading is a paragraph: "Autopsy® is the premier end-to-end open source digital forensics platform. Built by Sleuth Kit Labs with the core features you expect in commercial forensic tools, Autopsy is a fast, thorough, and efficient hard drive investigation solution that evolves with your needs." To the right of this text is a white button with a green download icon and the text "DOWNLOAD AUTOPSY >".



Analyzing Forensic Image with Autopsy

- **Step 1:** Launch Autopsy & Create a Case
 - Click "**New Case**"





Analyzing Forensic Image with Autopsy (Cont)

- **Step 1:** Launch Autopsy & Create a Case (Cont)
 - Enter **case information**: case name, number, examiner name
 - Autopsy creates a **workspace** folder

The image displays two overlapping screenshots of the 'New Case Information' dialog box in the Autopsy software.

The top screenshot shows the 'Case Information' tab, which is highlighted with a red rectangular box. It contains the following fields:

- Case Name:** A text input field.
- Base Directory:** A text input field showing 'C:\Forensic\' with a 'Browse' button to its right.
- Case Type:** Two radio buttons: 'Single-User' (selected) and 'Multi-User'.

The bottom screenshot shows the 'Optional Information' tab. It contains the following fields:

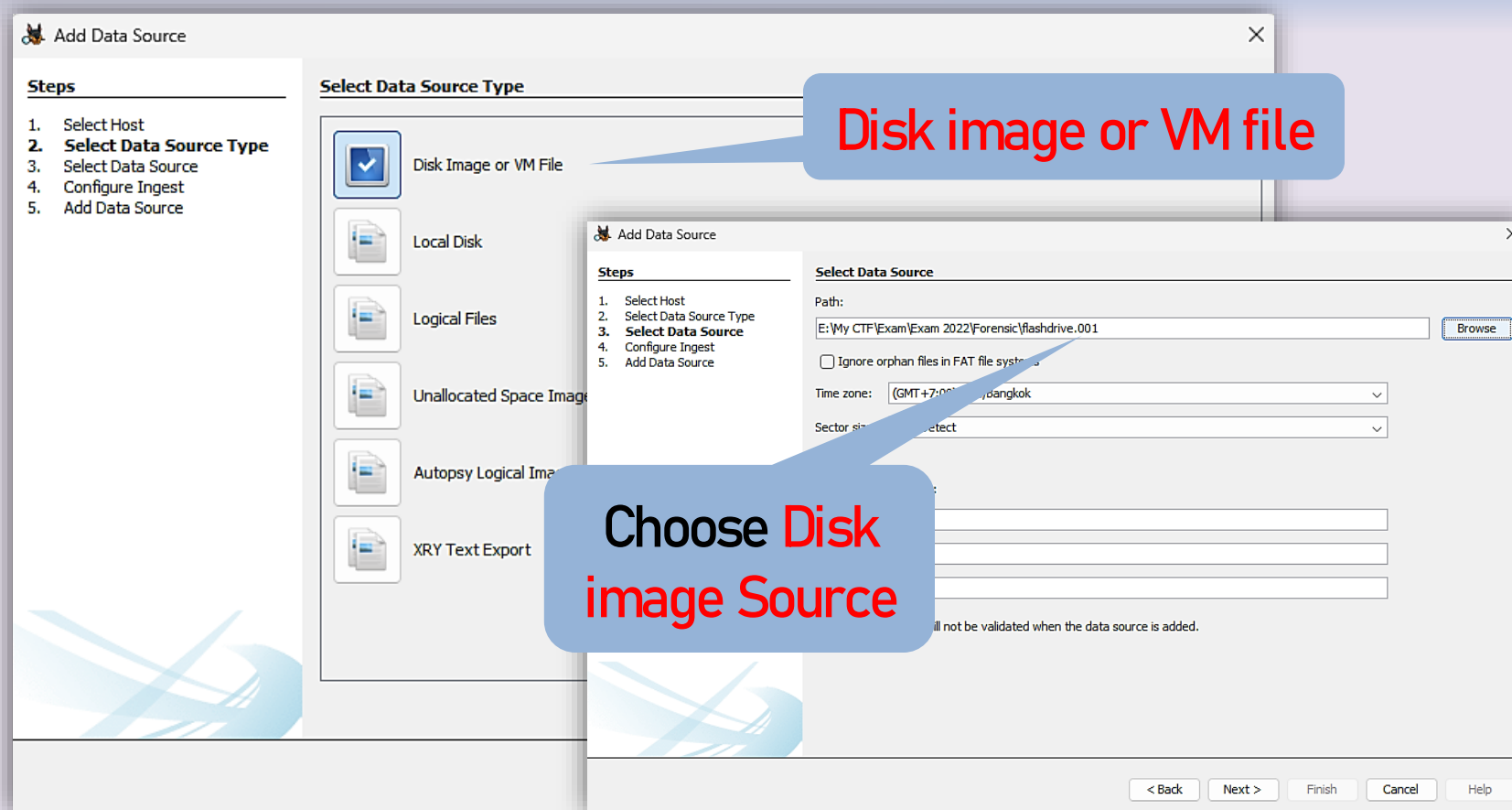
- Case Number:** A text input field.
- Examiner:** A section containing 'Name' (with 'Bank' entered), 'Phone' (with '0895993490' entered), 'Email', and 'Notes' (all in text input fields).
- Organization:** A section with 'Organization analysis is being done for:' (with 'Not Specified' in a dropdown menu) and a 'Manage Organizations' button.

At the bottom of the dialog box, there are buttons for '< Back', 'Next >', 'Finish', 'Cancel', and 'Help'. A blue callout bubble labeled 'Source' points to the 'Base Directory' field in the top screenshot. Another blue callout bubble labeled 'Case Information' points to the 'Case Name' field in the top screenshot.



Analyzing Forensic Image with Autopsy (Cont)

- **Step 2: Add Data Source**
 - Choose: **Disk image** (e.g., .E01, .dd), Local disk, and Folder of files
 - Autopsy will begin ingesting data

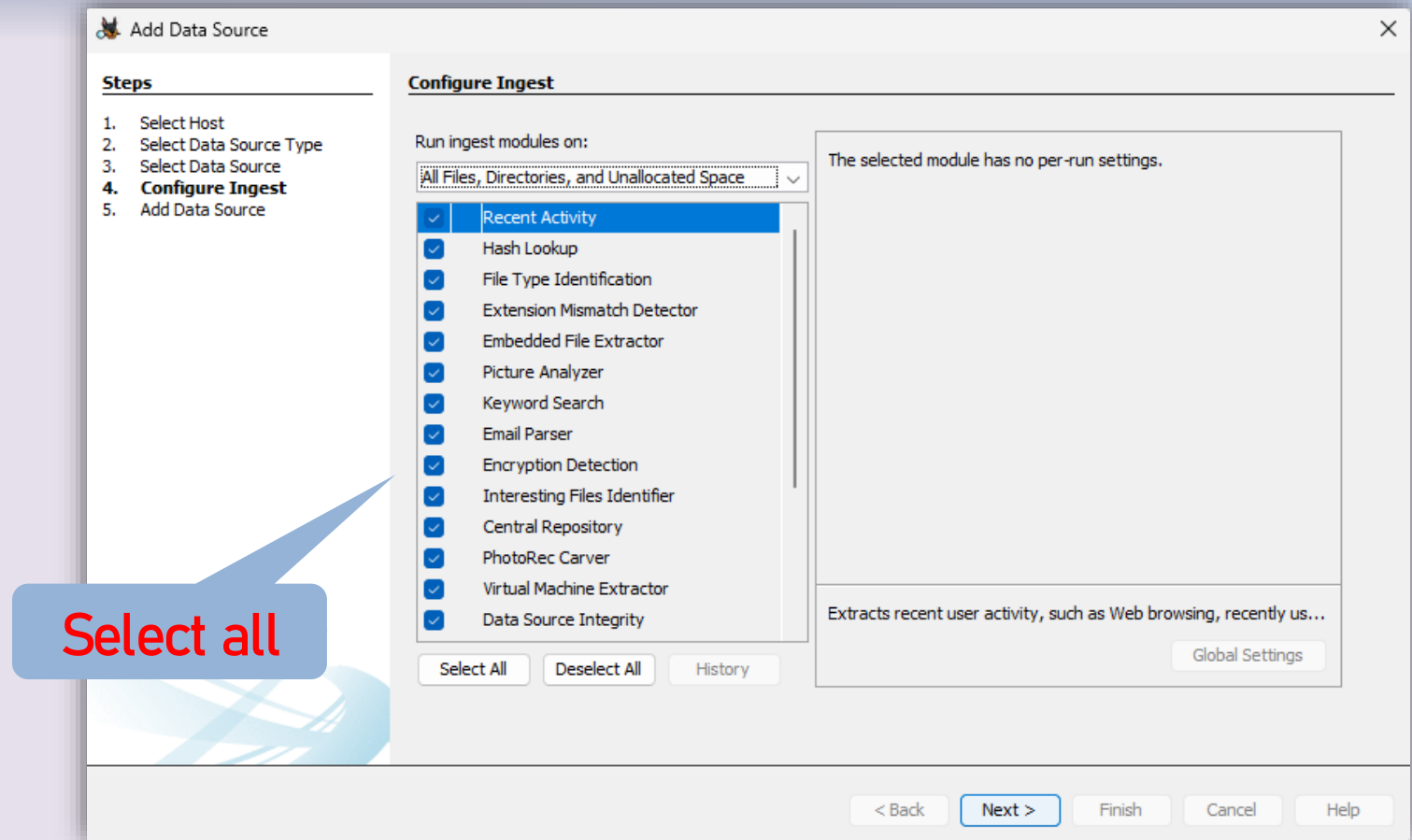




Analyzing Forensic Image with Autopsy (Cont)

■ Step 3: Configure Ingest Modules

- Hash lookup (e.g., NSRL)
- Keyword search
- File type identification
- Web artifacts
- EXIF metadata
- Registry parsing
- Recent activity



Analyzing Forensic Image with Autopsy (Cont)

■ Step 4: Review Evidence

- File Types
- Recent Documents
- Web History
- Cookies
- Deleted files
- Hidden files
- Specific extensions

The screenshot shows the Autopsy 4.19.3 interface. On the left, the 'Data Sources' tree is expanded to 'Images (2048)'. The main pane displays a table of image files. A blue callout bubble points to the row for 'image16.jpg'.

Name	S	C	O	Modified Time	Change Time	Access Time	Created Time	Size	Flags(Dir)	Flags(Meta)
_icture.bmp				2022-06-08 00:39:30 ICT	0000-00-00 00:00:00	2022-06-08 00:00:00 ICT	2022-06-08 00:39:23 ICT	2764854	Unallocated	Unallocated
the-picture.bmp			34	2022-06-08 00:58:12 ICT	0000-00-00 00:00:00	2022-06-08 00:00:00 ICT	2022-06-08 00:58:30 ICT	2764854	Allocated	Allocated
f0640576.bmp			34	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	2764854	Unallocated	Unallocated
image7.jpg			35	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	11141	Allocated	Allocated
image16.jpg			35	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	285514	Allocated	Allocated
image8.jpg			35	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	15374	Allocated	Allocated
image12.jpg			35	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	177517	Allocated	Allocated
image2.jpg			35	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	11339	Allocated	Allocated
image18.jpg			35	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	170356	Allocated	Allocated

The preview pane at the bottom shows a photo of three police officers in uniform serving food from a red container into small white bowls. The word 'DANANG' is visible in the bottom right corner of the photo.

Find the target file



Analyzing Forensic Image with Autopsy (Cont)

■ Step 5: Extracted Content

The screenshot shows the Autopsy forensic tool interface. A table lists several image files. A context menu is open over the file list, with the 'Extract File(s)' option highlighted by a red rectangle. A blue callout bubble with the text 'Extract Files' points to this option. Below the table, there are tabs for 'Hex', 'Timeline', 'Analysis Results', 'Context', 'Annotations', and 'Other Occurrences'. The 'Analysis Results' tab is selected, showing a preview of a video frame with police officers.

Name	S	C	O	Modified Time	Change Time	Access Time	Created Time	Size	Flags(Dir)	Flags(Meta)
image.bmp				2022-06-08 00:39:30 ICT	0000-00-00 00:00:00	2022-06-08 00:00:00 ICT	2022-06-08 00:39:23 ICT	2764854	Unallocated	Unallocated
the-picture.bmp			34	2022-06-08 00:58:12 ICT	0000-00-00 00:00:00	2022-06-08 00:00:00 ICT	2022-06-08 00:58:30 ICT	2764854	Allocated	Allocated
f0640576.bmp			34	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	2764854	Unallocated	Unallocated
image7.jpg			35	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	11141	Allocated	Allocated
image8.jpg				0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	285514	Allocated	Allocated
image9.jpg				0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	15374	Allocated	Allocated
image10.jpg				0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	177517	Allocated	Allocated
image11.jpg				0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	11339	Allocated	Allocated
image12.jpg				0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	170356	Allocated	Allocated

Context Menu Options:

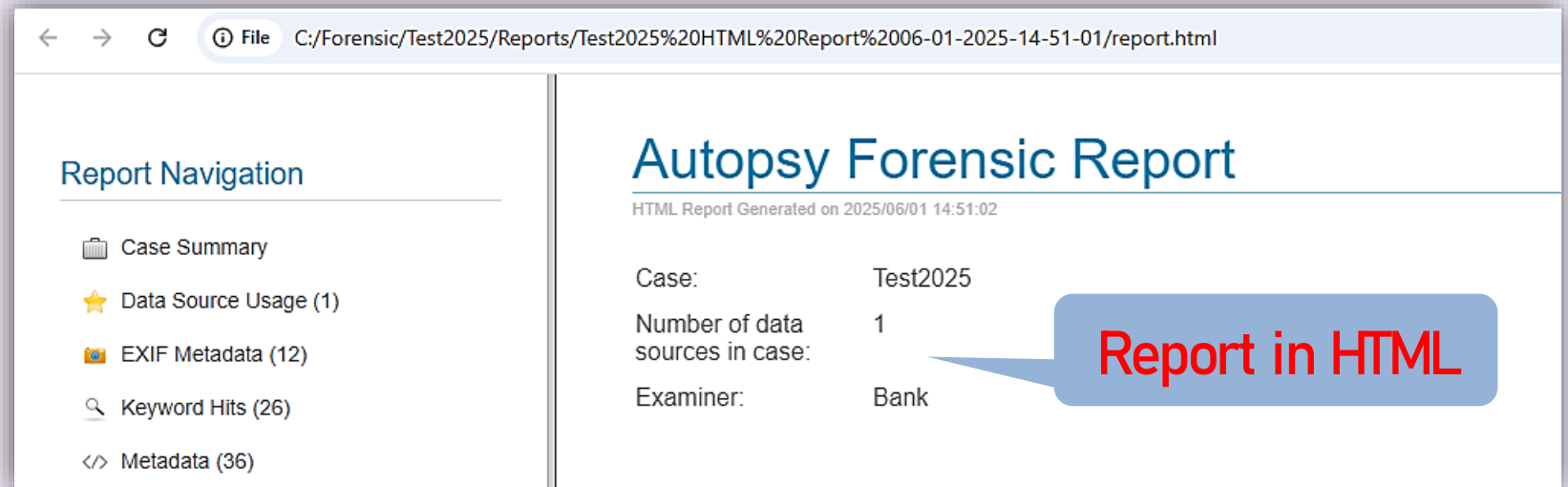
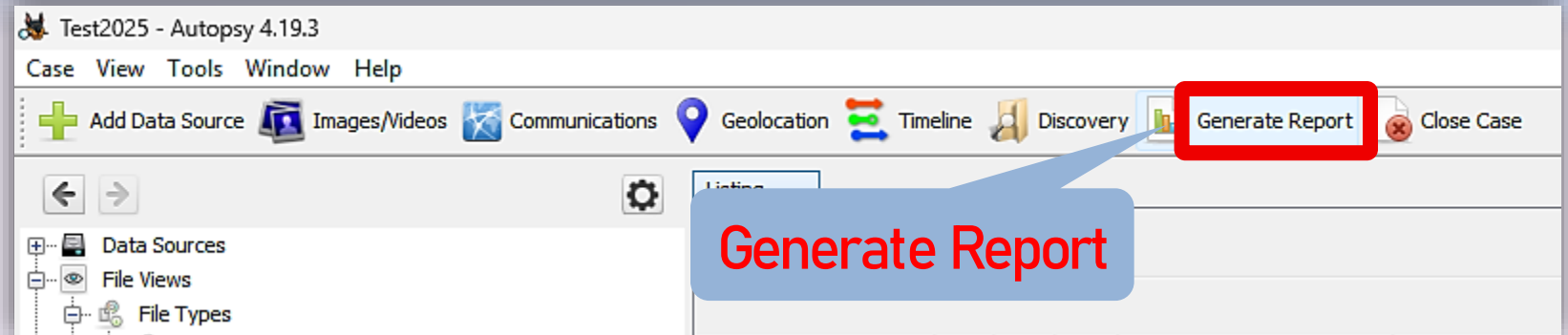
- Properties
- View File in Directory
- View in New Window
- Open in External Viewer Ctrl+E
- Extract File(s)**
- Export Selected Rows to CSV
- Add File Tag
- Remove File Tag
- Add/Edit Central Repository Comment
- Add File to Hash Set

Analysis Results Preview:

Analyzing Forensic Image with Autopsy (Cont)

■ Step 5: Generate Report

- Go to "Generate Report"
- Choose **format**: HTML, CSV, Excel, PDF
- Can include timeline, keywords, hash results, etc.





dd

- The **dd** command is a powerful **Unix/Linux tool** used in **digital forensics** to create **exact bit-by-bit** copies of **physical** or **logical drives**
- It's essential for making **forensic images** while preserving the integrity of digital evidence in **Linux-like system**

```
root@kerneltalks # dd if=/dev/xvdf of=/dev/xvdg  
4194304+0 records in  
4194304+0 records out  
2147483648 bytes (2.1 GB) copied, 181.495 s,  
11.8 MB/s
```

Learn dd command with examples

kerneltalks.com

<https://z5.kerneltalks.com/wp-content/uploads/2020/06/learn-dd-command.png>



dd (Cont)

- **Key Features** of **dd** command
 - Acquire raw **images** of **hard drives**, **SSDs**, **USBs**, or **partitions**
 - Preserve evidence in a forensically sound manner
 - Create **backup copies** with checksum validation
 - Wipe disks securely (with caution)
 - **File carving** or sector-level inspection



dd (Cont)

■ Basic Syntax

```
dd if=[input] of=[output] bs=[block size] conv=[options]
```

Parameter	Meaning
if	Input file or device (e.g., /dev/sdb)
of	Output file or device (e.g., usb_image.dd)
bs	Block size (e.g., 512, 1M)
conv	Options like noerror, sync





dd (Cont)

■ Example

#Acquire a USB Image

```
sudo dd if=/dev/sdb of=usb_image.dd bs=4M status=progress
```

#Advanced Command with Recommended Forensic Parameters

```
sudo dd if=/dev/sdX of=evidence.dd bs=512 conv=noerror,sync  
status=progress
```

#Disk Wiping (Use With Caution)

```
sudo dd if=/dev/zero of=/dev/sdX bs=1M status=progress
```



Guymager

- Guymager is a **free** and **open-source** forensic **imaging tool** used for acquiring **forensic disk images** in a forensically sound manner
- It provides a **graphical user interface (GUI)** on top of **Linux** disk imaging tools like **dd** and **dcfldd**, making it ideal for forensic professionals who prefer a visual workflow over the command line





Guymager (Cont)

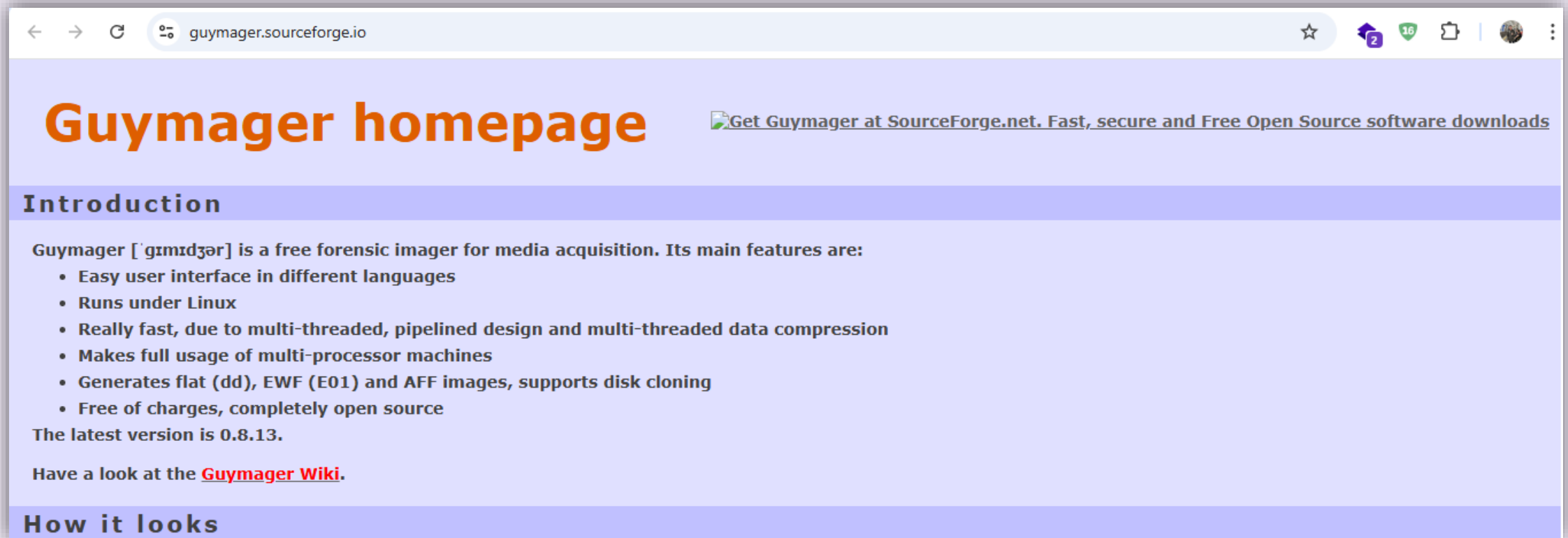
■ Key Features of Guymager

Feature	Description
Forensic Imaging	Acquire physical/logical images of disks
GUI Interface	User-friendly GUI for imaging tasks
Hash Verification	Supports MD5 and SHA-1 hashing
Multiple Output Formats	dd (raw), EWF (.E01)
Simultaneous Output	Write to multiple targets (e.g., file + device)
Live Progress Reporting	Real-time imaging statistics
Read-Only Access	Ensures source integrity during acquisition



Guymager (Cont)

- **Download** Guymager (<https://guymager.sourceforge.io/>)





Guymager (Cont)

■ Example

GUYMAGER

Devices Misc Help

Rescan

Serial nr.	Linux device	Model	State	Size	Bad sectors	Progress	Average Speed [MB/s]	Time remaining	FIFO queues usage [%]
1ATA_Hitachi_HDP725050GLA360_GEA534RF3Z90WA	/dev/sdd	ATA Hitachi HDP72505	Acquisition running	500.1GB	0	8%	89.73	01:21:17	r 0 c 0 w
1ATA_SAMSUNG_HD322HJ_S17AJ9BQ607434	/dev/sdc	ATA SAMSUNG HD322HJ	Acquisition running	320.1GB	0	12%	80.32	00:55:31	r 100 h 0 c 0 w
Sony_Storage_Media_BC05061400492-0:0	/dev/sde	Sony Storage Media	Finished	1.0GB	0	100%	9.72		
1ATA_MAXTOR_STM3250310AS_6RY761SP	/dev/sda	ATA MAXTOR STM325031	Local device	250.1GB					
1ATA_WDC_WD10EACS-00D6B1_WD-WCAU46176369	/dev/sdb	ATA WDC WD10EACS-00D	Local device	1.0TB					

Size 320,072,933,376 bytes (298GiB / 320GB)
Sector size 512
Image file /mnt/ext/hst/SAMSUNG_HD322HJ_S17AJ9BQ607434_320GB.Exx
Info file /mnt/ext/hst/SAMSUNG_HD322HJ_S17AJ9BQ607434_320GB.Info
Current speed 87.37 MB/s
Started 17. August 10:08:02 (00:07:49)
Hash calculation on
Source verification off

Memory Forensics

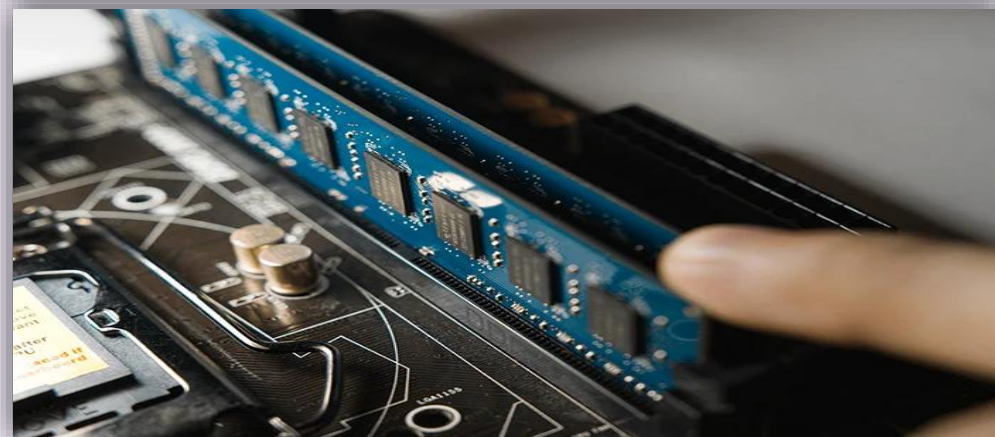
4





Memory Forensics Overviews

- **Memory Forensics** focuses on the capture and analysis of **volatile memory (RAM)** to uncover valuable information about what was happening on a system during a given time (**during the device open**) — **Live Forensics**
- It's critical for **incident response**, **malware analysis**, and cybercrime investigations — revealing details that are often not available in disk forensics





Memory Artifacts

■ Key Artifacts Recovered from RAM

Artifact Type	Examples
Processes	Running tasks, parent-child relationships
Network activity	Active connections, listening ports
Modules/Drivers	Kernel modules, injected DLLs
Passwords & keys	Decrypted credentials, session tokens
Clipboard contents	Copy-paste history
Open files	Temporary files, memory-mapped executables
Command history	Terminal or PowerShell history
Malware indicators	Code injection, hooks, hidden processes



Memory Acquisition Tools

Tool	OS	Format	Notes
FTK Imager	Windows	.mem, .raw	Easy GUI-based
Belkasoft RAM Capturer	Windows	.raw	Portable & free
WinPMEM	Windows	.aff4, .raw	Supports encrypted memory
LiME	Linux	.lime	Kernel module for live capture
Magnet RAM Capture	Windows	.raw	Lightweight and reliable



Memory Analysis Tools

Tool	Purpose	Platform
Volatility	Comprehensive memory analysis (open-source)	Cross-platform
Rekall	Advanced analysis + live response	Cross-platform
Redline	GUI-based analysis by FireEye	Windows
Memdump + strings + grep	Basic CLI methods	Linux/Unix



Volatility

- **Volatility** is the most widely used **open-source memory forensics tool**
- It allows forensic analysts to extract digital artifacts from **memory dumps (RAM)** to detect **malware**, analyze **incidents**, recover **user activity**, and uncover evidence of compromise
- The **Volatility Foundation** helps keep **Volatility** going so that it may be used in perpetuity, **free** and open to all





Volatility (Cont)

- Volatility **Versions**

Version	Language	Status	Notes
Volatility 2	Python 2	Stable	More plugins, but outdated Python
Volatility 3	Python 3	Modern	Actively developed, better architecture



Volatility (Cont)

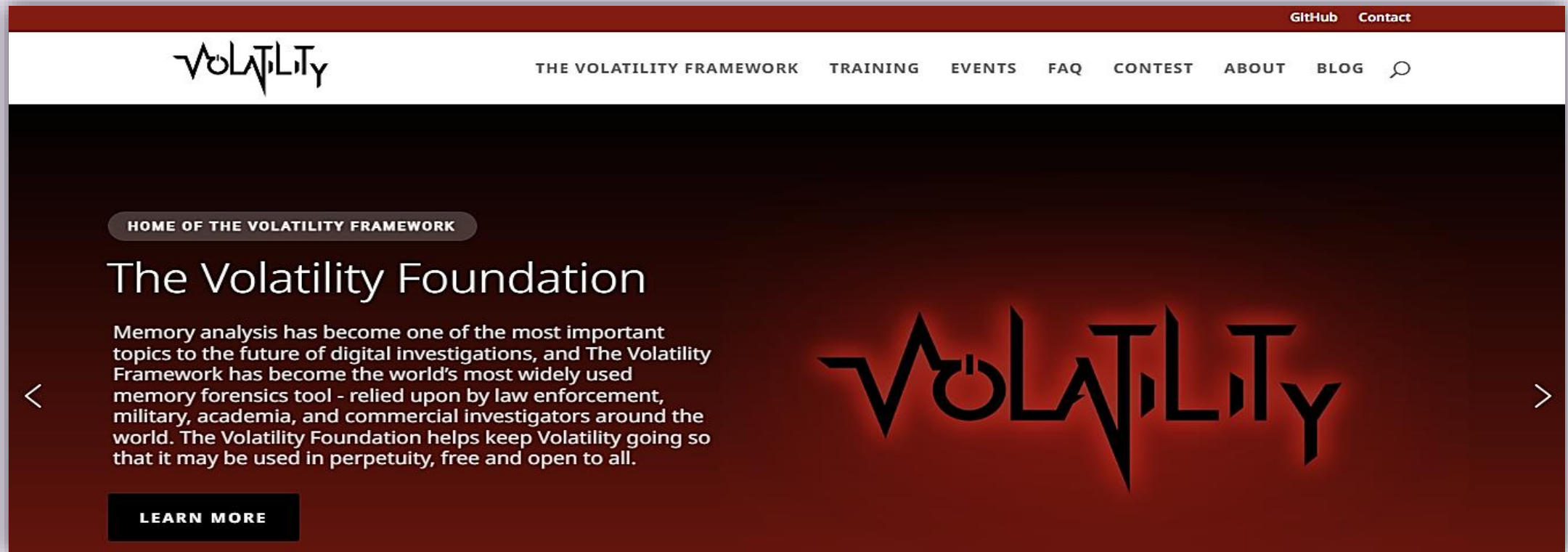
- Supported **Memory Dump Formats**
 - **.raw, .mem** – Raw memory
 - **.dd** – Disk dump containing RAM
 - **.lime** – Linux Memory Extractor format
 - **.hprof** – Java heap dump
 - **.vmem** – VMware memory
 - **.hiberfil.sys** – Windows hibernation file
 - **.pagefile.sys** – Windows page file





Volatility (Cont)

- **Official site:** <https://www.volatilityfoundation.org>





Volatility 3

- **Volatility 3** in **Github** (<https://github.com/volatilityfoundation/volatility3>)

The screenshot shows the GitHub repository page for Volatility 3. The browser address bar displays `github.com/volatilityfoundation/volatility3`. The repository name **Volatility 3: The volatile memory extraction framework** is prominently displayed. Below the title, a description states: "Volatility is the world's most widely used framework for extracting digital artifacts from volatile memory (RAM) samples. The extraction techniques are performed completely independent of the system being investigated but offer visibility into the runtime state of the system. The framework is intended to introduce people to the techniques and complexities associated with extracting digital artifacts from volatile memory samples and provide a platform for further work into this exciting area of research." A paragraph below mentions the 2019 rewrite by the Volatility Foundation. On the right sidebar, the "Contributors" section shows 73 contributors with a grid of avatars and a link to "+ 59 contributors". The "Languages" section features a horizontal bar chart showing Python at 99.9% and Shell at 0.1%.

github.com/volatilityfoundation/volatility3

README License

Volatility 3: The volatile memory extraction framework

Volatility is the world's most widely used framework for extracting digital artifacts from volatile memory (RAM) samples. The extraction techniques are performed completely independent of the system being investigated but offer visibility into the runtime state of the system. The framework is intended to introduce people to the techniques and complexities associated with extracting digital artifacts from volatile memory samples and provide a platform for further work into this exciting area of research.

In 2019, the Volatility Foundation released a complete rewrite of the framework, Volatility 3. The project was intended to address many of the technical and performance challenges associated with the original code base that became apparent over the previous 10 years. Another benefit of the rewrite is that Volatility 3 could be released under a custom license that was more aligned with the goals of the Volatility community, the Volatility Software License (VSL). See the [LICENSE](#) file for more details.

Contributors 73

+ 59 contributors

Languages

Python 99.9% Shell 0.1%



Volatility 3 (Cont)

■ Installation (Volatility 3 - Python 3)

#Kali Linux Command

```
pip install volatility3
```

#Clone from GitHub

```
git clone https://github.com/volatilityfoundation/volatility3  
cd volatility3
```

■ Basic Syntax

```
python3 vol.py -f [memory_image] [plugin] [options]
```




Volatility 3 (Cont)

- Common **Windows Plugins** for **Volatility 3**

Plugin	Description
windows.info	Show OS and kernel info
windows.pslist	List running processes (from EPROCESS)
windows.pstree	Show processes in tree format
windows.psscan	Scan for hidden/unlinked processes
windows.cmdline	Get command-line args of processes
windows.environ	Environment variables of processes



Volatility 3 (Cont)

- Common **Windows Plugins** for **Volatility 3** (Cont)

Plugin	Description
windows.getsids	Display SIDs linked to processes
windows.malfind	Detect injected/injected code in processes
windows.dlllist	List DLLs loaded by each process
windows.handles	List open handles (files, keys, mutexes)
windows.netscan	List open network connections (TCP/UDP)
windows.ldrmodules	Inspect loaded/unloaded modules (PEB)



Volatility 3 (Cont)

- Common **Windows Plugins** for **Volatility 3** (Cont)

Plugin	Description
windows.memdump	Dump memory of specific process
windows.driverscan	Scan for kernel drivers in memory
windows.registry.hivelist	List registry hives in memory
windows.registry.printkey	Display contents of a registry key
windows.hashdump	Extract LM/NTLM hashes from SAM
windows.vadinfo	Display Virtual Address Descriptors (VAD)



Volatility 3 (Cont)

- Common **Windows Plugins** for **Volatility 3** (Cont)

Plugin	Description
windows.bigpools	List kernel memory big pool allocations
windows.verinfo	Show version info for PE headers (DLL/EXE)
windows.sockets	Alternative network scanner
windows.clipboard	Extract clipboard contents



Volatility 3 (Cont)

- Usage **Command** in **Windows** for **Volatility 3**

#Detect OS Information

```
python3 vol.py -f memory.raw windows.info
```

#List Processes

```
python3 vol.py -f memory.raw windows.pslist
```

#Find Hidden or Suspicious Processes

```
python3 vol.py -f memory.raw windows.psscan
```

```
python3 vol.py -f memory.raw windows.malfind
```

#Dump Process Memory

```
python3 vol.py -f memory.raw windows.memdump --pid 1234 -D dumps/
```



Volatility 3 (Cont)

- Usage **Command** in **Windows** for **Volatility 3** (Cont)

#Extract Password Hashes

```
python3 vol.py -f memory.raw windows.hashdump
```

#Analyze Network Activity

```
python3 vol.py -f memory.raw windows.netscan
```

#Malware Detection

```
python3 vol.py -f infected.raw windows.malfind
```



Volatility 3 (Cont)

- Common **Linux Plugins** for **Volatility 3**

Plugin	Description
linux.pslist	List running processes (from task_struct)
linux.pstree	Show process tree relationships
linux.psscan	Scan memory for hidden/unlinked processes
linux.threads	List threads and their parent processes
linux.environ	Show environment variables for each task
linux.bash	Recover Bash command history



Volatility 3 (Cont)

- Common **Linux Plugins** for **Volatility 3** (Cont)

Plugin	Description
linux.lsof	List open files per process
linux.tty_check	Inspect TTYs and pseudo-terminals
linux.elfs	Locate and extract ELF binaries from memory
linux.malfind	Find suspicious memory regions (injected code)
linux.netstat	View network connections (TCP/UDP, ports)
linux.ifconfig	Show interface configurations (IP, MAC)



Volatility 3 (Cont)

- Common **Linux Plugins** for **Volatility 3** (Cont)

Plugin	Description
linux.route_cache	Print kernel routing cache entries
linux.proc_maps	Display process memory maps (VMA regions)
linux.arp	Show ARP cache from memory
linux.library_list	Show dynamic libraries used by processes
linux.mount	List mounted filesystems
linux.dmesg	Extract kernel ring buffer (dmesg) logs



Volatility 3 (Cont)

- Common **Linux Plugins** for **Volatility 3** (Cont)

Plugin	Description
linux.modules	List loaded kernel modules (like lsmod)
linux.check_afinfo	Detect AF_INET socket structure anomalies
linux.pidhashtable	Traverse PID hash table



Volatility 3 (Cont)

- Usage **Command** in **Linux** for **Volatility 3**

#View System Overview

```
python3 vol.py -f linux_mem.raw linux.pslist
```

#Detect Hidden Processes

```
python3 vol.py -f linux_mem.raw linux.psscan
```

#Analyze Bash History

```
python3 vol.py -f linux_mem.raw linux.bash
```

#Extract Loaded Kernel Modules

```
python3 vol.py -f linux_mem.raw linux.modules
```



Volatility 3 (Cont)

- Usage **Command** in **Linux** for **Volatility 3** (Cont)

```
#List Active Network Connections
```

```
python3 vol.py -f linux_mem.raw linux.netstat
```

```
#Inspect Injected Code
```

```
python3 vol.py -f linux_mem.raw linux.malfind
```



Volatility 3 (Cont)

■ Example of Result

```
(kali㉿kali)-[~/volatility3]
└─$ python3 vol.py -f command.dd windows.info
Volatility 3 Framework 2.26.2
/home/kali/volatility3/volatility3/framework/deprecation.py:105: FutureWarning: This plugin (PluginRequirement) has been renamed and will be removed in the first release after 2026-06-01. PluginRequirement is to be deprecated. Use VersionRequirement instead.
  warnings.warn(
Progress:      0.48           Reading file http://msdl.microsoft.com/download/symbols/ntkrnlp
Progress:      0.96F42D182C86ABEBEReading file http://msdl.microsoft.com/download/symbols/ntkrnlp
Progress:      1.44F42D182C86ABEBEReading file http://msdl.microsoft.com/download/symbols/ntkrnlp
Progress:      1.92F42D182C86ABEBEReading file http://msdl.microsoft.com/download/symbols/ntkrnlp
Progress:      2.40F42D182C86ABEBEReading file http://msdl.microsoft.com/download/symbols/ntkrnlp
Progress:      2.88F42D182C86ABEBEReading file http://msdl.microsoft.com/download/symbols/ntkrnlp
Progress:      3.36F42D182C86ABEBEReading file http://msdl.microsoft.com/download/symbols/ntkrnlp
```



Volatility 3 (Cont)

■ Example of Result (Cont)

Variable	Value
Kernel Base	0x804d7000
DTB	0x334000
Symbols file	file:///home/kali/volatility3/volatility3/symbols/windows/ntkrnlpa.pdb/7E4571CB945F42D182C86ABE8EA8E44D-1.json.xz
Is64Bit	False
IsPAE	True
layer_name	0 WindowsIntelPAE
memory_layer	1 FileLayer
KdDebuggerDataBlock	0x80545ce0
NTBuildLab	2600.xpsp_sp3_qfe.130704-0421
CSDVersion	3
KdVersionBlock	0x80545cb8
Major/Minor	15.2600
MachineType	332
KeNumberProcessors	1



Volatility 2

- **Volatility 2** in **GitHub** (<https://github.com/volatilityfoundation/volatility/releases>)

The screenshot shows the GitHub release page for Volatility 2.6.1. The browser address bar displays 'github.com/volatilityfoundation/volatility/releases'. On the left sidebar, the release date is 'Apr 10, 2024', the author is 'awalters', the version is '2.6.1', and the commit hash is 'c670176' with a green checkmark. A 'Compare' button is visible. The main content area features the title 'Volatility 2.6: December 2016' with a 'Latest' badge. Below the title is a bulleted list of features:

- Enhanced support for Windows 10 (including 14393.447)
- Added new profiles for recently patched Windows 7, Windows 8, and Server 2012
- Optimized page table enumeration and scanning algorithms, especially on 64-bit Windows 10
- Added support for carving Internet Explorer 10 history records
- Added support for memory dumps from the most recent VirtualBox version
- Updated the svcscan plugin to show FailureCommand (the command that runs when a service fails to start multiple times)



Volatility 2 (Cont)

- Download **Volatility 2.6 Standalone**

▼ Assets 7		
 hashes.txt	671 Bytes	Apr 10, 2024
 volatility-2.6.zip	3.99 MB	Apr 10, 2024
 volatility_2.6_lin64_standalone.zip	14.1 MB	Apr 10, 2024
 volatility_2.6_mac64_standalone.zip	18.9 MB	Apr 10, 2024
 volatility_2.6_win64_standalone.zip	14.8 MB	Apr 10, 2024
 Source code (zip)		Dec 19, 2018
 Source code (tar.gz)		Dec 19, 2018



Volatility 2 (Cont)

- Common **Windows Plugins** for **Volatility 2**

Plugin	Description
imageinfo	Auto-detect OS version and suggested profile
kdbgscan	Scan for KDBG structures (used when imageinfo fails)
pslist	List active processes (from EPROCESS linked list)
pstree	Display processes in a tree view
psscan	Scan memory for hidden/unlinked processes
dlllist	List DLLs loaded by each process



Volatility 2 (Cont)

- Common **Windows Plugins** for **Volatility 2** (Cont)

Plugin	Description
cmdscan	Extract in-memory command history (CMD.exe)
consoles	Reconstruct full command-line sessions
cmdline	Show command-line arguments for processes
getsids	Show security identifiers (SIDs) linked to processes
handles	List open handles per process (files, registry keys, etc.)
svcscan	Show Windows services running in memory



Volatility 2 (Cont)

- Common **Windows Plugins** for **Volatility 2** (Cont)

Plugin	Description
malfind	Detect injected code in memory (malware/hooking)
ldrmodules	Find stealthy injected modules (via PEB/linked list comparison)
modules	List loaded kernel drivers
modscan	Scan for kernel modules (useful for rootkits)
netscan	List network sockets and connections (Win7+)



Volatility 2 (Cont)

- Common **Windows Plugins** for **Volatility 2** (Cont)

Plugin	Description
connections	Show TCP endpoints (XP, 2003)
connscan	Scan memory for network connection remnants
hashdump	Dump password hashes from memory (NTLM/LM/SAM)
hivelist	List virtual addresses of loaded registry hives
printkey	Dump Windows registry key contents
filescan	Scan memory for FILE_OBJECTs (open files)



Volatility 2 (Cont)

- Common **Windows Plugins** for **Volatility 2** (Cont)

Plugin	Description
clipboard	Recover clipboard content
shellbags	Recover Windows Explorer folder access history
mftparser	Parse MFT entries recovered from memory
timeliner	Create a timeline of memory-based system events
apihooks	Detect userland API hooks (indicates malware)



Volatility 2 (Cont)

- Usage **Command** in **Windows** for **Volatility 2**

#Identify Memory Profile

```
python2 vol.py -f memory.raw imageinfo
```

#List Processes and Detect Hidden Ones

```
python2 vol.py -f memory.raw --profile=Win7SP1x64 pslist
```

```
python2 vol.py -f memory.raw --profile=Win7SP1x64 psscan
```

#Detect Malware Injection

```
python2 vol.py -f memory.raw --profile=Win7SP1x64 malfind
```

#Extract Password Hashes

```
python2 vol.py -f memory.raw --profile=Win7SP1x64 hashdump
```




Volatility 2 (Cont)

- Usage **Command** in **Windows** for **Volatility 2** (Cont)

```
#View Network Connections
```

```
python2 vol.py -f memory.raw --profile=Win7SP1x64 netscan
```

```
#Recovering data from the clipboards
```

```
python2 vol.py -f memory.raw --profile=Win7SP1x64 clipboard
```

```
#Scan memory for FILE_OBJECTs (open files)
```

```
python2 vol.py -f memory.raw --profile=Win7SP1x64 filescan
```



Volatility 2 (Cont)

- **Linux Plugins for Volatility 2:** You must have a **profile** corresponding to the **exact kernel version** of the **memory dump**

```
$ uname -a
Linux ubuntu 4.15.0-142-generic #146-Ubuntu SMP x86_64 GNU/Linux
```

- **Download a Volatility Profile for Linux from GitHub** (first options)

 gleeda	Merge pull request #71 from dfir-alvin/master	1c6bde8 · 6 years ago	🕒 125 Commits
 Linux	Merge pull request #71 from dfir-alvin/master		6 years ago
 Mac	Merge pull request #71 from dfir-alvin/master		6 years ago

<https://github.com/volatilityfoundation/profiles>



Volatility 2 (Cont)

- Build a Volatility Profile for Linux (second options)

```
$ sudo apt install dwarfdump build-essential linux-headers-$(uname -r)
$ git clone https://github.com/volatilityfoundation/volatility.git
$ cd volatility/tools/linux
$ make
```

This creates a .zip file in **volatility/tools/linux** to copy into the **volatility/plugins/overlays/linux** directory



Volatility 2 (Cont)

- Common **Linux Plugins** for **Volatility 2**

Plugin	Description
linux_pslist	List processes from task_struct
linux_pstree	Process tree view
linux_psaux	List PID, command-line, and UID
linux_psscan	Scan for hidden/unlinked processes
linux_proc_maps	Memory map of process (VMA regions)
linux_enumerate_files	Show open files by processes



Volatility 2 (Cont)

- Common **Linux Plugins** for **Volatility 2** (Cont)

Plugin	Description
linux_lsof	List open files (like lsof)
linux_netstat	Network connections from kernel
linux_ifconfig	Network interface details
linux_arp	Display ARP cache
linux_mount	Mounted filesystems in memory
linux_bash	Extract Bash history



Volatility 2 (Cont)

- Common **Linux Plugins** for **Volatility 2** (Cont)

Plugin	Description
linux_library_list	Shared library usage in processes
linux_check_afinfo	Validate AF_INET socket structures
linux_hidden_modules	Reveal kernel modules not in module list
linux_kernel_open_files	Files opened by the kernel
linux_threads	Thread information per task
linux_tmpfs	Analyze tmpfs in memory



Volatility 2 (Cont)

- Usage **Command** in **Linux** for **Volatility 2**

#Identify Correct Profile

```
python2 vol.py -f linux.raw --info | grep Linux
```

```
python2 vol.py -f linux.raw --profile=LinuxUbuntu_4_15_0_142x64  
linux_pslist
```

#List Running Processes

```
python2 vol.py -f linux.raw --profile=LinuxUbuntu_4_15_0_142x64  
linux_psaux
```

#View Bash History

```
python2 vol.py -f linux.raw --profile=LinuxUbuntu_4_15_0_142x64  
linux_bash
```



Volatility 2 (Cont)

- Usage **Command** in **Linux** for **Volatility 2** (Cont)

```
#Detect Hidden Processes
```

```
python2 vol.py -f linux.raw --profile=LinuxUbuntu_4_15_0_142x64  
linux_psscan
```

```
#Network and Interface Info
```

```
python2 vol.py -f linux.raw --profile=LinuxUbuntu_4_15_0_142x64  
linux_netstat
```

```
#Display network interface configuration (IP/MAC)
```

```
python2 vol.py -f linux.raw --profile=LinuxUbuntu_4_15_0_142x64  
linux_ifconfig
```




Volatility 2 (Cont)

- Usage **Command** in **Linux** for **Volatility 2** (Cont)

```
#Inspect open files and libraries
```

```
python2 vol.py -f linux.raw --profile=LinuxUbuntu_4_15_0_142x64  
linux_lsof
```

```
python2 vol.py -f linux.raw --profile=LinuxUbuntu_4_15_0_142x64  
linux_library_list
```



Volatility 2 (Cont)

■ Example of Result

```
C:\My Software\volatility>volatility -f command.dd imageinfo
Volatility Foundation Volatility Framework 2.6
INFO      : volatility.debug      : Determining profile based on KDBG search...
          Suggested Profile(s) : WinXPSP2x86, WinXPSP3x86 (Instantiated with WinXPSP2x86)
          AS Layer1           : IA32PagedMemoryPae (Kernel AS)
          AS Layer2           : FileAddressSpace (C:\My Software\volatility\command.dd)
          PAE type            : PAE
          DTB                  : 0x334000L
          KDBG                 : 0x80545ce0L
          Number of Processors : 1
          Image Type (Service Pack) : 3
          KPCR for CPU 0       : 0xffdff000L
          KUSER_SHARED_DATA    : 0xffdf0000L
          Image date and time   : 2020-06-10 10:07:13 UTC+0000
          Image local date and time : 2020-06-10 17:07:13 +0700

C:\My Software\volatility>
```



Volatility 2 (Cont)

■ Example of Result (Cont)

```
C:\My Software\volatility>volatility -f command.dd --profile=WinXPSP2x86 cmdscan
Volatility Foundation Volatility Framework 2.6
*****
CommandProcess: csrss.exe Pid: 524
CommandHistory: 0x504d88 Application: cmd.exe Flags: Allocated, Reset
CommandCount: 1 LastAdded: 0 LastDisplayed: 0
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x368
Cmd #0 @ 0x501fa0: cd fg_2547890fde25dac8ca341098d8dae6cf
*****
CommandProcess: csrss.exe Pid: 524
CommandHistory: 0x11d3660 Application: dd.exe Flags: Allocated
CommandCount: 0 LastAdded: -1 LastDisplayed: -1
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x6bc
*****
```



Volatility 2 (Cont)

- **Special Example for Memdump from Google CTF**
 - The Google Capture The Flag (CTF) was run on the **29-30 April 2016** for the forensics challenge **"For1"** which was worth **100 points**
 - The **200 MB** of **dump1.raw.lzma** file was provided
- **Write-up** for **"For1"**: <https://www.rootusers.com/google-ctf-2016-forensic-for1-write-up/>

```
root@kali:~# lzma -d dump1.raw.lzma
root@kali:~# file dump1.raw
dump1.raw: ELF 64-bit LSB core file x86-64, version 1 (SYSV)
root@kali:~/volatility-master# ./vol.py -f /root/dump1.raw pslist --
profile=Win10x64 pstree
```



Volatility 2 (Cont)

Volatility Foundation Volatility Framework 2.5

Offset (V)	Name	PID	PPID	Thds	Hnds	Sess	Wow64	Start	Exit
0xfffffe00032553780	System	4	0	126	0	-----	0	2016-04-04 16:12:33	UTC+0000
0xfffffe0003389c040	smss.exe	268	4	2	0	-----	0	2016-04-04 16:12:33	UTC+0000
0xfffffe0003381b080	csrss.exe	344	336	8	0	0	0	2016-04-04 16:12:33	UTC+0000
0xfffffe000325ba080	wininit.exe	404	336	1	0	0	0	2016-04-04 16:12:34	UTC+0000
0xfffffe000325c7080	csrss.exe	412	396	9	0	1	0	2016-04-04 16:12:34	UTC+0000
0xfffffe00033ec6080	winlogon.exe	460	396	2	0	1	0	2016-04-04 16:12:34	UTC+0000
0xfffffe000347c1080	sshd.exe	2028	1976	3	0	0	0	2016-04-04 16:12:36	UTC+0000
0xfffffe00033e00780	svchost.exe	1772	484	3	0	0	0	2016-04-04 16:12:37	UTC+0000
...									
0xfffffe00033f1f780	sihost.exe	92	796	10	0	1	0	2016-04-04 16:12:37	UTC+0000
0xfffffe0003259b3c0	taskhostw.exe	1532	796	9	0	1	0	2016-04-04 16:12:37	UTC+0000
0xfffffe000339d4340	NisSrv.exe	2272	484	6	0	0	0	2016-04-04 16:12:38	UTC+0000
...									
0xfffffe00034b0f780	mspaint.exe	4092	2336	3	0	1	0	2016-04-04 16:13:21	UTC+0000
0xfffffe00034ade080	svchost.exe	628	484	1	0	1	0	2016-04-04 16:14:43	UTC+0000
0xfffffe0003472b080	notepad.exe	2012	2336	1	0	1	0	2016-04-04 16:14:49	UTC+0000
0xfffffe000349e4780	WmiPrvSE.exe	3032	580	6	0	0	0	2016-04-04 16:16:37	UTC+0000
0xfffffe000349285c0	taskhostw.exe	332	796	10	0	1	0	2016-04-04 16:17:40	UTC+0000



Volatility 2 (Cont)

- Dump the **mspaint.exe** process (**PID 4092**) and

```
root@kali:~/volatility-master# ./vol.py -f /root/dump1.raw --profile=Win10x64
memdump -p 4092 -D /root/
Volatility Foundation Volatility Framework 2.5
*****
Writing mspaint.exe [ 4092] to 4092.dmp
```

- Renamed **4092.dmp** file to **4092.data** and analyze by **Gimp**
 - **GIMP** is a **free, open-source image editor** often seen as a powerful alternative to **Adobe Photoshop**
 - **GIMP** is used widely for **digital forensics**, graphic design, photo retouching, and image manipulation



Volatility 2 (Cont)

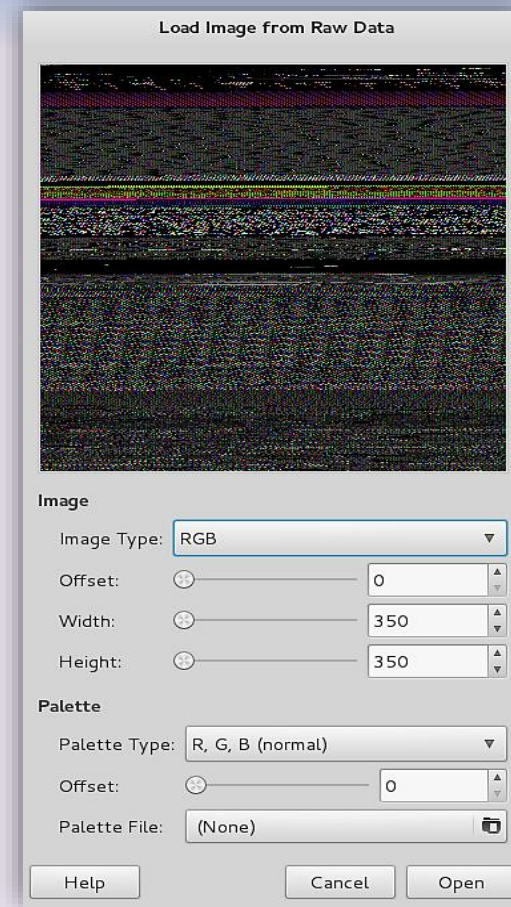
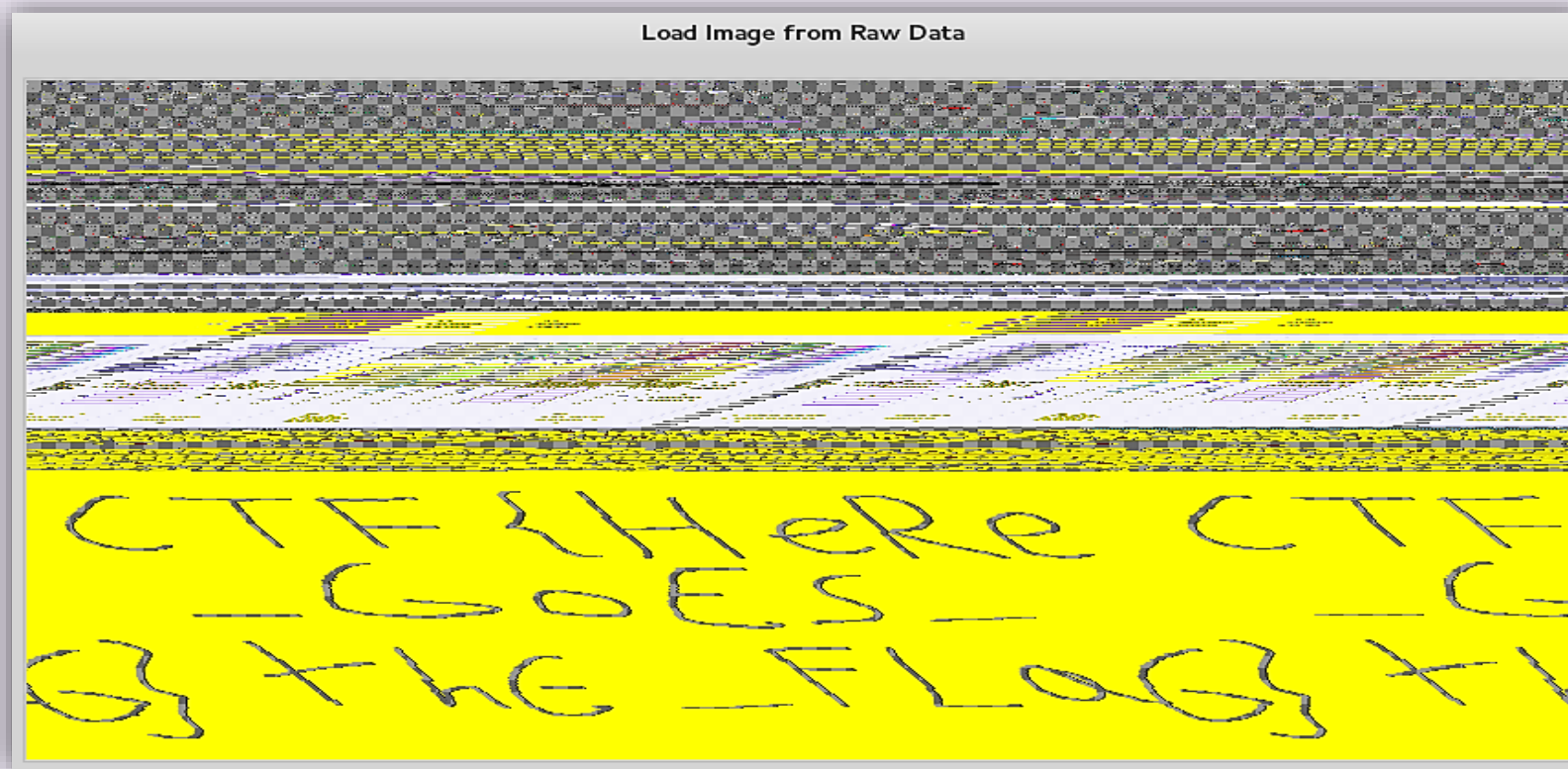
- **GIMP download** from <https://www.gimp.org/downloads>





Volatility 2 (Cont)

- This resulted in the flag of **CTF{HeRe_GoES_thE_FLaG}**



Network Forensics

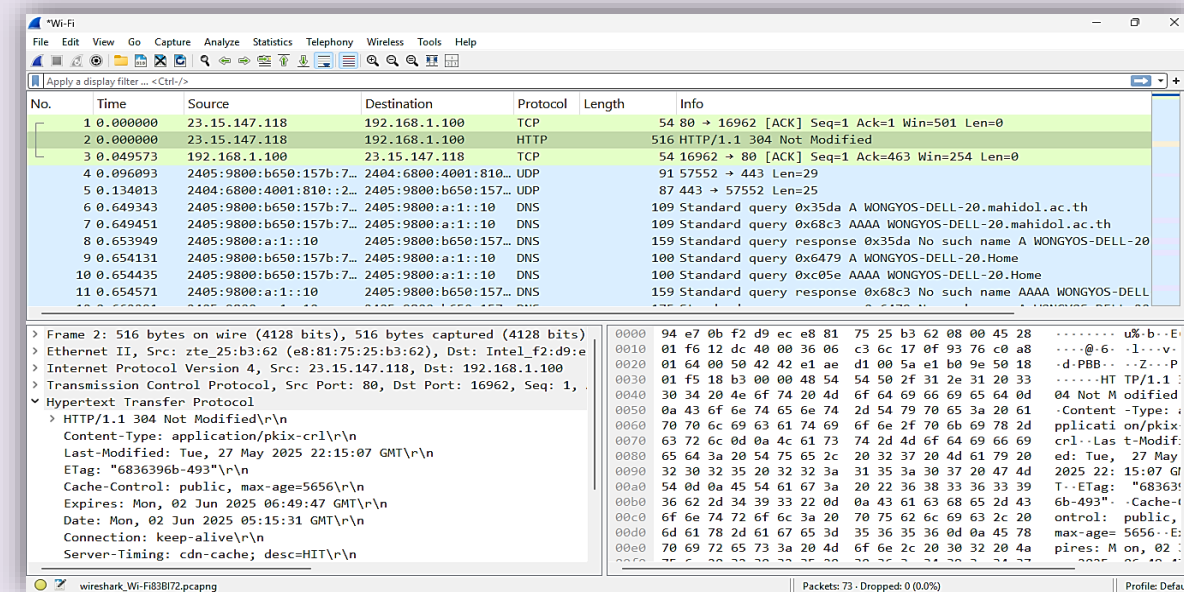
5

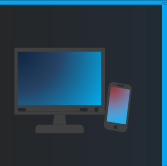




Network Forensics Overviews

- **Network Forensics** is a sub-discipline of **digital forensics** focused on **capturing, recording, and analyzing network traffic** to investigate and respond to security incidents, data breaches, intrusions, or policy violations
- It plays a crucial role in:
 - **Incident response** and threat hunting
 - **Cybercrime** investigations
 - Attribution and timeline **reconstruction**
 - **Malware** command-and-control detection





Components of Network Forensics

- Core **Components** of **Network Forensics**

Component	Description
Packet Capture (PCAP)	Raw network traffic data (captured using Wireshark or tcpdump)
Log Analysis	Firewall, proxy, IDS/IPS, router, and server logs
Flow Data	NetFlow, IPFIX (aggregated traffic flows instead of full packets)
Protocol Analysis	Understanding protocols like HTTP, DNS, FTP, SMTP, TCP/UDP
Timeline Correlation	Reconstructing attack sequence over time
Artifact Recovery	Extracting files, credentials, malware, or commands from captured traffic





Network Artifacts

■ **Artifacts** Extracted in Network Forensics

Artifact Type	Example
Credentials	Cleartext passwords, session tokens
File Transfers	Downloaded/uploaded files (HTTP, FTP, SMB)
Domains/IPs	Command and control (C2), suspicious endpoints
Malware Comms	Beaconing, tunneling, payload delivery
Network Scans	Port scanning, banner grabbing
Timestamps	Session creation, authentication times



Network Forensics Tools

Tool	Use Case
Wireshark	GUI packet capture and protocol analysis
TCPdump	CLI-based packet sniffer
NetworkMiner	PCAP analysis, OS fingerprinting, file extraction
TShark	Terminal version of Wireshark
Bro/Zeek	Real-time network traffic analysis and logging
Suricata	IDS/IPS with PCAP analysis capabilities
Arkime (Moloch)	PCAP indexing and web interface for investigation



Network Forensics Tools (Cont)

Tool	Use Case
Xplico	Extracts emails, HTTP content, VoIP, and FTP files from PCAP
NetFlow Tools	nfdump, nfsen for flow analysis
Security Onion	Network security monitoring and forensics platform



Wireshark

- **Wireshark** is a powerful, **free**, **open-source** tool used for **capturing**, **inspecting**, and **analyzing network traffic** in real time
- Originally named **Ethereal**, the project was renamed **Wireshark** in **May 2006** due to trademark issues
- Wireshark is **cross-platform**, it runs on **Linux**, **macOS**, **BSD**, **Solaris**, some other Unix-like operating systems, and **Windows**



WIRESHARK



Wireshark (Cont)

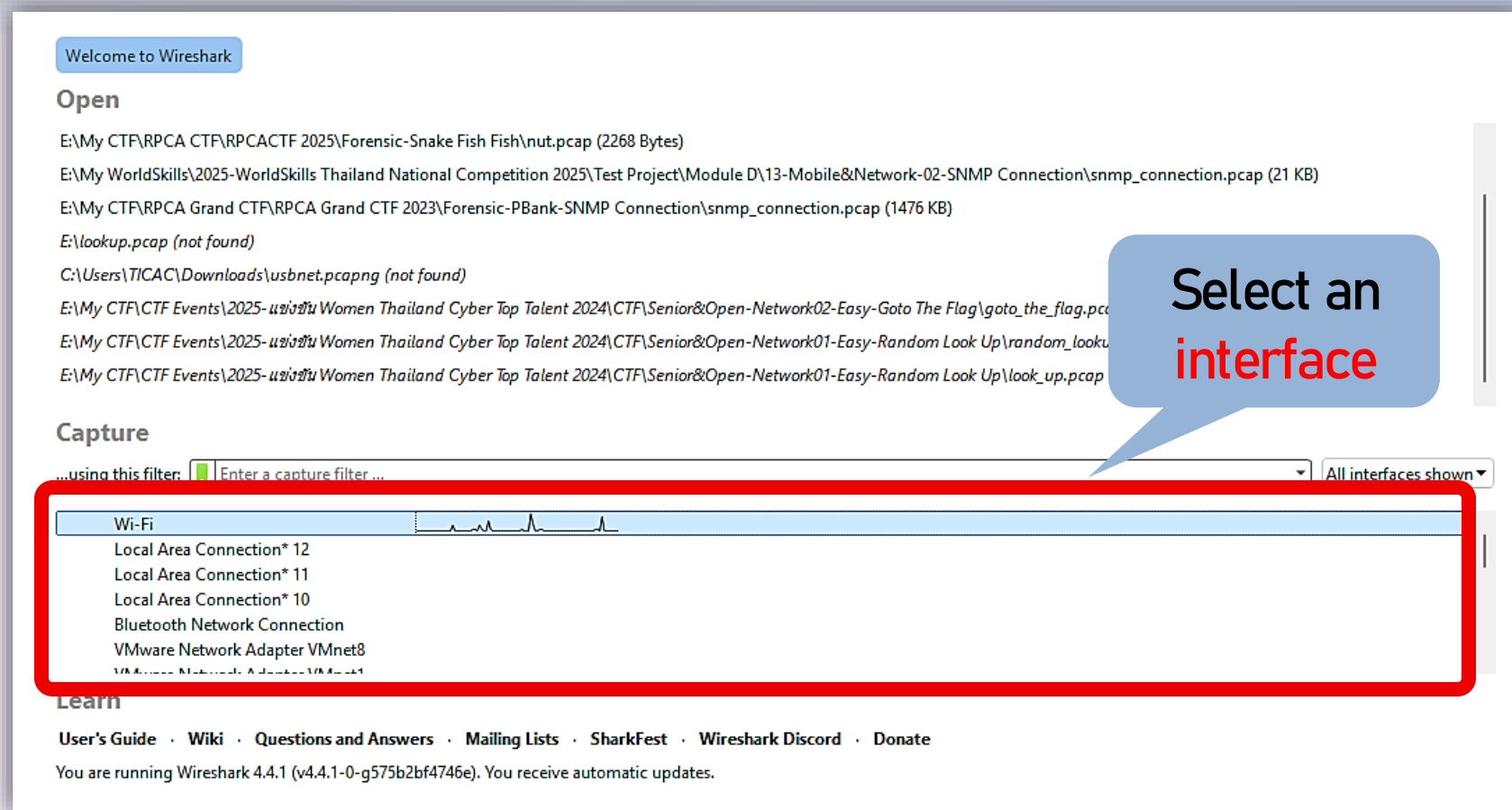
- **Wireshark download** from: <https://www.wireshark.org/>





Wireshark (Cont)

- Start a Capture
 - Open Wireshark:
Select an interface (e.g., **Wi-Fi**, **eth0**, **wlan0**)
 - Click the shark fin icon to begin capture





Wireshark (Cont)

■ Wireshark Panes

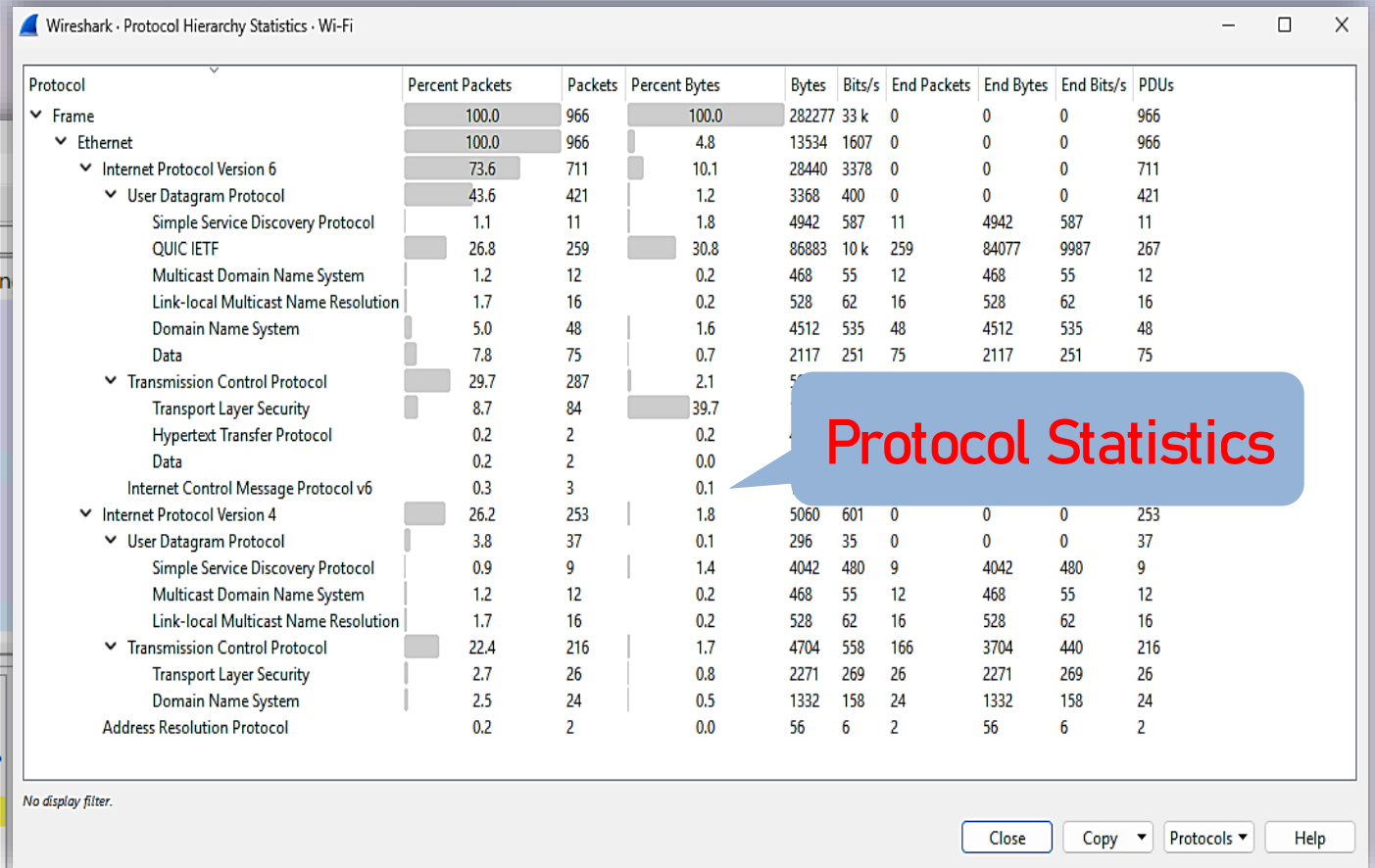
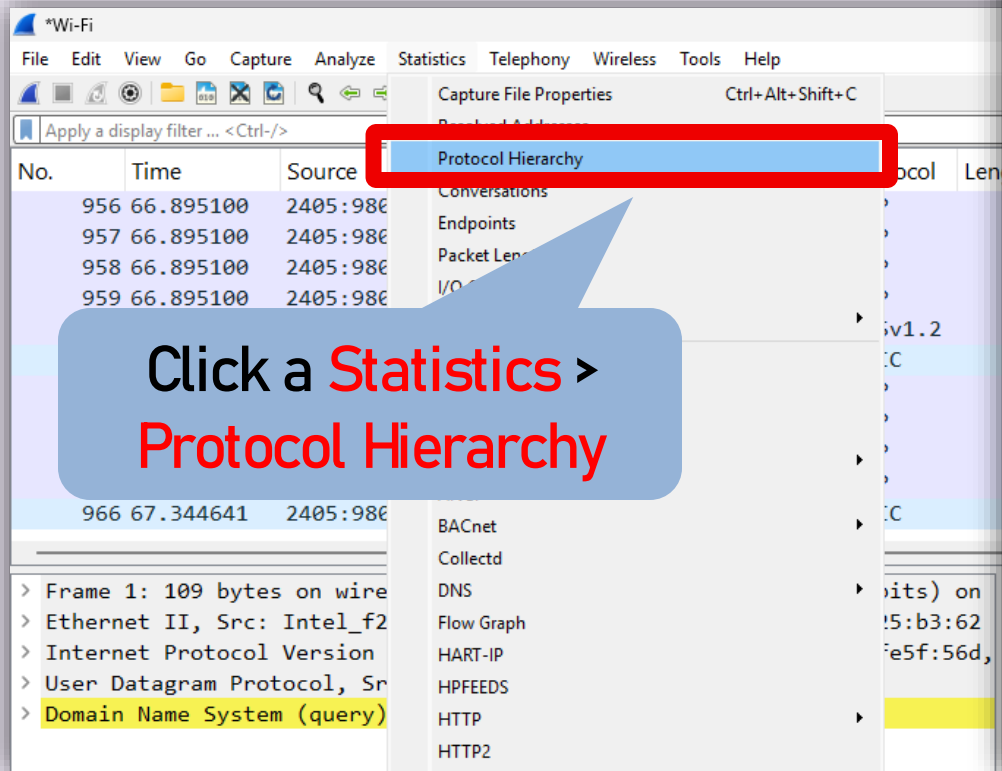
- **Packet List:** provides a high-level overview of all packets
- **Packet Details:** shows a hierarchical view of the protocol layers
- **Packet Bytes:** displays the raw data (hex dump)

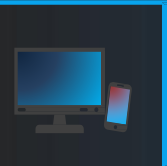
The image shows the Wireshark interface with three panes highlighted by red boxes and blue callout labels:

- Packet List:** A table showing a list of captured packets. The columns are No., Time, Source, Destination, Protocol, Length, and Info. The table contains 12 rows of data, including ARP, HTTP, and TCP packets.
- Packet Details:** A hierarchical view of the selected packet (No. 1502). It shows the protocol stack: Ethernet II, Internet Protocol Version 4, Transmission Control Protocol, and Hypertext Transfer Protocol. The details are expanded for the Hypertext Transfer Protocol, showing the Media Type.
- Packet Bytes:** A hex dump of the selected packet (No. 1502). It shows the raw data in hexadecimal and ASCII format.

Wireshark (Cont)

■ Check Protocol Statistics





Wireshark (Cont)

- Apply **Filters**: input **filter** and **follow** the **related packets**

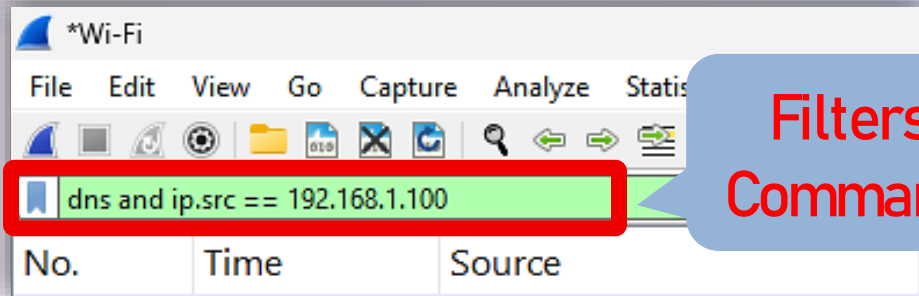
The screenshot shows the Wireshark interface with the following details:

- Filter Bar:** Contains the text `http`.
- Packet List:**

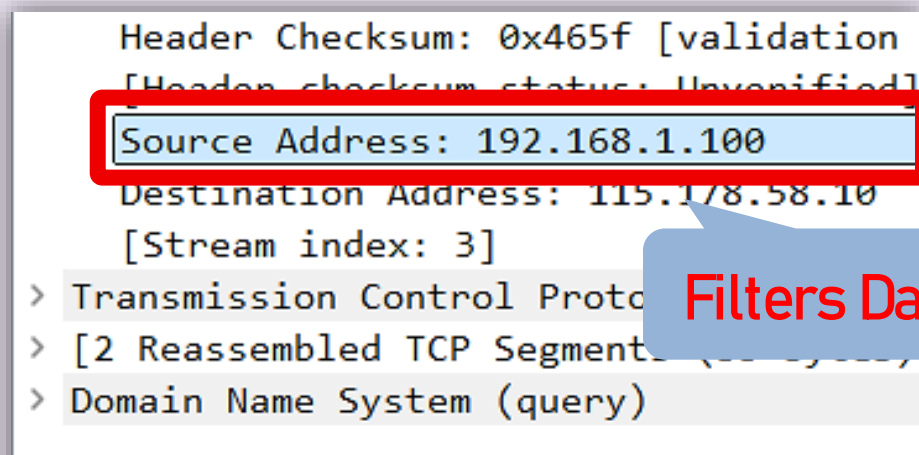
No.	Time	Source	Destination	Protocol	Length	Info
504	28.483713	2405:9800:b650:1574::7	2a04:4e42:69::684	HTTP	361	GET /msdownload/update/v3/static/trustedre/en/disallowedcertstl.cab?
506	28.491974	2a04:4e42:69::684	2405:9800:b650:1574::7	HTTP	276	HTTP/1.1 304 Not Modified
- Packet Details:** For packet 504, the details pane shows:
 - Frame 504: 361 bytes on wire (2888 bytes captured)
 - Ethernet II, Src: Intel_f2:d9:ec (98:f2:d9:ec:00:00:00), Dst: 2a04:4e42:69::684
 - Internet Protocol Version 6, Src: 2405:9800:b650:1574::7, Dst: 2a04:4e42:69::684
 - Transmission Control Protocol, Src Port: 5684, Dst Port: 80
 - Hypertext Transfer Protocol
- Context Menu:** Open over packet 506. The 'Follow' option is selected and highlighted in blue. A red box highlights the 'Follow' option in the context menu.



Wireshark (Cont)



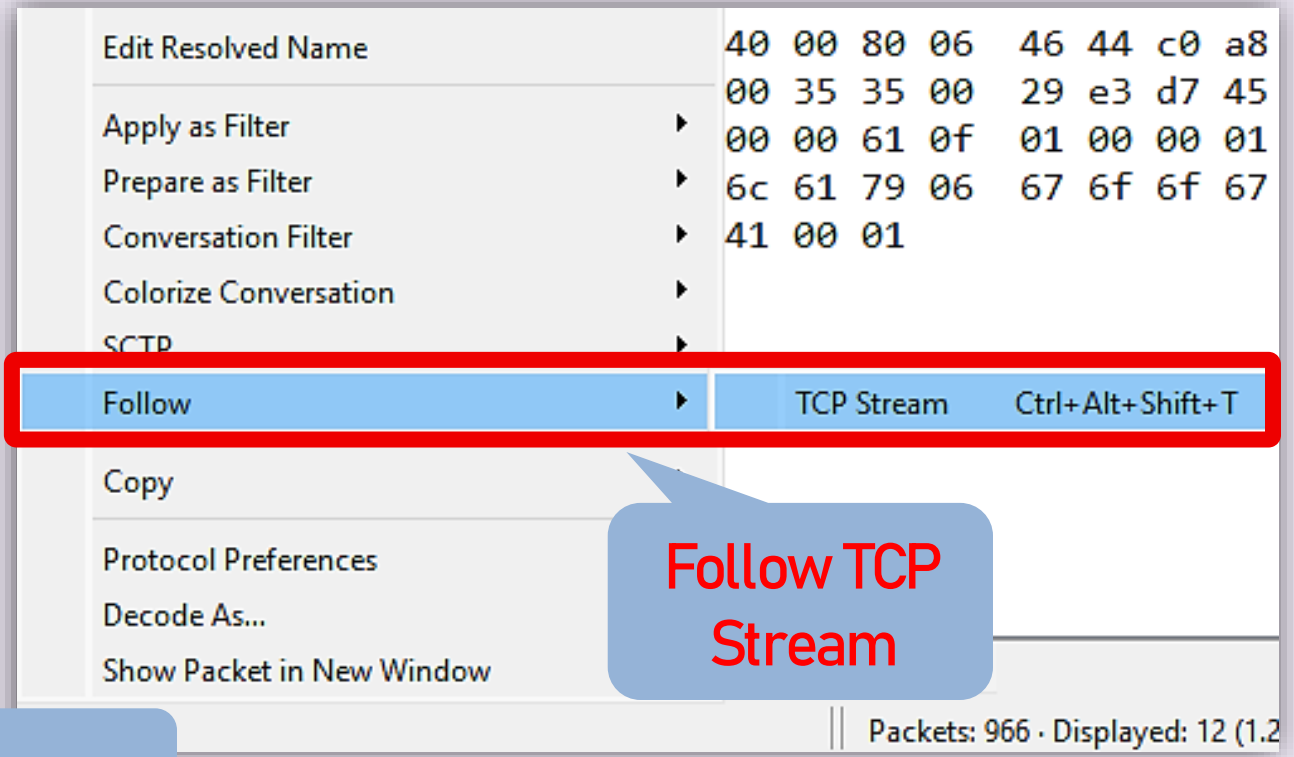
Filters
Command



Filters
Data



Filters
Parameter



Follow
TCP Stream





Wireshark (Cont)

■ Reconstruct Files

- File > **Export Objects** > HTTP/SMB/DICOM
- Save files transferred over unencrypted protocols

The screenshot shows the Wireshark application window with the 'File' menu open. The 'Export Objects' option is highlighted, and its submenu is visible, showing 'HTTP...' selected. A blue speech bubble with the text 'Export Objects from HTTP' points to the 'HTTP...' option. The background shows a packet capture of an HTTP GET request.

Destination	Protocol	Length	Info
2405:9800:a:1::10	DNS	103	Standard query 0x030e AAAA c
2405:9800:b650:157...	DNS	255	Standard query response 0x73
2405:9800:b650:157...	DNS	267	Standard query response 0x03
2a04:4e42:69::684	TCP	86	18947 → 80 [SYN] Seq=0 Win=6
2405:9800:b650:157...	TCP	86	80 → 18947 [SYN, ACK] Seq=0
2a04:4e42:69::684	TCP	74	18947 → 80 [ACK] Seq=1 Ack=1
2a04:4e42:69::684	HTTP	361	GET /msdownload/update/v3/st
2405:9800:b650:157...	TCP	74	80 → 18947 [ACK] Seq=1 Ack=2
2405:9800:b650:157...	HTTP	5	HTTP/1.1 304 Not Modified
2404:680...			Protected Payload (KP0), DCI
2a04:4e4...			18947 → 80 [ACK] Seq=288 Ack
4001:810::2...			Protected Payload (KP0)



Wireshark (Cont)

- **Wireshark User's Guide** by Richard Sharpe, Ed Warnicke, Ulf Lamping





NetworkMiner

- **NetworkMiner** is a **open source** and powerful **network forensics analysis tool (NFAT)** for **Windows** designed to analyze **PCAP** files or sniff **live traffic**
- **NetworkMiner** can extract **network artifacts**, such as files, images, emails and passwords, from captured network traffic in PCAP files
- **NetworkMiner** has, since the first release in **2007**, become a popular tool among incident response teams as well as **law enforcement**





NetworkMiner (Cont)

■ Key Features of NetworkMiner

Feature	Description
Host Identification	Maps IPs to MACs, hostnames, and OS fingerprints
File Extraction	Recovers transferred files (HTTP, FTP, SMB) directly from PCAP
Image Reconstruction	Extracts JPEGs, PNGs, and other media seen in network traffic
Credential Recovery	Parses usernames and passwords from unencrypted protocols
Session Reconstruction	Displays TCP/UDP conversations
Passive Analysis	Does not send packets (great for stealth forensics)



NetworkMiner (Cont)

- **NetworkMiner download:** <https://www.netresec.com/?page=NetworkMiner>

The screenshot shows the Netresec website. At the top left is the Netresec logo, which consists of a stylized 'N' inside a circle followed by the word 'ETRESEC'. To the right of the logo is the tagline 'Experts in network security monitoring and network forensics'. Below this is a dark navigation bar with white text links: 'NETRESEC | Products | Training | Resources | Blog | About Netresec'. Under the navigation bar, the breadcrumb trail 'NETRESEC » Products » NetworkMiner' is visible. The main heading 'NetworkMiner' is in a large, bold, serif font. Below the heading, a paragraph describes the tool: 'NetworkMiner is an open source network forensics tool that extracts artifacts, such as files, images, emails and passwords, from captured network traffic in PCAP files.' To the right of the text is a large, stylized purple graphic that resembles a semi-circle or a stylized 'N'.

NETRESEC Experts in network security monitoring and network forensics

NETRESEC | Products | Training | Resources | Blog | About Netresec

NETRESEC » Products » NetworkMiner

NetworkMiner

NetworkMiner is an open source network forensics tool that extracts artifacts, such as files, images, emails and passwords, from captured network traffic in PCAP files.



NetworkMiner (Cont)

NetworkMiner 2.7.3

File Tools Help

— Select a network adapter in the list —

Anomalies

Hosts (179) Files (287) Images (8) Messages Credentials (4) Sessions (242) DNS (1497) Parameters (4074) Keywords

Sort Hosts On: IP Address (ascending) Sort and Refresh

Case Panel

Filename MD5

snort.log... f9b239b...

192.168.0.1

192.168.0.2 (Linux)

192.168.0.50

IP: 192.168.0.50

MAC: 00606E42B635

NIC Vendor: DAVICOM SEMICONDUCTOR, INC.

MAC Age: 1998-04-22

Hostname:

OS: Unknown

TTL: 1 (distance: 31)

Open TCP Ports:

Sent: 205 packets (15,400 Bytes), 0.00% cleartext (0 of 0 Bytes)

Received: 0 packets (0 Bytes), 0.00% cleartext (0 of 0 Bytes)

Incoming sessions: 0

Outgoing sessions: 0

Host Details

Queried DNS names : _ipps._tcp.local

UPnP field : HOST: 239.255.255.250 : 1900

UPnP field : MAN: "ssdp: discover"

UPnP field : M-SEARCH * HTTP/1.1

UPnP field : MX : 2

UPnP field : ST: roku : ecp

192.168.0.51 (Windows)

IP: 192.168.0.51

MAC: ECF4BB4FB245

NIC Vendor: Dell Inc.

MAC Age: 2013-11-08

NetworkMiner 2.0

File Tools Help

— Select a network adapter in the list —

Keywords Anomalies

Hosts (129) Files (131) Images (33) Messages Credentials (2) Sessions (113) DNS (271) Parameters (1199)

Filter keyword:

☐ Case sensitive ExactPhrase Clear Apply

D. port	Protocol	Filename	Extension	Size	Details
TCP 53130	TlsCertificate	nr-data.net.cer	cer	1 203 B	TLS Certificate: C
TCP 53130	TlsCertificate	GeoTrust SSL CA - G2.cer	cer	1 117 B	TLS Certificate: C
TCP 53130	TlsCertificate	GeoTrust Global CA.cer	cer	897 B	TLS Certificate: C
TCP 53138	HttpGetNormal	index.html[2].ocsp-response	ocsp-response	1 455 B	gb.symcd.com/
TCP 53139	HttpGetChunked	index.html	html	86 958 B	www.meetup.com
TCP 53142	HttpGetNormal	almond.min.js	javascript	2 758 B	static2.meetupsta
TCP 53140	HttpGetNormal	meetup_query_ui.css	css	6 725 B	static2.meetupsta
TCP 53144	HttpGetNormal	client.min.js	javascript	3 692 B	static2.meetupsta
TCP 53145	HttpGetNormal	infoWidget.min.js	javascript	20 639 B	static2.meetupsta
TCP 53151	HttpGetNormal	groupMetadata.min.js	javascript	2 409 B	static1.meetupsta
TCP 53149	HttpGetNormal	mt-twoButtonCTA-testimonial.css	css	445 B	static1.meetupsta
TCP 53147	HttpGetNormal	print.css	css	2 171 B	static1.meetupsta
TCP 53141	HttpGetNormal	meetup-modern.css	css	223 971 B	static2.meetupsta
TCP 53139	HttpGetNormal	index.html.6D1A30C1.css	css	5 582 B	www.meetup.com
TCP 53146	HttpGetNormal	whitney.css	css	83 455 B	static1.meetupsta
TCP 53150	HttpGetNormal	ghome.min.js	javascript	102 378 B	static1.meetupsta
TCP 53148	HttpGetNormal	chapterbase.css	css	165 101 B	static1.meetupsta
TCP 53143	HttpGetNormal	Meetup.Base.jquery.min.js	javascript	414 355 B	static2.meetupsta
TCP 53152	HttpGetNormal	thumb_156167702.jpeg	jpeg	2 611 B	photos3.meetupst
TCP 53156	HttpGetNormal	thumb_151699612.jpeg.PNG	PNG	2 571 B	photos3.meetupst

Case Panel

Filename MD5

snort.log... 2f301c2...

Reload Case Files

Live Sniffing Buffer Usage:



NetworkMiner (Cont)

- NetworkMiner run on Kali Linux

The screenshot shows a Kali Linux terminal window with the command `mono NetworkMiner.exe` being executed. The output indicates that GTK is not found due to a missing `LD_LIBRARY_PATH`. A blue-bordered box highlights the commands to install the Mono framework and run the application again: `$ sudo apt install mono-complete` and `$ mono NetworkMiner.exe`. In the background, the NetworkMiner 3.0 application is open, displaying a menu bar (File, Tools, Help) and a tabbed interface with categories like Hosts, Files, Images, Messages, Credentials, Sessions, DNS, Parameters, and Keywords. The 'Hosts' tab is active, showing a search filter, a sort dropdown set to 'IP Address (ascending)', and buttons for 'Clear', 'Apply', and 'Update'. A 'Case Panel' on the right contains a table with columns 'Filename' and 'MDS'. At the bottom, there is a 'Reload Case Files' button and a status bar showing 'Buffered Frames to Parse'.

```
(kali@kali)-[~/NetworkMiner_3-0]
$ mono NetworkMiner.exe
Gtk not found (missing LD_LIBRARY_PATH)

$ sudo apt install mono-complete
$ mono NetworkMiner.exe
```



TCPdump

- TCPdump is a **lightweight**, powerful **command-line packet analyzer** used to capture and inspect **live** and **real-time** network traffic or save it into **.pcap** files for later forensic analysis
- It's widely used by **cybersecurity professionals**, network engineers, and forensic analysts
- It's often used in **Linux** and **macOS**, and a similar tool, **WinDump**, is available for **Windows**





TCPdump (Cont)

■ Basic Syntax

```
$ sudo tcpdump [options] [filter expression]
```

```
root@kali:~# tcpdump -h
tcpdump version 4.99.5
libpcap version 1.10.5 (with TPACKET_V3)
OpenSSL 3.5.0 8 Apr 2025
64-bit build, 64-bit time_t
Usage: tcpdump [-AbdDefhHIJKlLnNOpqStuUvxX#] [ -B size ] [ -c count ] [--count]
               [ -C file_size ] [ -E algo:secret ] [ -F file ] [ -G seconds ]
               [ -i interface ] [ --immediate-mode ] [ -j tstamptype ]
               [ -M secret ] [ --number ] [ --print ] [ -Q in|out|inout ]
               [ -r file ] [ -s snaplen ] [ -T type ] [ --version ]
               [ -V file ] [ -w file ] [ -W filecount ] [ -y datalinktype ]
               [ --time-stamp-precision precision ] [ --micro ] [ --nano ]
               [ -z postrotate-command ] [ -Z user ] [ expression ]
```



TCPdump (Cont)

■ Common Options

Option	Description
-i <iface>	Capture from a specific interface (e.g., eth0)
-w <file.pcap>	Write output to PCAP file
-r <file.pcap>	Read packets from a PCAP file
-n	Don't resolve hostnames (faster)
-nn	Don't resolve hostnames or ports
-c <count>	Capture a specific number of packets



TCPdump (Cont)

- Common **Options** (Cont)

Option	Description
-v, -vv, -vvv	Increase verbosity levels
-X	Show hex + ASCII dump of packet contents
-A	Print packet contents as ASCII



TCPdump (Cont)

■ Practical Examples

#Capture HTTP Traffic to a File

```
sudo tcpdump -i eth0 port 80 -w http_traffic.pcap
```

#Capture 100 Packets of All Traffic

```
sudo tcpdump -i eth0 -c 100 -w capture.pcap
```

#Monitor DNS Queries in Real Time

```
sudo tcpdump -i eth0 port 53
```

#Inspect Live Packets in ASCII & Hex

```
sudo tcpdump -i wlan0 -X
```



TShark

- **TShark** is a network protocol analyzer and the **terminal-based** version of **Wireshark**, offering powerful packet capture and analysis capabilities directly from the **command line**
- It's ideal for automation, scripting, and remote analysis where a **GUI is not feasible**
- **TShark** will work much like **TCPdump**
- **TShark** manual page:
<https://www.wireshark.org/docs/man-pages/tshark.html>





TShark (Cont)

- Basic Syntax

```
tshark [options]
```

- Common TShark Options

Option	Description
-i <iface>	Specify network interface
-w <file.pcap>	Write output to a .pcap file
-r <file.pcap>	Read and analyze packets from a .pcap file
-c <count>	Capture only N packets



TShark (Cont)

■ Common TShark Options (Cont)

Option	Description
-f "<capture filter>"	BPF-style capture filter (like in tcpdump)
-Y "<display filter>"	Wireshark display filter
-T fields	Extract specific fields
-e <field>	Specify which field(s) to print
-E header=y	Print field names in the output
-V	Verbose output (all layers)
-X	Show packet bytes in hex and ASCII



TShark (Cont)

■ Example Commands

```
#Capture 100 packets and save to PCAP  
tshark -i eth0 -c 100 -w capture.pcap
```

```
#Read a PCAP and show all layers  
tshark -r capture.pcap -V
```

```
#Filter for HTTP requests  
tshark -r capture.pcap -Y "http.request"
```

```
#Extract fields (e.g., IPs and URLs)  
tshark -r capture.pcap -T fields -e ip.src -e ip.dst -e http.host -e  
http.request.uri -E header=y
```

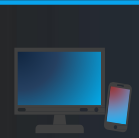
Steganography





Steganography Overviews

- **Steganography** is the practice of **concealing** a **message**, **file**, or **data** within another seemingly innocuous file — like an **image**, **audio**, **video**, or **document**
- Unlike encryption (which hides content), steganography hides the existence of content
- **Steganography** is the technique of **hiding secret data** within an ordinary, non-secret, file or message in order to avoid detection
- The word steganography comes from **New Latin** steganographia, which combines the **Greek words** steganós (στεγανός), meaning "**covered or concealed**", and -graphia (γραφή) meaning "**writing**"



Steganography Overviews (Cont)

- Common **File Types** Used for **Steganography**

File Type	Example Usage
.jpg, .png	Hide data in pixel bits (especially with LSB)
.wav, .mp3	Embed data in audio samples
.mp4, .avi	Use video frames or metadata
.docx, .pdf	Use whitespace, fonts, or invisible objects
.zip, .rar	Append hidden payload at end of archive



Steganography Tools

- Popular **Tools** for **Steganography**

Tool	Description
Steghide	CLI tool to hide/extract data in BMP/JPEG/WAV/AU
Stegseek	A lightning fast Steghide cracker
Stegsolve	GUI Java tool for color plane analysis on images
Stegdetect	Performs statistical tests to find if a stego tool was used
StegoVeritas	Python toolset for image-based steg challenges
StegoSuite	GUI for similar steghide-type functionality
OpenStego	GUI tool for LSB image-based steganography



Steganography Tools (Cont)

- Popular **Tools** for **Steganography** (Cont)

Tool	Description
MP3Stego	Steganographic tools to hide the information in MP3 files
AudioStego	Java code for hiding a JPG file inside a WAV file
JSteg	A package for hiding data inside JPEG files
ZSteg	Advanced tool for PNG/BMP LSB analysis (great for CTFs)
OutGuess	Hiding the image files in the common Netpbm and JPEG formats
Audacity & Sonic Visualiser	Software for recording and editing audio
Online Steganography	The tools for online steganography



Steghide

- Steghide is a **steganography program** that can hide data in various kinds of **image** and **audio** files.
- The original **color** and **frequencies** of data are **not changed** thus making the embedding resistant against first-order statistical tests
- Steghide supports for **JPEG**, **BMP**, **WAV** and **AU** files
- How to use **Steghide**

```
#Embed a file (with passphrase)
```

```
steghide embed -ef message.txt -cf picture.jpg -p p@sS
```

```
#Extract a file
```

```
steghide extract -sf picture.jpg -xf data.txt -p p@sS
```



Steghide (Cont)

```
(kali㉿kali)-[~]  
$ steghide embed -ef message.txt -cf image.jpg -p banana  
embedding "message.txt" in "image.jpg" ... done
```

```
(kali㉿kali)-[~]  
$ steghide extract -sf image.jpg -p banana  
wrote extracted data to "message.txt".
```

```
(kali㉿kali)-[~]  
$ █
```



Stegseek

- **Stegseek** is a lightning-fast **password cracker** for **steghide-based** steganography that can be used to extract hidden data from files
- It can **brute-force** passwords used to hide data inside **.jpg**, **.bmp**, or **.wav** files with Steghide
- How to use **Stegseek**

#Basic Usage

```
stegseek <file.jpg> <wordlist.txt>
```

#Password Cracker

```
stegseek secret.jpg rockyou.txt
```



Stegseek (Cont)

```
(kali㉿kali)-[~]  
$ stegseek image.jpg rockyou.txt  
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek
```

```
[i] Found passphrase: "banana"  
[i] Original filename: "message.txt".  
[i] Extracting to "image.jpg.out".
```

```
(kali㉿kali)-[~]  
$ █
```



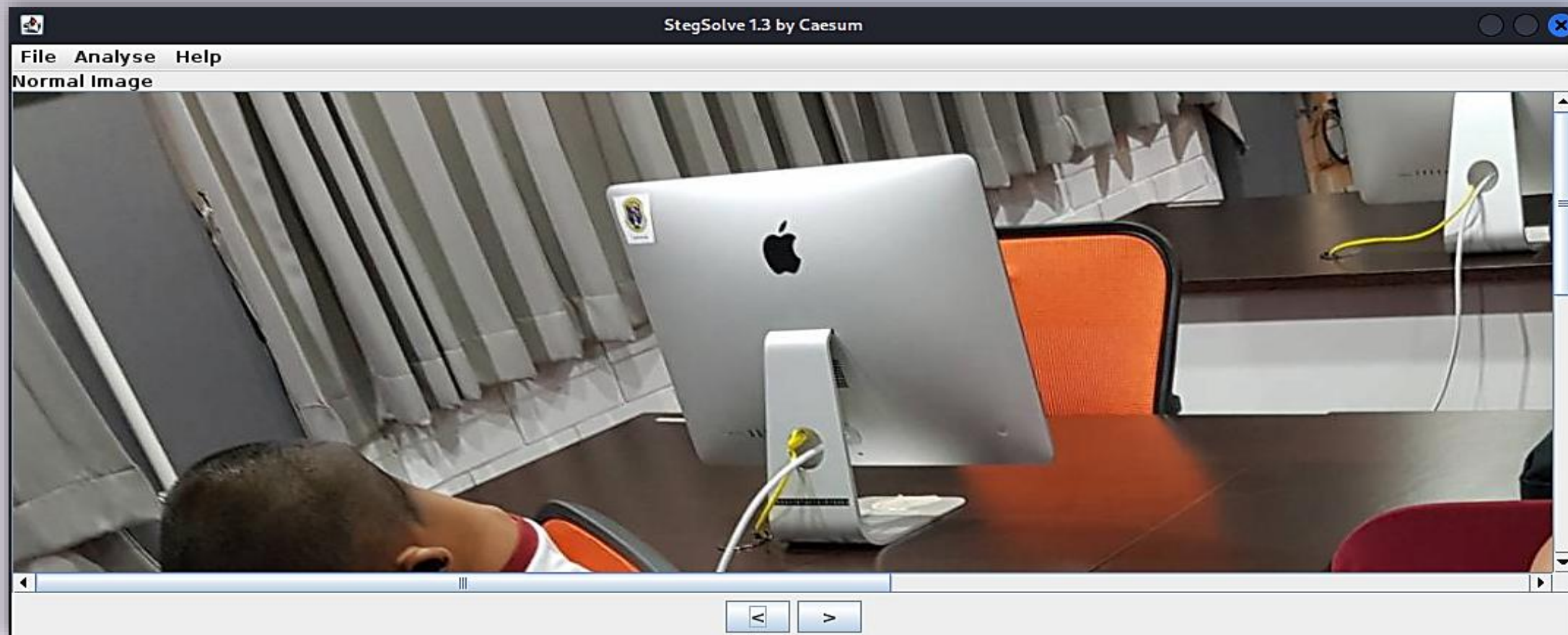
Stegsolve

- **Stegsolve** is a popular **Java-based tool** used in **steganography** analysis, especially for image files
- It provides multiple ways to manually inspect and manipulate **images** to uncover **hidden data** — a common technique in **Capture The Flag (CTF)**
- **Download** from: <http://www.caesum.com/handbook/Stegsolve.jar>
- How to use **Stegsolve**

```
wget http://www.caesum.com/handbook/Stegsolve.jar -O stegsolve.jar  
chmod +x stegsolve.jar  
java -jar stegsolve.jar
```

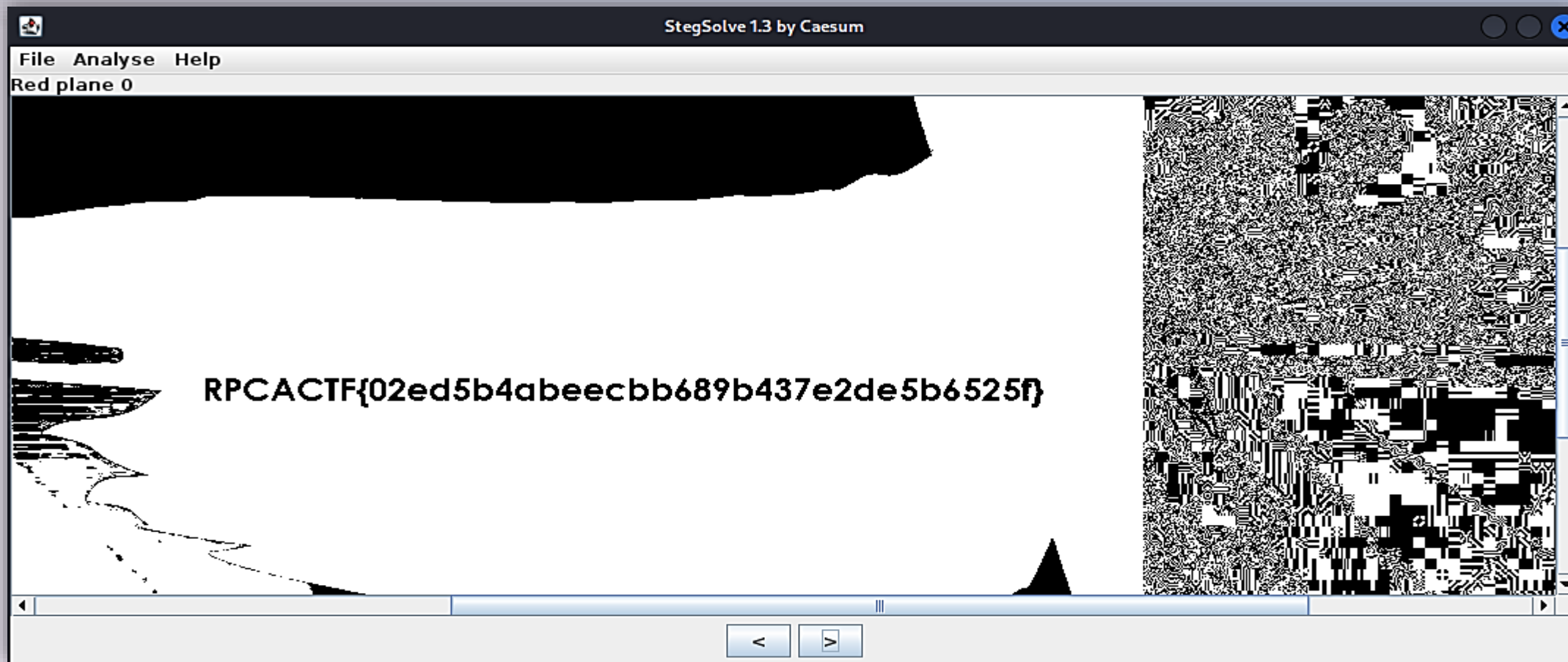


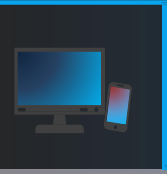
Stegsolve (Cont)





Stegsolve (Cont)





Stegdetect

- **Stegdetect** is an automated **steganography detection tool** focused primarily on analyzing **JPEG** images for the presence of hidden data
- It supports detecting several well-known stego algorithms, such as **JPHide**, **JSteg**, **OutGuess**, and **F5**
- How to use **Stegdetect**

```
git clone https://github.com/abeluck/stegdetect
cd stegdetect
linux32 ./configure
linux32 make
#Example
stegdetect image.jpg
```





StegoVeritas

- **StegoVeritas** is an automatic **image steganography analysis** tool written with **Python**
- Currently supports for **ubuntu, debian, kali, parrotOS, fedora, and archlinux**
- StegoVeritas has default actions for most **image** types (**gif, jpeg, png, tiff, bmp**) and will attempt to run on any file
- How to use **StegoVeritas**

```
git clone https://github.com/bannsec/stegoveritas.git
cd stegoveritas
pip install -r requirements.txt
python3 stegoVeritas.py -i image.png
```



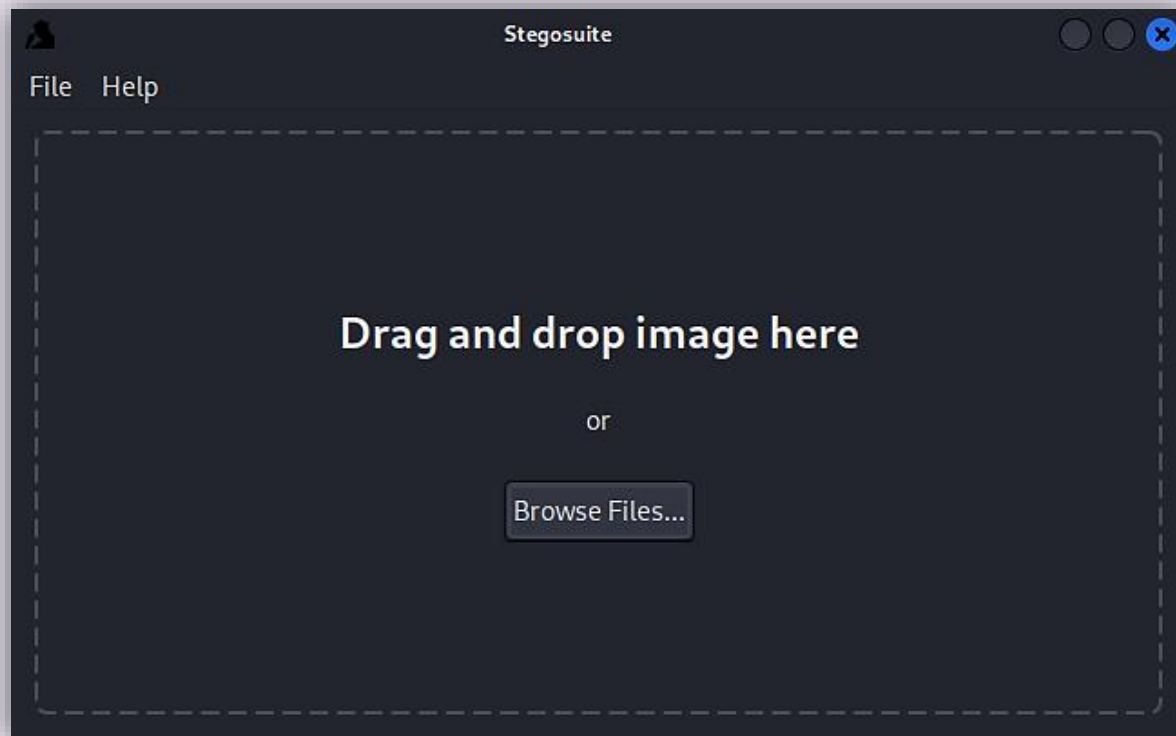
StegoSuite

- **StegoSuite** is a **free steganography tool** written in **Java** which hide the information in **image** files
- **StegoSuite** is pretty much a **GUI** for **similar steghide-type functionality**
- **StegoSuite** supports **BMP**, **GIF** and **JPG** file
- Use the **AES** encryption for embedding data
- How to use **StegoSuite**

```
sudo apt-get install stegosuite  
stegosuite gui
```



StegoSuite (Cont)





OpenStego

- **OpenStego** is a cross-platform, user-friendly **steganography tool** that supports both **data hiding** and **watermarking** in **image files**
- How to use **OpenStego**: <https://github.com/syvaidya/openstego/releases>

 openstego-0.8.6-1.noarch.rpm	185 KB	Jan 30, 2023
 openstego-0.8.6.zip	183 KB	Jan 30, 2023
 openstego_0.8.6-1_all.deb	177 KB	Jan 30, 2023
 Setup-OpenStego-0.8.6.exe	1.7 MB	Jan 30, 2023
 Source code (zip)		Jan 30, 2023
 Source code (tar.gz)		Jan 30, 2023



OpenStego (Cont)

OpenStego

File Help

Data Hiding

Hide Data

Extract Data

Digital Watermarking (Beta)

Generate Signature

Embed Watermark

Verify Watermark

Hide data in harmless looking files

Message File
C:\secret\message.txt ...

Cover File
(Select multiple files or provide wildcard (*, ?) to embed same message in multiple files)
C:\secret\nature.png ...

Output Stego File
C:\secret\send.png ...

Options

Encryption Algorithm: AES128

Password:

Confirm Password:

Hide Data

OpenStego

File Help

Data Hiding

Hide Data

Extract Data

Digital Watermarking (Beta)

Generate Signature

Embed Watermark

Verify Watermark

Verify files for existence of your watermark

File to be Checked for Watermark
(Select multiple files or provide wildcard (*, ?) to verify watermark in multiple files)
C:\final*.jpg ...

Original Signature File
C:\documents\sv_photography.sig ...

Verify Watermark

Results

The watermark strength check results are:

File Name	Strength
IMG_0127.JPG	100%
IMG_0128.JPG	89%
IMG_0129.JPG	100%
IMG_0130.JPG	100%
IMG_0131.JPG	100%

OK

<https://www.openstego.com/>



MP3Stego

- **MP3Stego** is a **steganographic tool** to hide information in **MP3 files**, that is, sound tracks compressed using the MPEG Audio Layer III format
- **MP3Stego** is written in **C** and run in **Windows** platform
- The latest version is **MP3Stego 1.1.19** (Nov. 2018)
- MP3Stego has been written with steganographic applications and can be used as a **copyright marking system** for **MP3** files
- Download **MP3Stego**: <https://www.petitcolas.net/steganography/mp3stego/>



MP3Stego (Cont)

```
C:\WINDOWS\System32\cmd.exe

Z:\Development\MP3Stego>encode -E hidden_text.txt -P pass svega.wav svega_stego.mp3
MP3StegoEncoder 1.1.15
See README file for copyright info
Microsoft RIFF, WAVE audio, PCM, mono 44100Hz 16bit, Length: 0: 0:20
MPEG-I layer III, mono Psychoacoustic Model: AT&T
Bitrate=128 kbps De-emphasis: none CRC: off
Encoding "svega.wav" to "svega_stego.mp3"
Hiding "hidden_text.txt"
[Frame 791 of 791] (100.00%) Finished in 0: 0: 6

Z:\Development\MP3Stego>decode -X -P pass svega_stego.mp3
MP3StegoEncoder 1.1.15
See README file for copyright info
Input file = 'svega_stego.mp3' output file = 'svega_stego.mp3.pcm'
Will attempt to extract hidden information. Output: svega_stego.mp3.txt
the bit stream file svega_stego.mp3 is a BINARY file
HDR: s=FFF, id=1, l=3, ep=off, br=9, sf=0, pd=1, pr=0, m=3, js=0, c=0, o=0, e=0
alg.=MPEG-1, layer=III, tot bitrate=128, sfrq=44.1
mode=single-ch, sblim=32, jsbd=32, ch=1
[Frame 791] Avg slots/frame = 417.434; b/smp = 2.90; br = 127.839 kbps
Decoding of "svega_stego.mp3" is finished
The decoded PCM output file name is "svega_stego.mp3.pcm"

Z:\Development\MP3Stego>_
```



AudioStego

- **AudioStego** is a Java code for **hiding** a secret **JPG image** inside **WAV audio** files
- How to use **AudioStego**: <https://danielcardeenass.github.io/audiostego.html>

Hiding a file

Alright, so let's set up our files before starting. First of all identify the arguments passed from the command line and.

For this explanation I am going to focus on hiding a file inside an wav file.

```
./program output.wav file.jpg
```

Note that this is not the actual project code structure, this is just simplified for explanation purposes

```
1 // Get input files into stream
2 ifstream input(inputPath, std::ios::binary );
3 if(!input.is_open())
4
5 binStremFile.open(string(argv[2]), std::ios::binary );
6     binFileSize = binStremFile.tellg();
7     msgBuffer.reserve(binFileSize); // Reserve the amount of the file size of memory to the vector
```



JSteg

- **JSteg** is one of the earliest and simplest **image steganography** algorithms, specifically designed to hide data inside **JPEG images** by modifying the **Least Significant Bits (LSB)** of discrete cosine transform (DCT) coefficients
- Download **JSteg**: <https://github.com/lukechampine/jsteg/releases>

 jsteg-darwin-amd64	2.38 MB	Aug 7, 2018
 jsteg-darwin-arm64	2.2 MB	Oct 10, 2022
 jsteg-linux-amd64	2.4 MB	Aug 7, 2018
 jsteg-linux-arm64	2.13 MB	Oct 10, 2022
 jsteg-windows-amd64.exe	2.47 MB	Aug 7, 2018
 jsteg-windows-arm64	2.26 MB	Oct 10, 2022



ZSteg

- **ZSteg** is a powerful **command-line** tool designed to analyze and detect **hidden data** in **PNG** and **BMP** image files using common **LSB** (Least Significant Bit) **steganography** techniques
- How to use **Zsteg** for extracting text: <https://github.com/zed-0xff/zsteg>

```
#Install
```

```
sudo gem install zsteg
```

```
#Extract text
```

```
zsteg image.png
```

```
#Extract special text
```

```
zsteg -E b1,r image.png > output.bin
```



ZSteg (Cont)

- Embed text for ZSteg using Python script (LSB of the red channel)

```
from PIL import Image
```

```
$ pip install pillow
```

```
def embed_text_in_image(input_image, output_image, message):  
    img = Image.open(input_image)  
    pixels = img.load()  
  
    # Convert message to binary and add a terminator  
    binary_message = ''.join(format(ord(char), '08b') for char in message) +  
    '00000000'  
  
    width, height = img.size  
    index = 0
```



ZSteg (Cont)

```
for y in range(height):
    for x in range(width):
        if index < len(binary_message):
            r, g, b = pixels[x, y]
            # Modify the LSB of the red channel
            r = (r & ~1) | int(binary_message[index])
            pixels[x, y] = (r, g, b)
            index += 1

img.save(output_image)
print(f"[+] Message embedded in {output_image}")

# Example usage
embed_text_in_image("cover.png", "stego.png", "RPCACTF{hello_zsteg}")
```



ZSteg (Cont)

- Use **stegano** Python Library

```
pip install stegano
```

```
from stegano import lsb
secret = lsb.hide("cover.png", "RPCACTF{zsteg_test}")
secret.save("stego.png")
```

However, **stegano** may store data in a way that **ZSteg cannot decode**, as it may not follow pure bit-plane **LSB patterns**



OutGuess

- **OutGuess** is a **command-line steganography tool** developed by **Niels Provos** in **1999**, it allows users to embed hidden data within the redundant bits of data sources, particularly **JPEG** and **PNM** image formats
- How to use **OutGuess**

```
#Installation
```

```
sudo apt install outguess
```

```
#Embedding a Message
```

```
outguess -k "your_passphrase" -d secret.txt cover.jpg stego.jpg
```

```
#Extracting a Message
```

```
outguess -k "your_passphrase" -r stego.jpg extracted.txt
```




OutGuess (Cont)

```
(kali@kali)-[~]  
$ outguess -k "RPCACyberClub" -d message.txt image.jpg stego.jpg
```

Reading image.jpg....

JPEG compression quality set to 75

Extracting usable bits: 312452 bits

Correctable message size: 16137 bits, 5.16%

Encoded 'message.txt': 240 bits, 30 bytes

Finding best embedding...

0:	124(45.6%)[51.7%],	bias	69(0.56),	saved:	0,	total:	0.04%
4:	124(45.6%)[51.7%],	bias	58(0.47),	saved:	0,	total:	0.04%
15:	131(48.5%)[54.6%],	bias	50(0.38),	saved:	-1,	total:	0.04%
70:	116(43.0%)[48.3%],	bias	57(0.49),	saved:	0,	total:	0.04%
200:	113(41.5%)[47.1%],	bias	53(0.47),	saved:	0,	total:	0.04%

200, 166: Embedding data: 240 in 312452

Bits embedded: 272, changed: 113(41.5%)[47.1%], bias: 53, tot: 312547, skip: 312275

Foiling statistics: corrections: 45, failed: 0, offset: 292.000000 +- 489.998639



OutGuess (Cont)

```
(kali㉿kali)-[~]  
$ outguess -k "RPCACyberClub" -r stego.jpg extracted.txt
```

Reading stego.jpg....

Extracting usable bits: 312452 bits

Steg retrieve: seed: 200, len: 30

```
(kali㉿kali)-[~]  
$ cat extracted.txt
```

Hello, We are RPCA Cyber Club

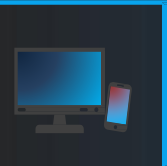
```
(kali㉿kali)-[~]  
$
```



Audacity

- **Audacity** is a powerful, **free**, and **open-source** digital **audio editor** and recording application available on **Windows**, **macOS**, and **Linux**
- While not a **steganography tool** by design, it is widely used in forensics, cybersecurity **CTFs**, and audio steganalysis due to its ability to visually inspect and manipulate audio **waveforms** and **spectrograms**





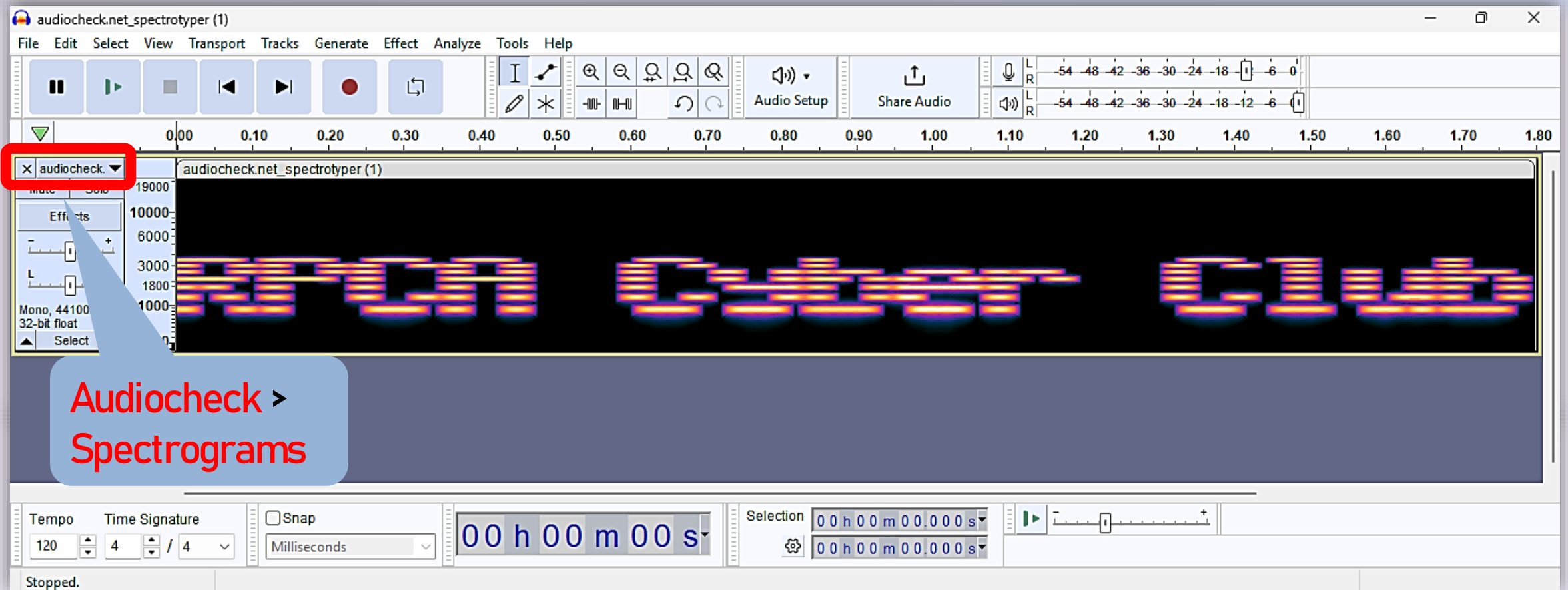
Audacity (Cont)

- **Download** Audacity: <https://www.audacityteam.org/>

The screenshot shows the Audacity website homepage. At the top, there is a navigation bar with the Audacity logo, links for Downloads, FAQ, Blog, Help, and Forum, and a 'Sign up' button for Audacity Cloud saving. Below the navigation bar is a yellow banner with the text 'Voice Cleaner: studio-quality plugin with 3 simple controls' and a '25% OFF on MuseHub' badge. The main content area has a dark blue background with the text 'Audacity is the world's most popular audio editing and recording app'. Below this text is a yellow button with a download icon and the text 'Download Audacity 3.7.3', and a link 'Download without Muse Hub'. To the right of the text is an image of a laptop displaying the Audacity software interface.



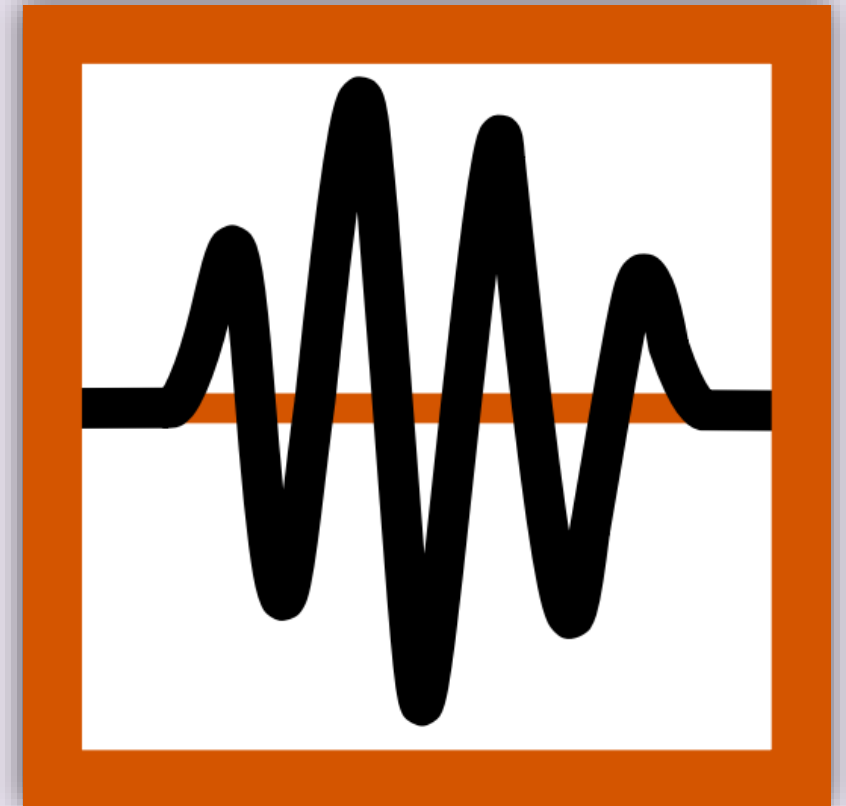
Audacity (Cont)





Sonic Visualiser

- **Sonic Visualiser** is a **free, open-source audio analysis tool** designed for visualizing audio content in high detail
- **Unlike** traditional editors like **Audacity**, it focuses on providing **advanced spectral analysis**, annotations, and time-frequency visualizations, making it a popular tool for:
 - Detecting **hidden messages** (e.g., images in **spectrograms**)
 - Performing **audio forensics**
 - Analyzing **frequency** and **pitch** in music or signals





Sonic Visualiser (Cont)

- **Download** Sonic Visualiser: <https://www.sonicvisualiser.org/>

SONIC VISUALISER

NEWS

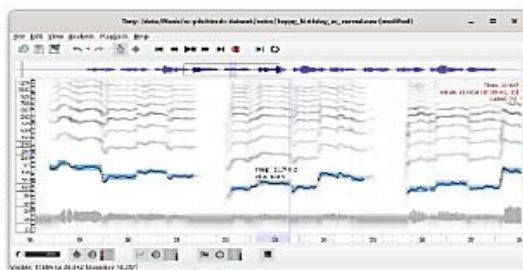
DOWNLOAD

DOCUMENTATION

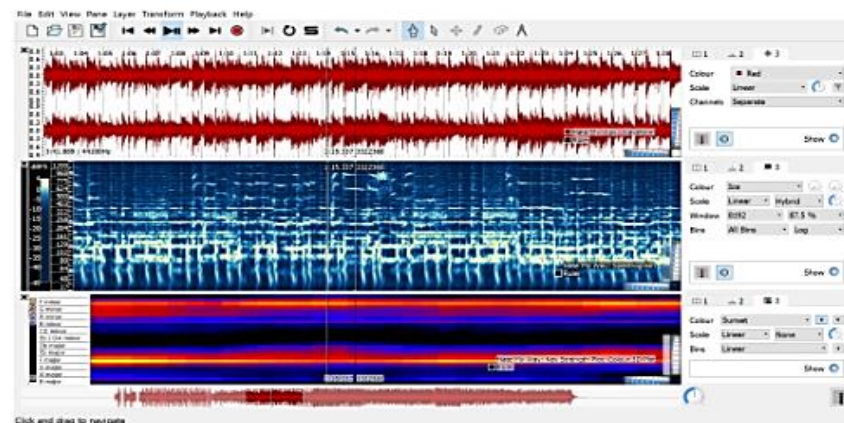
VIDEOS

PLUGINS

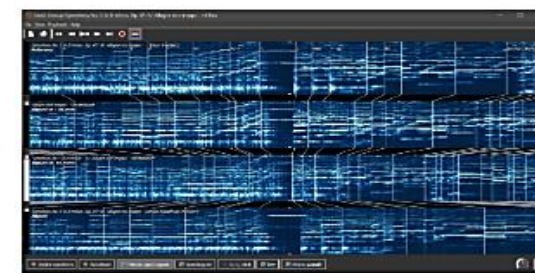
Visualisation, analysis, and annotation of music audio recordings



Tony

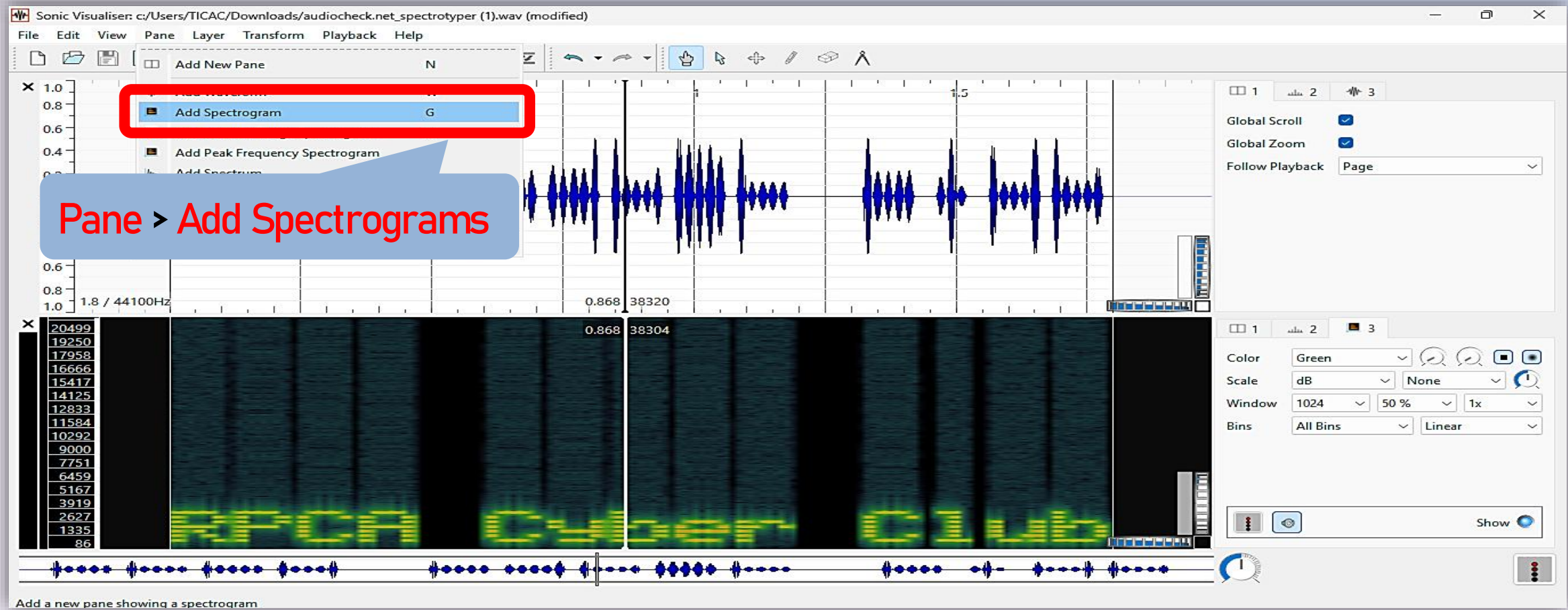


Sonic Visualiser



Sonic Lineup

Sonic Visualiser (Cont)





Online Steganography: Steg Online by Stylesuxx

- **Official web site:** <https://stylesuxx.github.io/steganography/>

Steganography Online

Encode Decode

Encode message

To encode a message into an image, choose the image you want to use, enter your text and hit the Encode button. Save the last image, it will contain your hidden message. Remember, the more text you want to hide, the larger the image has to be. In case you chose a large image, the message will be hidden in a smaller area of the image.

Neither the image nor the message you hide will be at any moment transmitted over the web.

Choose File No file chosen

Enter your message here

Encode

This tool is particularly useful for **educational purposes**, it offers a straightforward, browser-based tool for **hiding** and **revealing text** messages within images using **steganography techniques**



Online Steganography: Steg Online by Alan Eliassen

- **Official web site:** <https://futureboy.us/stegano/decinput.html>

Steganographic Decoder

This form decodes the payload that was hidden in a JPEG image or a WAV or AU audio file using the [encoder form](#). When you submit, you will be asked to save the resulting payload file to disk. This form may also help you guess at what the payload is and its file type...

Select a JPEG, WAV, or AU file to decode:

No file chosen

Password (may be blank):

☒ View raw output as MIME-type

☐ Guess the payload

☐ Prompt to save (you must guess the file type yourself.)

To use this form, you must first [encode a file](#).

These pages use the [steghide](#) program to perform steganography, and the files generated are fully compatible with steghide.

Please send comments or questions to [Alan Eliassen](#).

Stegano online is a tool to encode and decode the payload that was hidden in a **JPEG** image or a **WAV** or **AU** audio file



Online Steganography: Image Stegano by James Stanley

- **Official web site:** <https://incoherency.co.uk/image-steganography/>

Image Steganography

[How it works](#) [How to defeat it](#)

Hide images inside other images.

This is a client-side Javascript tool to steganographically hide images inside the lower "bits" of other images. Select either "Hide image" or "Unhide image". Play with the **example** images (all 200x200 px) to get a feel for it.

Hide image

Unhide image

Cover image:

Choose File

 No file chosen

Secret image:

Choose File

 No file chosen

Hidden bits: 1

Example:

N/A

Example:

N/A

Download Full-size Image

This is a **client-side Javascript tool** to steganographically hide **images** inside the lower "bits" of other **images**



Online Steganography: Stegano by Manytools

- **Official web site:** <https://manytools.org/hacker-tools/steganography-encode-text-into-image/>

Steganography (encode text into image)

Image steganography is the art of hiding messages in an image.

This is a great way to send a secret message to a friend without drawing attention to it. Compare this method to simply sending someone an encrypted piece of text. No matter how strong the encryption method is, if someone is monitoring the communication, they'll find it highly suspicious.

Look at some nice [example 'Stegafiles'](#) to get an idea!

Hide message (max ~250.000 characters + 256 KB Host-image (PNG))

OR:

Hide image (max 64 KB, 'Hide message' above ignored)

No file chosen

Host-image (max 256 KB/encoding or 384KB/decoding)

download.png

☒ Decode this image instead

manytools

Your online toolshed

Home Network HTTP, HTML & Text CSS3 Image Social Finance Hacker Handy

Hacker Tools

Hacker Steganography

ASCII banners

Ascii Art converter

ANSI Art converter

Steganography

Examples

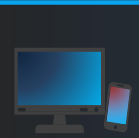
Image to Byte array

Your message

Here it is!

Image steganography is the art of **hiding messages** in an **image**

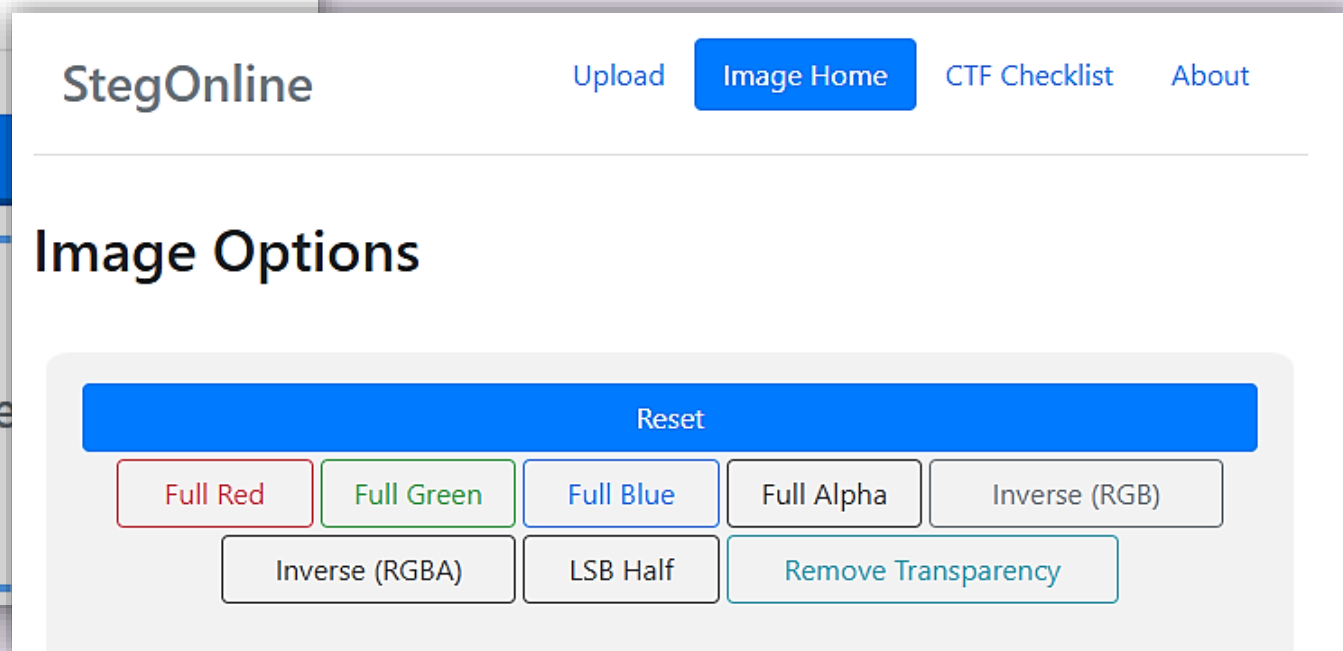
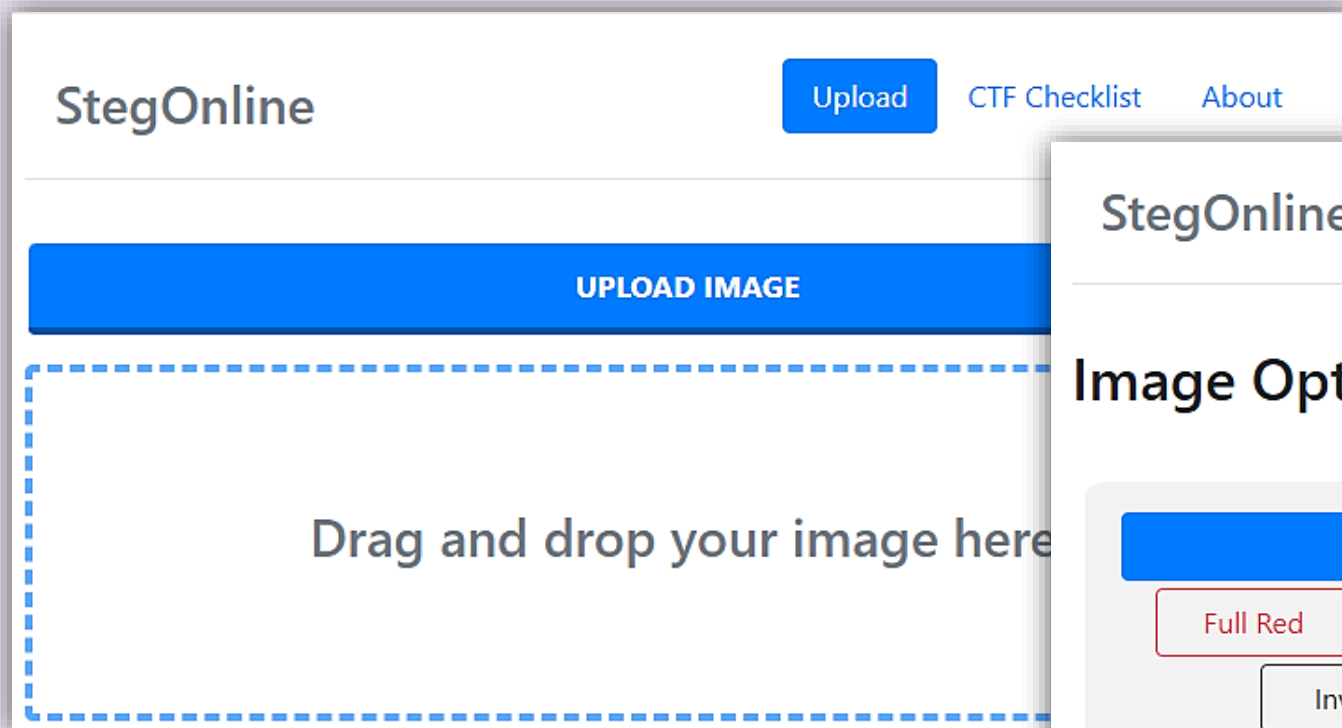
RPCA

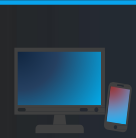


Online Steganography: StegOnline

- Official web site: <https://georgeom.net/StegOnline/>

A **web-based**, enhanced and **open-source** port of **Stegsolve**





Online Steganography: Stegify

- **Official web site:** <https://stegify-web.netlify.app/>

STEGIFY

Encode Message

Choose File No file chosen

Enter your message here

STEGIFY

Decode Image

Choose the image from which you want to decode the message.

Choose File No file chosen

Encode Decode About

Decode

Stegify is a free, browser-based steganography tool that allows users to hide and extract text messages within images



Steganography Toolkit

- **Steganography Toolkit** is a **Docker image** useful for solving **steganography** challenges, the image comes pre-installed with **many popular tools**

Steganography Toolkit

This project is a Docker image useful for solving Steganography challenges as those you can find at CTF platforms like hackthebox.eu. The image comes pre-installed with many popular tools (see list [below](#)) and several screening scripts you can use check simple things (for instance, run `check_jpg.sh image.jpg` to get a report for a JPG file).

 [Docker build status](#)



<https://github.com/DominicBreuker/stego-toolkit>

Q & A

Asst.Prof.Pol.Lt.Col.Dr. Wongyos Keardsri (Bank)

Lecturer (Level 3) | Faculty of Forensic Science | Royal Police Cadet Academy | Thailand

Email: wongyos@gmail.com | Line ID: wongyos | Phone/WhatsApp: +6689-599-3490

Facebook: facebook.com/wongyos



End